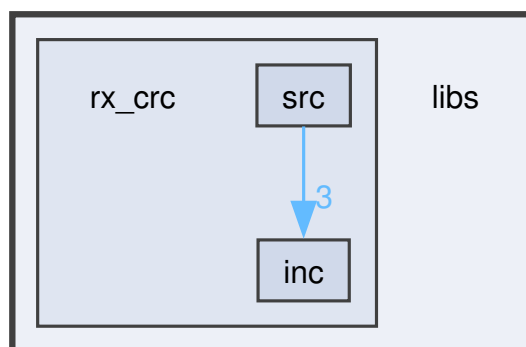
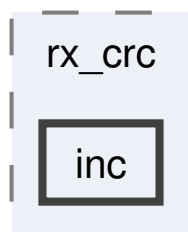
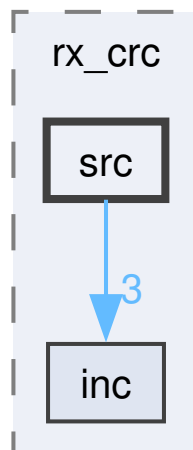
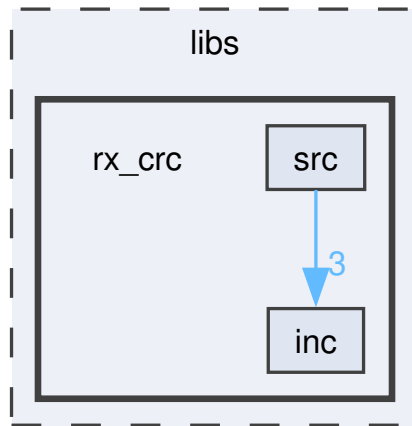
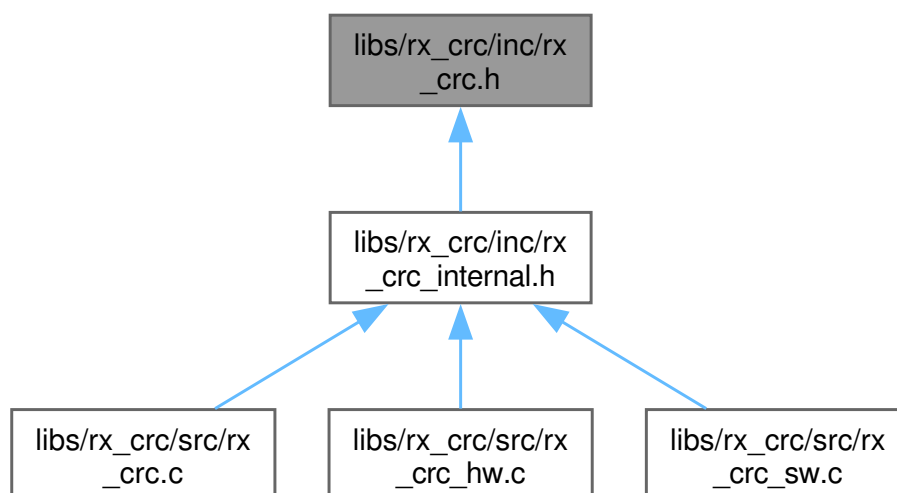
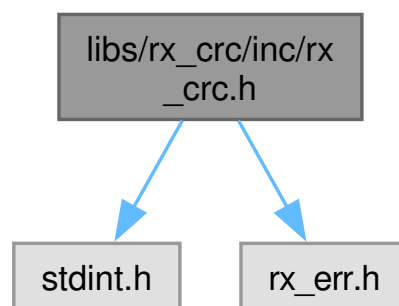





```
#include <stdint.h>
#include "rx_err.h"
```



**

*

**

	~	
	~	
	~	

~

~

~

[rx_crc32_ieee\(\)](#)~
[rx_crc8_maxim\(\)](#)~
[rx_crc_init\(\)](#)~

$$P(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

$$\begin{aligned} &= \oplus 0xFFFFFFFF = 0xFFFFFFFF \\ b : &= (\gg 8) \oplus [(\oplus b) \& 0xFF] \\ &= \oplus 0xFFFFFFFF \end{aligned}$$

<

$$P(x) = x^8 + x^5 + x^4 + 1$$

$$\begin{aligned} &= 0x00() \\ b : &= [\oplus b] \end{aligned}$$

rx_crc_init()		
rx_crc32_ieee()		
rx_crc32_update()		
rx_crc8_maxim()		
rx_crc_deinit()		

```

#include "rx_crc.h"
#include "rx_spi_comm.h"

void system_init(void)
{
    // Initialize CRC hardware (if available)
    rx_err_t err = rx_crc_init();
    if (err != k_rx_ok) {
        rx_log_warn("CRC", "Hardware CRC unavailable, using software");
    }
}

// Transmit side: Compute CRC for outgoing frame
void send_spi_frame(const uint8_t* payload, uint32_t len)
{
    // Build frame header
    uint8_t frame[279]; // Max SPI frame size
    frame[0] = 0xAA; // Sync high
    frame[1] = 0x55; // Sync low
    // ... populate sequence, length, type, flags ...

    // Copy payload
    memcpy(&frame[10], payload, len);

    // Compute CRC-32 over header + payload
    uint32_t crc_offset = 10 + len;
    uint32_t crc = 0U;
    (void)rx_crc32_ieee(frame, crc_offset, &crc);

    // Append CRC (little-endian)
    frame[crc_offset + 0] = (crc » 0) & 0xFF;
    frame[crc_offset + 1] = (crc » 8) & 0xFF;
    frame[crc_offset + 2] = (crc » 16) & 0xFF;
    frame[crc_offset + 3] = (crc » 24) & 0xFF;

    // Send via SPI
    spi_transmit(frame, crc_offset + 4);
}

// Receive side: Validate CRC of incoming frame
rx_err_t receive_spi_frame(uint8_t* frame, uint32_t frame_len)
{
    // Extract received CRC (last 4 bytes, little-endian)
    uint32_t crc_offset = frame_len - 4;
    uint32_t received_crc = (frame[crc_offset + 0] « 0) |
        (frame[crc_offset + 1] « 8) |
        (frame[crc_offset + 2] « 16) |
        (frame[crc_offset + 3] « 24);

    // Compute CRC over header + payload (exclude CRC itself)
    uint32_t calculated_crc = 0U;
    (void)rx_crc32_ieee(frame, crc_offset, &calculated_crc);

    // Validate
    if (received_crc != calculated_crc) {
        rx_log_error("SPI", "CRC mismatch: rx=0x%08X calc=0x%08X",
            received_crc, calculated_crc);
        return k_rx_err_crc_mismatch;
    }

    return k_rx_ok; // Frame valid
}

#include "rx_crc.h"
#include "rx_hcsr04.h" // OneWire interface

// Validate DS18B20 64-bit ROM code
bool validate_onewire_rom(const uint8_t* rom)
{
    // ROM format: [Family Code][48-bit Serial][CRC-8]
    // Example: 28 FF 64 1F 21 16 04 B3
    //           ^^      ^^
    //           Family (DS18B20) CRC

    // Compute CRC over first 7 bytes
    uint32_t crc_out = 0U;
    (void)rx_crc8_maxim(rom, 7, &crc_out);
    uint8_t received_crc = rom[7];

    if ((uint8_t)crc_out != received_crc) {
        rx_log_error("OneWire", "ROM CRC mismatch: rx=0x%02X calc=0x%02X",
            received_crc, (uint8_t)crc_out);
        return false;
    }

    rx_log_debug("OneWire", "ROM valid: Family=0x%02X Serial=%02X%02X%02X%02X%02X%02X",
        rom[0], rom[6], rom[5], rom[4], rom[3], rom[2], rom[1]);
    return true;
}

```

```
}

#include "rx_crc.h"

// Compute CRC-32 over multiple buffers without concatenation
uint32_t compute_multi_buffer_crc(void)
{
    // Initialize CRC: use 0U as seed -- rx_crc32_update internally un-finalizes/re-finalizes
    uint32_t crc = 0U;

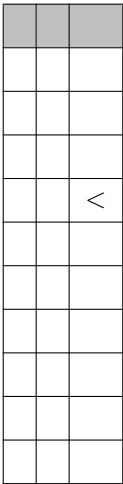
    // Process buffer 1
    uint8_t buf1[64] = { ... };
    crc = rx_crc32_update(crc, buf1, sizeof(buf1));

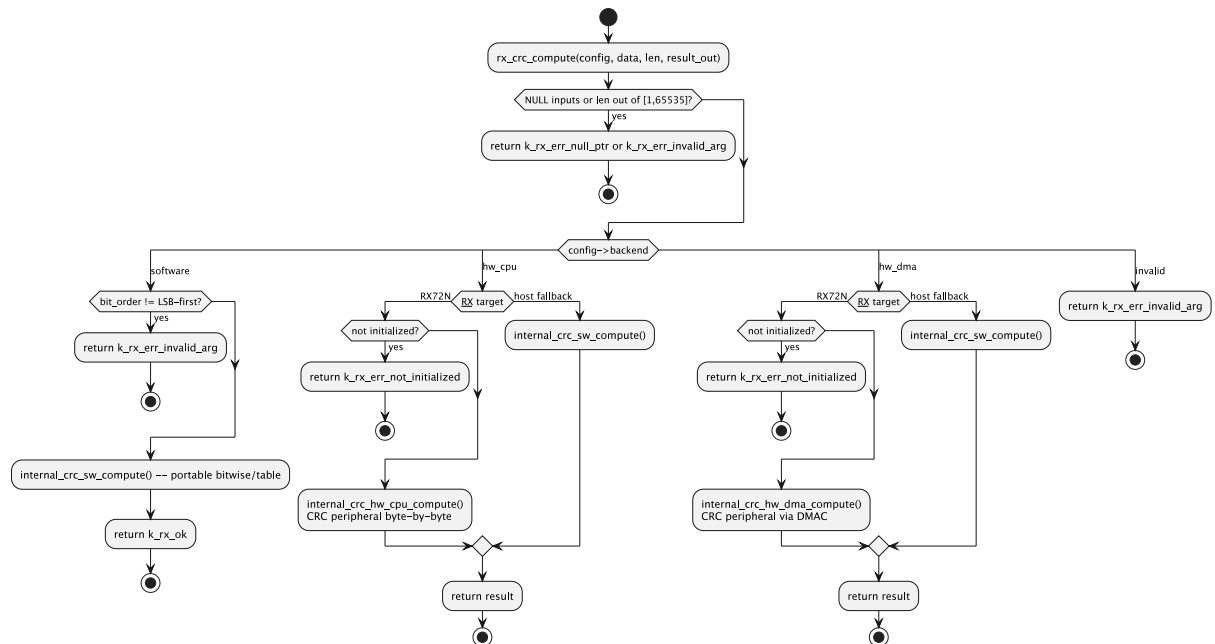
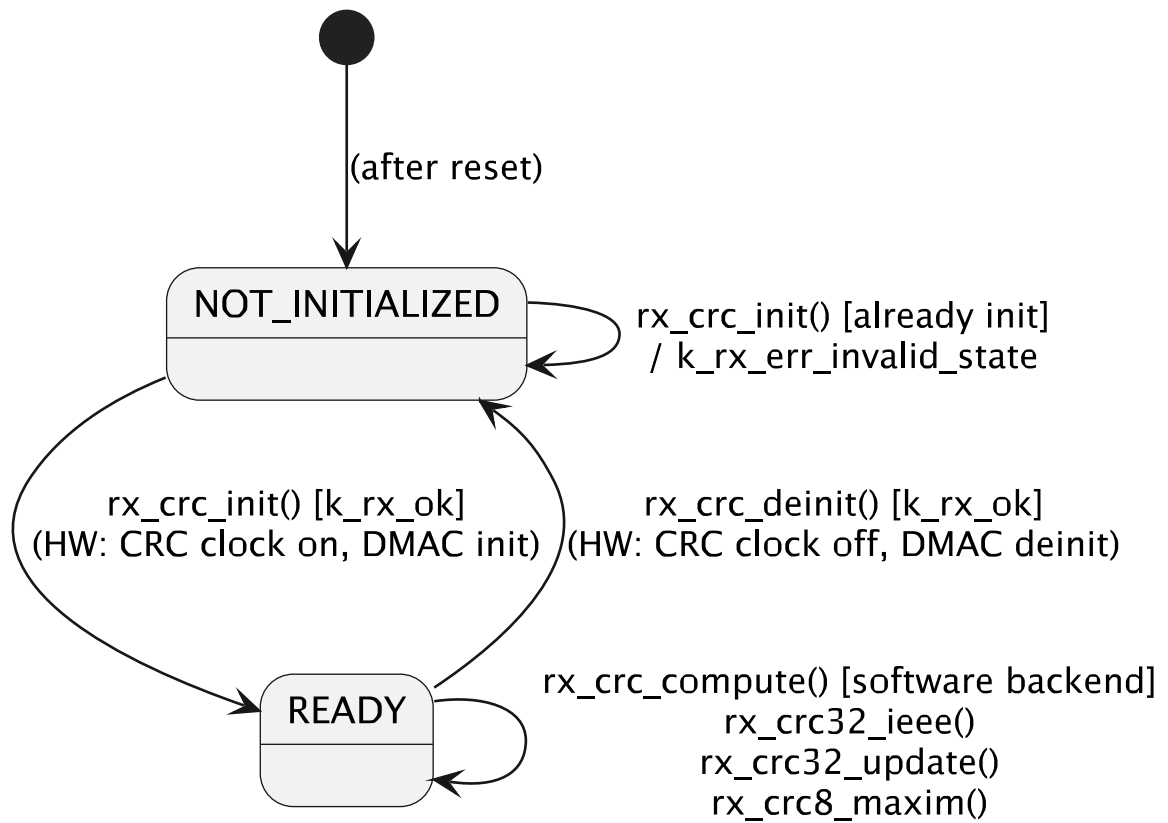
    // Process buffer 2
    uint8_t buf2[128] = { ... };
    crc = rx_crc32_update(crc, buf2, sizeof(buf2));

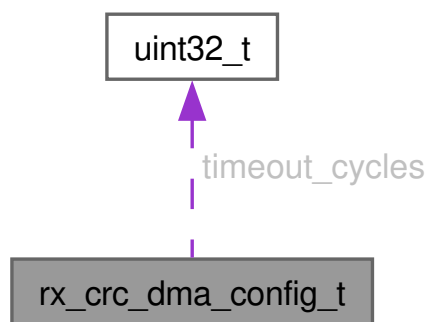
    // Process buffer 3
    uint8_t buf3[32] = { ... };
    crc = rx_crc32_update(crc, buf3, sizeof(buf3));

    return crc; // Finalized CRC-32/IEEE value (rx_crc32_update handles finalization)
}

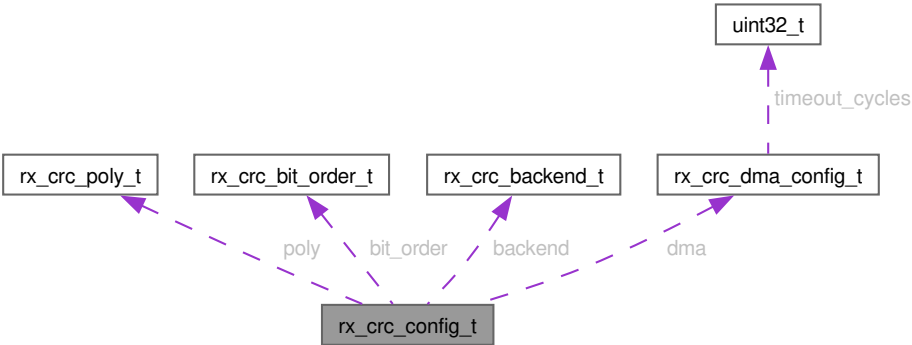
// Equivalent to single-shot computation:
// uint8_t all_data[224];
// memcpy(&all_data[0], buf1, 64);
// memcpy(&all_data[64], buf2, 128);
// memcpy(&all_data[192], buf3, 32);
// uint32_t crc = 0U;
// (void)rx_crc32_ieee(all_data, 224, &crc);
```







--	--	--




```
enum rx_crc_backend_t : uint8_t
```


```
00470         : uint8_t {
00471   k_rx_crc_backend_software = 0U, /**< Pure software (portable, all platforms) */
00472   k_rx_crc_backend_hw_cpu   = 1U, /**< HW CRC peripheral, CPU byte loop (RX72N) */
00473   k_rx_crc_backend_hw_dma   = 2U, /**< HW CRC peripheral, DMA-driven (RX72N) */
00474 } rx_crc_backend_t;
```

```
enum rx_crc_bit_order_t : uint8_t
```


```
00455         : uint8_t {
00456   k_rx_crc_bit_order_lsb_first = 0U, /**< LSB-first (reflected) - standard */
00457   k_rx_crc_bit_order_msb_first = 1U, /**< MSB-first (non-reflected) */
00458 } rx_crc_bit_order_t;
```

```
enum rx_crc_poly_t : uint8_t
```


```
00437         : uint8_t {
00438   k_rx_crc_poly_crc8      = 0x00U, /**< CRC-8/Maxim (poly 0x31 reflected) - SW only */
00439   k_rx_crc_poly_crc16     = 0x01U, /**< CRC-16/IBM (poly 0x8005 reflected) */
00440   k_rx_crc_poly_crc_ccitt = 0x02U, /**< CRC-CCITT/Kermit (poly 0x1021 reflected) */
00441   k_rx_crc_poly_crc32     = 0x03U, /**< CRC-32/IEEE 802.3 (poly 0x04C11DB7 reflected) */
00442   k_rx_crc_poly_crc32c    = 0x04U, /**< CRC-32C/Castagnoli (poly 0x1EDC6F41 reflected) */
00443 } rx_crc_poly_t;
```

```
rx_err_t rx_crc32_ieee (  
    const uint8_t * data,  
    uint32_t len,  
    uint32_t * out)  [inline], [static]
```

$$x^{32} + x^{26} + x^{23} + \dots + x + 1$$

```
crc32.ChecksumIEEE()  
zlib.crc32()
```

```
crc = 0xFFFFFFFF
```

```
    table_index = (crc ^ byte) & 0xFF  
    crc = (crc >> 8) ^ lookup_table[table_index]  
crc ^= 0xFFFFFFFF
```

in		>
in		
out		

	*
	>

*
*

>

~
~
~

```
uint8_t data[] = "Hello World";
uint32_t crc = 0U;
(void)rx_crc32_ieee(data, sizeof(data) - 1, &crc); // Exclude null terminator
printf("CRC-32: 0x%08X\n", crc);
// Output: CRC-32: 0x4A17B156
```

```
// Transmit side: Append CRC to frame
uint8_t tx_frame[279];
// ... populate header and payload ...
uint32_t crc = 0U;
(void)rx_crc32_ieee(tx_frame, 275, &crc); // CRC over first 275 bytes
```

```
// Append CRC (little-endian)
tx_frame[275] = (crc >> 0) & 0xFF;
tx_frame[276] = (crc >> 8) & 0xFF;
tx_frame[277] = (crc >> 16) & 0xFF;
tx_frame[278] = (crc >> 24) & 0xFF;
```

```
// Receive side: Validate CRC
uint8_t rx_frame[279];
// ... receive frame from SPI ...
```

```
uint32_t received_crc = (rx_frame[275] << 0) |
    (rx_frame[276] << 8) |
    (rx_frame[277] << 16) |
    (rx_frame[278] << 24);
```

```
uint32_t calculated_crc = 0U;
(void)rx_crc32_ieee(rx_frame, 275, &calculated_crc);
```

```
if (received_crc == calculated_crc) {
    // Frame valid, process data
} else {
    // Corruption detected, discard frame
    rx_log_error("CRC", "Mismatch: rx=0x%08X calc=0x%08X",
        received_crc, calculated_crc);
}
```

```
uint32_t crc = 0U;
rx_err_t err = rx_crc32_ieee(nullptr, 0, &crc);
// Returns: k_rx_err_null_ptr (nullptr is invalid)
```

```
#include "rx_time.h"
```

```
uint8_t data[1024];
memset(data, 0x55, sizeof(data));
```

```
// Without hardware CRC initialization
uint32_t start = rx_time_get_us();
uint32_t crc_sw = 0U;
(void)rx_crc32_ieee(data, sizeof(data), &crc_sw); // Software
uint32_t elapsed_sw = rx_time_get_us() - start;
rx_log_info("CRC", "Software: %u us", elapsed_sw);
// Output: Software: 1320 us
```

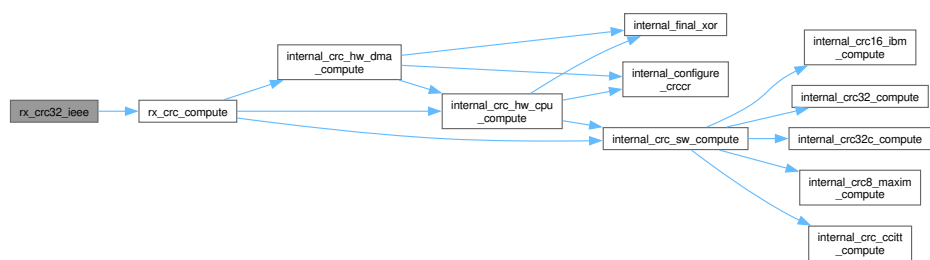
```
// With hardware CRC initialization
rx_crc_init();
start = rx_time_get_us();
uint32_t crc_hw = 0U;
(void)rx_crc32_ieee(data, sizeof(data), &crc_hw); // Hardware
uint32_t elapsed_hw = rx_time_get_us() - start;
rx_log_info("CRC", "Hardware: %u us (%.1fx faster)",
    elapsed_hw, (float)elapsed_sw / elapsed_hw);
// Output: Hardware: 320 us (4.1x faster)
```

```
// Both produce same result
assert(crc_sw == crc_hw);
```

```

00899 {
00900     static const rx_crc_config_t s_cfg = {
00901         .poly      = k_rx_crc_poly_crc32,
00902         .bit_order = k_rx_crc_bit_order_lsb_first,
00903         .backend   = k_rx_crc_backend_software, /* Software: always available, no rx_crc_init() needed */
00904         .dma       = {.timeout_cycles = 0U},
00905     };
00906     return rx_crc_compute(&s_cfg, data, len, out);
00907 }

```



```

uint32_t rx_crc32_update (
    uint32_t crc,
    const uint8_t * data,
    uint32_t len)

```

~

crc ^= 0xFFFFFFFF

```
table_index = (crc ^ byte) & 0xFF
crc = (crc >> 8) ^ lookup_table[table_index]

crc ^= 0xFFFFFFFF
```

```
// Method 1: Incremental
uint32_t crc = 0U;
(void)rx_crc32_ieee(buf1, len1, &crc);
crc = rx_crc32_update(crc, buf2, len2);

// Method 2: Concatenated (equivalent)
uint8_t combined[len1 + len2];
memcpy(&combined[0], buf1, len1);
memcpy(&combined[len1], buf2, len2);
uint32_t crc = 0U;
(void)rx_crc32_ieee(combined, len1 + len2, &crc);

// Both methods produce identical CRC
```

in		
in		
in		^

<<
>

~
~

```
// Compute CRC over two buffers without concatenation
uint8_t header[10] = { 0xAA, 0x55, ... };
uint8_t payload[64] = { ... };
```

```
// Start with header
uint32_t crc = 0U;
(void)rx_crc32_ieee(header, sizeof(header), &crc);
```

```
// Update with payload
crc = rx_crc32_update(crc, payload, sizeof(payload));
```

```
// Final CRC is over header + payload (74 bytes total)
printf("CRC-32: 0x%08X\n", crc);
```

```
// Compute CRC as data arrives in chunks
uint32_t crc = 0U; // Correct seed: rx_crc32_update() un-finalizes 0 to 0xFFFFFFFF internally
```

```
// Process chunk 1
uint8_t chunk1[32];
spi_receive(chunk1, sizeof(chunk1));
crc = rx_crc32_update(crc, chunk1, sizeof(chunk1));
```

```
// Process chunk 2
uint8_t chunk2[64];
spi_receive(chunk2, sizeof(chunk2));
crc = rx_crc32_update(crc, chunk2, sizeof(chunk2));
```

```
// Process chunk 3
uint8_t chunk3[16];
spi_receive(chunk3, sizeof(chunk3));
crc = rx_crc32_update(crc, chunk3, sizeof(chunk3));
```

```
// Final CRC is over all chunks (112 bytes total)
printf("Streaming CRC-32: 0x%08X\n", crc);
```

```

// SPI frame structure
typedef struct {
    uint16_t sync;    // 0xAA55
    uint16_t seq;     // Sequence number
    uint8_t len;      // Payload length
    uint8_t type;     // Message type
    uint8_t flags;    // Control flags
    uint8_t reserved; // Reserved
} spi_header_t;

spi_header_t header;
uint8_t payload[255];

// ... populate header and payload ...

// Compute CRC over header + payload
uint32_t crc = 0U;
(void)rx_crc32_ieee((uint8_t*)&header, sizeof(header), &crc);
crc = rx_crc32_update(crc, payload, header.len);

// Append CRC to frame
uint8_t crc_bytes[4] = {
    (crc » 0) & 0xFF,
    (crc » 8) & 0xFF,
    (crc » 16) & 0xFF,
    (crc » 24) & 0xFF,
};
spi_transmit(crc_bytes, sizeof(crc_bytes));

// Compute CRC over 3 buffers without allocating concatenation buffer
uint8_t buf1[64]; // 64 bytes
uint8_t buf2[128]; // 128 bytes
uint8_t buf3[32]; // 32 bytes

// Method 1: Incremental (no extra memory)
uint32_t crc = 0U;
(void)rx_crc32_ieee(buf1, sizeof(buf1), &crc);
crc = rx_crc32_update(crc, buf2, sizeof(buf2));
crc = rx_crc32_update(crc, buf3, sizeof(buf3));

// Method 2: Concatenated (requires 224-byte buffer)
// uint8_t combined[224];
// memcpy(&combined[0], buf1, 64);
// memcpy(&combined[64], buf2, 128);
// memcpy(&combined[192], buf3, 32);
// uint32_t crc = 0U;
// (void)rx_crc32_ieee(combined, 224, &crc);

// Both methods produce identical CRC, but Method 1 saves 224 bytes

// [FAIL] WRONG: Don't use 0xFFFFFFFF as seed for rx_crc32_update()
// rx_crc32_update() un-finalizes by XOR with 0xFFFFFFFF internally.
// Seeding with 0xFFFFFFFF produces work=0x00000000 (wrong initial state).
uint32_t crc = 0xFFFFFFFF; // Wrong: un-finalizes to 0, not 0xFFFFFFFF
crc = rx_crc32_update(crc, buf1, len1); // [FAIL] Incorrect CRC!

// [PASS] CORRECT: Use 0U as seed for first rx_crc32_update() call
// rx_crc32_update(0, ...) un-finalizes to 0xFFFFFFFF (correct initial state).
uint32_t crc = 0U;
crc = rx_crc32_update(crc, buf1, len1); // [PASS] Correct
crc = rx_crc32_update(crc, buf2, len2); // [PASS] Continues correctly

// OR: Start with rx_crc32_ieee() on first buffer (also correct)
uint32_t crc = 0U;
(void)rx_crc32_ieee(buf1, len1, &crc);
crc = rx_crc32_update(crc, buf2, len2); // [PASS] Correct

```

>

```
00257 {  
00258     if (data == nullptr || len == 0U) {  
00259         return crc;  
00260     }  
00261  
00262     return internal_crc32_sw_update(crc, data, len);  
00263 }
```



```
rx_err_t rx_crc8_maxim (  
    const uint8_t * data,  
    uint32_t len,  
    uint32_t * out)  [inline], [static]
```

$$x^8 + x^5 + x^4 + 1$$

```
crc = 0x00
```

```
table_index = crc ^ byte  
crc = lookup_table[table_index]
```

$$P(x) = x^8 + x^5 + x^4 + 1$$

$$\begin{aligned} &= 0x00() \\ b : &= [\oplus b] \end{aligned}$$

<

in		>
in		
out		

	*
	>

*

*

~
~

```

// DS18B20 ROM code format: [Family][Serial 6 bytes][CRC-8]
uint8_t rom[8] = { 0x28, 0xFF, 0x64, 0x1F, 0x21, 0x16, 0x04, 0xB3 };
//
//          Family (DS18B20)          CRC

// Compute CRC over first 7 bytes
uint32_t crc_out = 0U;
(void)rx_crc8_maxim(rom, 7, &crc_out);
uint8_t calculated_crc = (uint8_t)crc_out;
uint8_t received_crc = rom[7];

if (calculated_crc == received_crc) {
    printf("ROM valid: Family=0x%02X CRC=0x%02X\n", rom[0], rom[7]);
    // Output: ROM valid: Family=0x28 CRC=0xB3
} else {
    printf("ROM invalid: expected=0x%02X actual=0x%02X\n",
        calculated_crc, received_crc);
}

// DS18B20 scratchpad: 9 bytes total (8 data + 1 CRC)
uint8_t scratchpad[9];
rx_owewire_read_scratchpad(scratchpad, sizeof(scratchpad));

// Compute CRC over first 8 bytes
uint32_t crc_out = 0U;
(void)rx_crc8_maxim(scratchpad, 8, &crc_out);
uint8_t calculated_crc = (uint8_t)crc_out;
uint8_t received_crc = scratchpad[8];

if (calculated_crc == received_crc) {
    // Extract temperature (bytes 0-1, little-endian)
    int16_t raw_temp = (scratchpad[1] « 8) | scratchpad[0];
    float temp_c = raw_temp / 16.0f; // 12-bit resolution
    printf("Temperature: %.2f degC\n", temp_c);
} else {
    rx_log_error("OneWire", "Scratchpad CRC mismatch");
}

// Send custom command to OneWire device with CRC
void send_owewire_command(uint8_t cmd, uint8_t param)
{
    uint8_t buffer[3];
    buffer[0] = cmd;
    buffer[1] = param;

    // Compute CRC over command + parameter
    uint32_t crc_out = 0U;
    (void)rx_crc8_maxim(buffer, 2, &crc_out);
    buffer[2] = (uint8_t)crc_out;

    // Transmit with CRC
    rx_owewire_write(buffer, sizeof(buffer));
}

// Validate multiple DS18B20 ROM codes
typedef struct {
    uint8_t rom[8];
    bool valid;
} onewire_device_t;

onewire_device_t devices[4];

// Search for devices and validate ROM codes
for (uint8_t i = 0; i < 4; i++) {
    if (rx_owewire_search(&devices[i].rom[0])) {
        // Validate ROM CRC
        uint32_t crc_out = 0U;
        (void)rx_crc8_maxim(devices[i].rom, 7, &crc_out);
        devices[i].valid = ((uint8_t)crc_out == devices[i].rom[7]);

        if (devices[i].valid) {
            rx_log_info("OneWire", "Device %u: Family=0x%02X Serial=%02X%02X%02X%02X%02X%02X",
                i, devices[i].rom[0],

```

```

        devices[i].rom[6], devices[i].rom[5], devices[i].rom[4],
        devices[i].rom[3], devices[i].rom[2], devices[i].rom[1]);
    } else {
        rx_log_error("OneWire", "Device %u: Invalid ROM CRC", i);
    }
}
}

```

```

uint32_t crc_out = 0U;
rx_err_t err = rx_crc8_maxim(nullptr, 0, &crc_out);
// Returns: k_rx_err_null_ptr (data is NULL)
(void)err;

```

```

// Demonstrate CRC-8/Maxim algorithm step-by-step
uint8_t data[] = { 0x28, 0xFF, 0x64, 0x1F, 0x21, 0x16, 0x04 };

```

```

// Method 1: Use library function

```

```

uint32_t crc_lib_out = 0U;
(void)rx_crc8_maxim(data, sizeof(data), &crc_lib_out);
uint8_t crc_lib = (uint8_t)crc_lib_out;
printf("Library CRC: 0x%02X\n", crc_lib);
// Output: Library CRC: 0xB3

```

```

// Method 2: Manual bitwise computation (for algorithm understanding)

```

```

uint8_t crc_manual = 0x00; // CRC-8/Maxim initial value
for (uint8_t i = 0; i < sizeof(data); i++) {
    crc_manual ^= data[i];
    for (uint8_t bit = 0; bit < 8; bit++) {
        if (crc_manual & 0x01) {
            crc_manual = (uint8_t)((crc_manual » 1) ^ 0x8C); // reflected poly 0x31
        } else {
            crc_manual »= 1;
        }
    }
}
printf("Manual CRC: 0x%02X\n", crc_manual);
// Output: Manual CRC: 0xB3

```

```

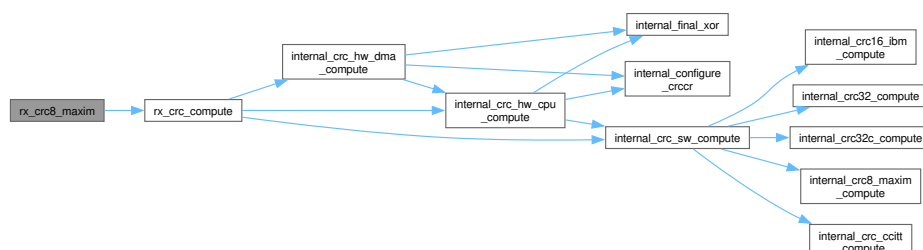
// Both produce identical result
assert(crc_lib == crc_manual);

```

```

01334 {
01335     static const rx_crc_config_t s_cfg = {
01336         .poly      = k_rx_crc_poly_crc8,
01337         .bit_order = k_rx_crc_bit_order_lsb_first,
01338         .backend    = k_rx_crc_backend_software,
01339         .dma        = { .timeout_cycles = 0U },
01340     };
01341     return rx_crc_compute(&s_cfg, data, len, out);
01342 }

```



```
rx_err_t rx_crc_compute (  
    const rx_crc_config_t * config,  
    const uint8_t * data,  
    uint32_t len,  
    uint32_t * result_out)  [nodiscard]
```

in		
in		
in		
out		

	*
	>

*
*

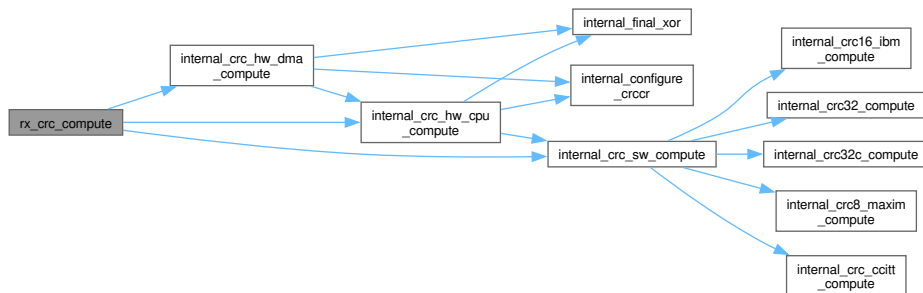
*

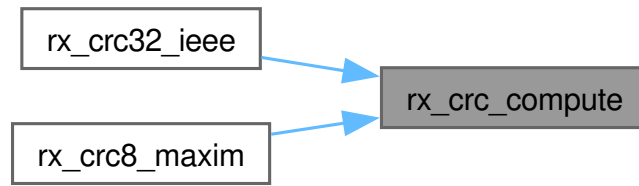


```

00185 {
00186     if (config == nullptr || data == nullptr || result_out == nullptr) {
00187         return k_rx_err_null_ptr;
00188     }
00189
00190     if (len < k_crc_len_min || len > k_crc_len_max) {
00191         return k_rx_err_invalid_arg;
00192     }
00193
00194     if (config->poly > k_rx_crc_poly_crc32c || config->bit_order > k_rx_crc_bit_order_msb_first) {
00195         return k_rx_err_invalid_arg;
00196     }
00197
00198     switch (config->backend) {
00199         case k_rx_crc_backend_software:
00200             if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00201                 return k_rx_err_invalid_arg;
00202             }
00203             *result_out = internal_crc_sw_compute(config->poly, data, len);
00204             return k_rx_ok;
00205
00206 #ifdef __RX__
00207         case k_rx_crc_backend_hw_cpu:
00208             if (!s_crc_initialized) {
00209                 return k_rx_err_not_initialized;
00210             }
00211             return internal_crc_hw_cpu_compute(config, data, len, result_out);
00212
00213         case k_rx_crc_backend_hw_dma:
00214             if (!s_crc_initialized) {
00215                 return k_rx_err_not_initialized;
00216             }
00217             return internal_crc_hw_dma_compute(config, data, len, result_out);
00218 #else
00219         /* Host build: hardware backends not available - fall back to software */
00220         case k_rx_crc_backend_hw_cpu:
00221             [[fallthrough]]; /* hw_cpu has same software fallback as hw_dma on host */
00222         case k_rx_crc_backend_hw_dma:
00223             if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00224                 return k_rx_err_invalid_arg;
00225             }
00226             *result_out = internal_crc_sw_compute(config->poly, data, len);
00227             return k_rx_ok;
00228 #endif
00229
00230         default:
00231             return k_rx_err_invalid_arg;
00232     }
00233 }

```





```
rx_err_t rx_crc_deinit (
    void ) [nodiscard]
```

```
__RX__RX_CRC32_USE_HARDWARE
```



~
<

```
void system_shutdown(void)
{
    // Deinitialize CRC (optional, for power savings)
    rx_err_t err = rx_crc_deinit();
    if (err == k_rx_ok) {
        rx_log_info("CRC", "CRC deinitialized (power saving)");
    } else {
        rx_log_warn("CRC", "CRC deinit failed: %d", err);
    }

    // Other shutdown tasks...
    rx_gpio_deinit();
    rx_clock_deinit();
}
```

```
void enter_low_power_mode(void)
{
    // Disable CRC to save power
    rx_crc_deinit();

    // CRC still works (software fallback)
    uint32_t crc = 0U;
    (void)rx_crc32_ieee(data, len, &crc); // Slower, but functional

    // Enter low-power sleep
    rx_power_enter_sleep();
}

void exit_low_power_mode(void)
{
    // Re-enable CRC for performance
    rx_crc_init();

    // CRC now fast again (hardware)
    uint32_t crc = 0U;
    (void)rx_crc32_ieee(data, len, &crc); // Fast path
}
```

```
00092 {
00093     if (!s_crc_initialized) {
00094         return k_rx_err_invalid_state;
00095     }
00096
00097     #ifdef __RX__
00098     rx_err_t err = internal_crc_hw_deinit();
00099     if (err != k_rx_ok) {
00100         return err;
00101     }
00102     #endif
00103
00104     s_crc_initialized = false;
00105     return k_rx_ok;
00106 }
```



```
rx_err_t rx_crc_init (
    void ) [nodiscard]
```

```
__RX__RX_CRC32_USE_HARDWARE
```

```
void system_init(void)
{
    // Initialize clocks first
    rx_clock_init();

    // Initialize CRC (enables hardware if available)
    rx_err_t err = rx_crc_init();
    if (err == k_rx_ok) {
        rx_log_info("CRC", "CRC initialized (HW accelerated)");
    } else {
        rx_log_warn("CRC", "CRC init failed: %d (SW fallback)", err);
    }

    // Now CRC functions can use hardware acceleration
    uint32_t crc = 0U;
    (void)rx_crc32_ieee(data, len, &crc); // Fast path
}
```

```
#ifdef __RX__
// RX72N: Try to enable hardware CRC
if (rx_crc_init() == k_rx_ok) {
    rx_log_info("CRC", "Using hardware CRC (4x faster)");
} else {
    rx_log_warn("CRC", "Using software CRC");
}
#else
// Host: Software CRC always available
rx_crc_init(); // No-op, but safe to call
rx_log_info("CRC", "Using software CRC (portable)");
#endif
```

```
00058 {
00059     if (s_crc_initialized) {
00060         return k_rx_err_invalid_state;
00061     }
00062
00063     #ifdef __RX__
00064         rx_err_t err = internal_crc_hw_init();
00065         if (err != k_rx_ok) {
00066             return err;
00067         }
00068     #endif
00069     s_crc_initialized = true;
00070     return k_rx_ok;
00071 }
00072 }
```

rx_crc_init



internal_crc_hw_init

```

00001 /**
00002  * @file rx_crc.h
00003  * @brief Cyclic Redundancy Check (CRC) API for Data Integrity Validation
00004  *
00005  * @details
00006  * ## Overview
00007  * Production-quality CRC (Cyclic Redundancy Check) implementation providing both
00008  * hardware-accelerated and software-based checksumming for data integrity validation
00009  * across all STAR firmware communication protocols and storage subsystems.
00010  *
00011  * Supports two CRC algorithms:
00012  * - **CRC-32/IEEE 802.3** - For frame protocols, SPI, USB, file integrity
00013  * - **CRC-8/Maxim** - For OneWire (DS18B20) and Dallas/Maxim devices
00014  *
00015  * ## CRC Algorithm Comparison
00016  *
00017  * | Feature | CRC-32/IEEE 802.3 | CRC-8/Maxim |
00018  * |-----|-----|-----|
00019  * | **Polynomial** | 0x04C11DB7 (reflected: 0xEDB88320) | 0x31 (reflected: 0x8C) |
00020  * | **Output size** | 32 bits | 8 bits |
00021  * | **Initial value** | 0xFFFFFFFF | 0x00 |
00022  * | **Final XOR** | 0xFFFFFFFF | 0x00 |
00023  * | **Bit order** | LSB-first (reflected) | LSB-first |
00024  * | **Detection** | 4.3 billion errors | 256 errors |
00025  * | **Usage** | Ethernet, ZIP, PNG, SPI frames | OneWire devices, ROM codes |
00026  * | **Compatibility** | Go crc32.ChecksumIEEE() | DS18B20, DS2482 |
00027  * | **Performance (SW)** | ~130 us/100 bytes @ 240 MHz | ~15 us/8 bytes @ 240 MHz |
00028  * | **Performance (HW)** | ~32 us/100 bytes @ 240 MHz | N/A (SW only) |
00029  *
00030  * ## Hardware vs Software Implementation
00031  *
00032  * **Hardware CRC (RX72N only):**
00033  * - Uses RX72N CRC peripheral (32-bit, polynomial 0x04C11DB7)
00034  * - **4x faster** than software for CRC-32 (32 us vs 130 us per 100 bytes)
00035  * - Requires initialization via rx_crc_init()
00036  * - Zero CPU overhead during computation
00037  * - Limited to CRC-32/IEEE 802.3 polynomial
00038  *
00039  * **Software CRC (Host and RX72N):**
00040  * - Table-based lookup for fast software computation
00041  * - Works on host (x86/ARM) and embedded (RX72N)
00042  * - No initialization required
00043  * - Supports both CRC-32 and CRC-8 algorithms
00044  * - Portable across all platforms
00045  *
00046  * **Automatic Selection:**
00047  * ```c
00048  * // Compile-time selection based on RX_CRC32_USE_HARDWARE define
00049  * #ifdef RX_CRC32_USE_HARDWARE
00050  * // Use hardware CRC peripheral (RX72N)
00051  * #else
00052  * // Use software table-based CRC (portable)
00053  * #endif
00054  * ```
00055  *
00056  * ## Performance Characteristics
00057  *
00058  * **CRC-32/IEEE 802.3** (@ 240 MHz RX72N):
00059  *
00060  * | Data Size | Hardware CRC | Software CRC | Speedup |
00061  * |-----|-----|-----|-----|
00062  * | 8 bytes | 5 us | 10 us | 2.0x |
00063  * | 64 bytes | 20 us | 85 us | 4.25x |
00064  * | 100 bytes | 32 us | 130 us | 4.06x |
00065  * | 255 bytes | 80 us | 330 us | 4.13x |
00066  * | 1024 bytes | 320 us | 1320 us | 4.13x |
00067  *
00068  * **CRC-8/Maxim** (Software only, @ 240 MHz):
00069  *
00070  * | Data Size | Time | Notes |
00071  * |-----|-----|-----|
00072  * | 8 bytes | ~15 us | Typical OneWire ROM code |
00073  * | 9 bytes | ~17 us | ROM + CRC byte |
00074  * | 64 bytes | ~120 us | Scratchpad data |
00075  *
00076  * **Throughput Analysis:**
00077  * - **Hardware CRC-32**: ~3.2 MB/s (1024 bytes / 320 us)
00078  * - **Software CRC-32**: ~0.78 MB/s (1024 bytes / 1320 us)
00079  * - **Software CRC-8**: ~0.47 MB/s (64 bytes / 120 us, limited use case)
00080  *
00081  * ## Memory Usage
00082  *
00083  * **Static Memory:**
00084  * - **CRC-32 lookup table**: 1024 bytes (256 entries x 4 bytes)
00085  * - **CRC-8 lookup table**: 256 bytes (256 entries x 1 byte)
00086  * - **Total**: 1280 bytes (shared globally, one-time cost)

```

```

00087 *
00088 * **Stack Usage:**
00089 * - `rx_crc32_ieee()`: ~16 bytes (loop counter, temp vars)
00090 * - `rx_crc8_maxim()`: ~12 bytes (loop counter, temp vars)
00091 * - `rx_crc_init()`: ~8 bytes (minimal)
00092 *
00093 * ## Hardware Requirements
00094 *
00095 * **For Hardware CRC (RX72N only):**
00096 * - **Peripheral**: CRC Calculator (CRCA)
00097 * - **Clock**: PCLKA (120 MHz typical)
00098 * - **Register base**: 0x0008C000
00099 * - **Power domain**: Module stop control register (MSTPCRB bit 23)
00100 * - **Initialization**: Must call rx_crc_init() before use
00101 *
00102 * **For Software CRC (All platforms):**
00103 * - No hardware dependencies
00104 * - Works on RX72N, x86, ARM, etc.
00105 * - No initialization required
00106 *
00107 * ## CRC-32/IEEE 802.3 Mathematical Definition
00108 *
00109 * **Polynomial:**
00110 * @f[
00111 *  $P(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} +$ 
00112 *  $x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ 
00113 * @f]
00114 *
00115 * **Binary representation:** 0x04C11DB7 (normal), 0xEDB88320 (reflected)
00116 *
00117 * **Algorithm (reflected, LSB-first):**
00118 * @f[
00119 *  $\text{\texttt{\text{CRC}}} = \text{\texttt{\text{CRC}}\_} \text{\texttt{\text{init}}} \oplus 0xFFFFFFFF = 0xFFFFFFFF$ 
00120 * @f]
00121 * @f[
00122 *  $\text{\texttt{\text{For each byte } b:}} \text{\texttt{\text{CRC}}} = (\text{\texttt{\text{CRC}}} \gg 8) \oplus$ 
00123 *  $\text{\texttt{\text{Table}}}[(\text{\texttt{\text{CRC}}} \oplus b) \& 0xFF]$ 
00124 * @f]
00125 * @f[
00126 *  $\text{\texttt{\text{CRC}}\_} \text{\texttt{\text{final}}} = \text{\texttt{\text{CRC}}} \oplus 0xFFFFFFFF$ 
00127 * @f]
00128 *
00129 * **Properties:**
00130 * - Detects all single-bit errors
00131 * - Detects all double-bit errors
00132 * - Detects all burst errors  $\leq 32$  bits
00133 * - Detects 99.9999% of all other errors
00134 *
00135 * ## CRC-8/Maxim Mathematical Definition
00136 *
00137 * **Polynomial:**
00138 * @f[
00139 *  $P(x) = x^8 + x^5 + x^4 + 1$ 
00140 * @f]
00141 *
00142 * **Binary representation:** 0x31 (normal), 0x8C (reflected)
00143 *
00144 * **Algorithm (LSB-first):**
00145 * @f[
00146 *  $\text{\texttt{\text{CRC}}} = 0x00 \text{\texttt{\text{quad}}} (\text{\texttt{\text{initial value}}})$ 
00147 * @f]
00148 * @f[
00149 *  $\text{\texttt{\text{For each byte } b:}} \text{\texttt{\text{CRC}}} = \text{\texttt{\text{Table}}}[\text{\texttt{\text{CRC}}} \oplus b]$ 
00150 * @f]
00151 *
00152 * **Properties:**
00153 * - Detects all single-bit errors
00154 * - Detects all 2-bit errors within 8 bits
00155 * - Used by Dallas/Maxim OneWire devices (DS18B20, DS2482, etc.)
00156 *
00157 * ## Thread Safety
00158 *
00159 * | Function | Thread Safe? | Notes |
00160 * |-----|-----|-----|
00161 * | `rx_crc_init()` | [FAIL] No | Call once during system init, single-threaded |
00162 * | `rx_crc32_ieee()` | [PASS] Yes | Stateless, read-only lookup table |
00163 * | `rx_crc32_update()` | [PASS] Yes | Stateless, pure function |
00164 * | `rx_crc8_maxim()` | [PASS] Yes | Stateless, read-only lookup table |
00165 * | `rx_crc_deinit()` | [FAIL] No | Call once during shutdown, single-threaded |
00166 *
00167 * **Note:** CRC computation functions are thread-safe because they use read-only
00168 * lookup tables and have no shared mutable state.
00169 *
00170 * ## Integration Example - SPI Frame Validation
00171 *
00172 * @code{c}
00173 * #include "rx_crc.h"

```

```

00174 * #include "rx_spi_comm.h"
00175 *
00176 * void system_init(void)
00177 * {
00178 *     // Initialize CRC hardware (if available)
00179 *     rx_err_t err = rx_crc_init();
00180 *     if (err != k_rx_ok) {
00181 *         rx_log_warn("CRC", "Hardware CRC unavailable, using software");
00182 *     }
00183 * }
00184 *
00185 * // Transmit side: Compute CRC for outgoing frame
00186 * void send_spi_frame(const uint8_t* payload, uint32_t len)
00187 * {
00188 *     // Build frame header
00189 *     uint8_t frame[279]; // Max SPI frame size
00190 *     frame[0] = 0xAA; // Sync high
00191 *     frame[1] = 0x55; // Sync low
00192 *     // ... populate sequence, length, type, flags ...
00193 *
00194 *     // Copy payload
00195 *     memcpy(&frame[10], payload, len);
00196 *
00197 *     // Compute CRC-32 over header + payload
00198 *     uint32_t crc_offset = 10 + len;
00199 *     uint32_t crc = 0U;
00200 *     (void)rx_crc32_ieee(frame, crc_offset, &crc);
00201 *
00202 *     // Append CRC (little-endian)
00203 *     frame[crc_offset + 0] = (crc >> 0) & 0xFF;
00204 *     frame[crc_offset + 1] = (crc >> 8) & 0xFF;
00205 *     frame[crc_offset + 2] = (crc >> 16) & 0xFF;
00206 *     frame[crc_offset + 3] = (crc >> 24) & 0xFF;
00207 *
00208 *     // Send via SPI
00209 *     spi_transmit(frame, crc_offset + 4);
00210 * }
00211 *
00212 * // Receive side: Validate CRC of incoming frame
00213 * rx_err_t receive_spi_frame(uint8_t* frame, uint32_t frame_len)
00214 * {
00215 *     // Extract received CRC (last 4 bytes, little-endian)
00216 *     uint32_t crc_offset = frame_len - 4;
00217 *     uint32_t received_crc = (frame[crc_offset + 0] << 0) |
00218 *                             (frame[crc_offset + 1] << 8) |
00219 *                             (frame[crc_offset + 2] << 16) |
00220 *                             (frame[crc_offset + 3] << 24);
00221 *
00222 *     // Compute CRC over header + payload (exclude CRC itself)
00223 *     uint32_t calculated_crc = 0U;
00224 *     (void)rx_crc32_ieee(frame, crc_offset, &calculated_crc);
00225 *
00226 *     // Validate
00227 *     if (received_crc != calculated_crc) {
00228 *         rx_log_error("SPI", "CRC mismatch: rx=0x%08X calc=0x%08X",
00229 *             received_crc, calculated_crc);
00230 *         return k_rx_err_crc_mismatch;
00231 *     }
00232 *
00233 *     return k_rx_ok; // Frame valid
00234 * }
00235 * @endcode
00236 *
00237 * ## Integration Example - OneWire ROM Code Validation
00238 *
00239 * @code{.c}
00240 * #include "rx_crc.h"
00241 * #include "rx_hcsr04.h" // OneWire interface
00242 *
00243 * // Validate DS18B20 64-bit ROM code
00244 * bool validate_onewire_rom(const uint8_t* rom)
00245 * {
00246 *     // ROM format: [Family Code][48-bit Serial][CRC-8]
00247 *     // Example: 28 FF 64 1F 21 16 04 B3
00248 *     //             ^^          ^^
00249 *     //             Family (DS18B20)   CRC
00250 *
00251 *     // Compute CRC over first 7 bytes
00252 *     uint32_t crc_out = 0U;
00253 *     (void)rx_crc8_maxim(rom, 7, &crc_out);
00254 *     uint8_t received_crc = rom[7];
00255 *
00256 *     if ((uint8_t)crc_out != received_crc) {
00257 *         rx_log_error("OneWire", "ROM CRC mismatch: rx=0x%02X calc=0x%02X",
00258 *             received_crc, (uint8_t)crc_out);
00259 *         return false;
00260 *     }

```

```

00261 *
00262 * rx_log_debug("OneWire", "ROM valid: Family=0x%02X Serial=%02X%02X%02X%02X%02X",
00263 *             rom[0], rom[6], rom[5], rom[4], rom[3], rom[2], rom[1]);
00264 * return true;
00265 * }
00266 * @endcode
00267 *
00268 * ## Integration Example - Incremental CRC for Large Data
00269 *
00270 * @code{.c}
00271 * #include "rx_crc.h"
00272 *
00273 * // Compute CRC-32 over multiple buffers without concatenation
00274 * uint32_t compute_multi_buffer_crc(void)
00275 * {
00276 *     // Initialize CRC: use 0U as seed -- rx_crc32_update internally un-finalizes/re-finalizes
00277 *     uint32_t crc = 0U;
00278 *
00279 *     // Process buffer 1
00280 *     uint8_t buf1[64] = { ... };
00281 *     crc = rx_crc32_update(crc, buf1, sizeof(buf1));
00282 *
00283 *     // Process buffer 2
00284 *     uint8_t buf2[128] = { ... };
00285 *     crc = rx_crc32_update(crc, buf2, sizeof(buf2));
00286 *
00287 *     // Process buffer 3
00288 *     uint8_t buf3[32] = { ... };
00289 *     crc = rx_crc32_update(crc, buf3, sizeof(buf3));
00290 *
00291 *     return crc; // Finalized CRC-32/IEEE value (rx_crc32_update handles finalization)
00292 * }
00293 *
00294 * // Equivalent to single-shot computation:
00295 * // uint8_t all_data[224];
00296 * // memcpy(&all_data[0], buf1, 64);
00297 * // memcpy(&all_data[64], buf2, 128);
00298 * // memcpy(&all_data[192], buf3, 32);
00299 * // uint32_t crc = 0U;
00300 * // (void)rx_crc32_ieee(all_data, 224, &crc);
00301 * @endcode
00302 *
00303 * ## NASA Power of 10 Compliance
00304 *
00305 * | Rule | Status | Implementation Notes |
00306 * |-----|-----|-----|
00307 * | 1. Simple control flow | [PASS] Pass | No goto/setjmp/recursion, simple loops only |
00308 * | 2. Fixed loop bounds | [PASS] Pass | All loops bounded by data length (caller-controlled) |
00309 * | 3. No dynamic memory | [PASS] Pass | Lookup tables statically allocated, no malloc |
00310 * | 4. Short functions | [PASS] Pass | All functions <40 lines, single responsibility |
00311 * | 5. Assertions | [PASS] Pass | nullptr checks, parameter validation |
00312 * | 6. Small scope | [PASS] Pass | Variables declared near use, minimal scope |
00313 * | 7. Check returns | [PASS] Pass | rx_crc_init() return checked, CRC functions return value directly |
00314 * | 8. Limited preprocessor | [PASS] Pass | Only compile-time HW/SW selection, no magic numbers |
00315 * | 9. Restrict pointers | [PASS] Pass | No function pointers, simple data pointers only |
00316 * | 10. Compiler warnings | [PASS] Pass | -Wall -Wextra -Werror, zero warnings |
00317 *
00318 * ## SOLID Principles
00319 *
00320 * | Principle | Implementation |
00321 * |-----|-----|
00322 * | **Single Responsibility** | Module handles ONLY CRC computation, no frame parsing or validation logic |
00323 * | **Open/Closed** | Supports multiple CRC algorithms (CRC-32, CRC-8) without modifying existing code |
00324 * | **Liskov Substitution** | Hardware/software CRC implementations interchangeable via compile-time selection |
00325 * | **Interface Segregation** | Minimal API (4 functions), each serves specific purpose |
00326 * | **Dependency Inversion** | CRC functions independent of frame protocol, reusable across OneWire, SPI, USB |
00327 *
00328 * ## Module Lifecycle State Machine
00329 *
00330 * @startuml
00331 * [*] --> NOT_INITIALIZED : (after reset)
00332 *
00333 * NOT_INITIALIZED --> READY : rx_crc_init() [k_rx_ok]\n(HW: CRC clock on, DMAC init)
00334 * NOT_INITIALIZED --> NOT_INITIALIZED : rx_crc_init() [already init]\n/ k_rx_err_invalid_state
00335 *
00336 * READY --> READY : rx_crc_compute() [software backend]\nrx_crc32_ieee()\nrx_crc32_update()\nrx_crc8_maxim()
00337 *
00338 * READY --> NOT_INITIALIZED : rx_crc_deinit() [k_rx_ok]\n(HW: CRC clock off, DMAC deinit)
00339 * @enduml
00340 *
00341 * ## Backend Selection for rx_crc_compute()
00342 *
00343 * @startuml
00344 * start
00345 * :rx_crc_compute(config, data, len, result_out);
00346 * if (NULL inputs or len out of [1,65535]?) then (yes)
00347 * :return k_rx_err_null_ptr or k_rx_err_invalid_arg;

```

```

00348 * stop
00349 * endif
00350 * switch (config->backend)
00351 * case (software)
00352 *   if (bit_order != LSB-first?) then (yes)
00353 *     :return k_rx_err_invalid_arg;
00354 *   stop
00355 *   endif
00356 *   :internal_crc_sw_compute() -- portable bitwise/table;
00357 *   :return k_rx_ok;
00358 *   stop
00359 * case (hw_cpu)
00360 *   if (___RX___ target) then (RX72N)
00361 *     if (not initialized?) then (yes)
00362 *       :return k_rx_err_not_initialized;
00363 *     stop
00364 *     endif
00365 *     :internal_crc_hw_cpu_compute()\nCRC peripheral byte-by-byte;
00366 *   else (host fallback)
00367 *     :internal_crc_sw_compute();
00368 *   endif
00369 *   :return result;
00370 *   stop
00371 * case (hw_dma)
00372 *   if (___RX___ target) then (RX72N)
00373 *     if (not initialized?) then (yes)
00374 *       :return k_rx_err_not_initialized;
00375 *     stop
00376 *     endif
00377 *     :internal_crc_hw_dma_compute()\nCRC peripheral via DMAC;
00378 *   else (host fallback)
00379 *     :internal_crc_sw_compute();
00380 *   endif
00381 *   :return result;
00382 *   stop
00383 * case (invalid)
00384 *   :return k_rx_err_invalid_arg;
00385 *   stop
00386 * endswitch
00387 * @enduml
00388 *
00389 * ## References
00390 *
00391 * - **IEEE 802.3**: Ethernet CRC-32 specification
00392 * - **Go crc32 package**: Compatible ChecksumIEEE() implementation
00393 * - **RX72N Manual**: Section 30 (CRC Calculator peripheral)
00394 * - **Dallas/Maxim App Note 27**: Understanding and Using Cyclic Redundancy Checks
00395 * - **DS18B20 Datasheet**: OneWire temperature sensor with CRC-8
00396 *
00397 * @see rx_crc32.c CRC-32 implementation (dispatcher)
00398 * @see rx_crc32_hw.c Hardware CRC-32 (RX72N peripheral)
00399 * @see rx_crc32_sw.c Software CRC-32 (table-based)
00400 * @see rx_crc8.c CRC-8/Maxim implementation
00401 * @see rx_frame.h Frame protocol using CRC-32
00402 * @see rx_spi_comm.h SPI communication using CRC-32
00403 *
00404 * @since Version 1.0.0
00405 * @date 2026-01-01
00406 * @copyright Copyright (c) 2026 Locked Inc.
00407 * SPDX-License-Identifier: MIT
00408 */
00409
00410 #pragma once
00411
00412 #include <stdint.h>
00413
00414 #include "rx_err.h"
00415
00416 #ifdef __cplusplus
00417 extern "C" {
00418 #endif
00419
00420 /*
=====
00421 * CRC Polynomial, Bit Order, Backend, and Config Types
00422 *
=====
00423 */
00424
00425 /**
00426 * @enum rx_crc_poly_t
00427 * @brief CRC polynomial selection for rx_crc_compute()
00428 *
00429 * @details
00430 *   Selects the CRC generator polynomial. Values match the RX72N CRC peripheral
00431 *   * GPS field (bits [2:0] of CRCCR) for CRC-16 through CRC-32C. CRC-8 is always
00432 *   * computed in software because the hardware GPS=0x00 uses poly 0x07 (SMBus),

```

```

00433 * not the Dallas/Maxim poly 0x31 required for OneWire devices.
00434 *
00435 * @since Version 1.0.0
00436 */
00437 typedef enum : uint8_t {
00438     k_rx_crc_poly_crc8      = 0x00U, /**< CRC-8/Maxim (poly 0x31 reflected) - SW only */
00439     k_rx_crc_poly_crc16     = 0x01U, /**< CRC-16/IBM (poly 0x8005 reflected) */
00440     k_rx_crc_poly_crc_ccitt = 0x02U, /**< CRC-CCITT/Kermit (poly 0x1021 reflected) */
00441     k_rx_crc_poly_crc32     = 0x03U, /**< CRC-32/IEEE 802.3 (poly 0x04C11DB7 reflected) */
00442     k_rx_crc_poly_crc32c    = 0x04U, /**< CRC-32C/Castagnoli (poly 0x1EDC6F41 reflected) */
00443 } rx_crc_poly_t;
00444
00445 /**
00446  * @enum rx_crc_bit_order_t
00447  * @brief CRC bit order (reflection mode)
00448  *
00449  * @details
00450  * LSB-first (reflected) is the standard for IEEE 802.3, Dallas/Maxim, and
00451  * Kermit CRC variants. MSB-first is provided for completeness.
00452  *
00453  * @since Version 1.0.0
00454  */
00455 typedef enum : uint8_t {
00456     k_rx_crc_bit_order_lsb_first = 0U, /**< LSB-first (reflected) - standard */
00457     k_rx_crc_bit_order_msb_first = 1U, /**< MSB-first (non-reflected) */
00458 } rx_crc_bit_order_t;
00459
00460 /**
00461  * @enum rx_crc_backend_t
00462  * @brief CRC computation backend selection
00463  *
00464  * @details
00465  * Selects the hardware or software backend at runtime. On host builds
00466  * (non-RX72N), hw_cpu and hw_dma both fall back to software automatically.
00467  *
00468  * @since Version 1.0.0
00469  */
00470 typedef enum : uint8_t {
00471     k_rx_crc_backend_software = 0U, /**< Pure software (portable, all platforms) */
00472     k_rx_crc_backend_hw_cpu   = 1U, /**< HW CRC peripheral, CPU byte loop (RX72N) */
00473     k_rx_crc_backend_hw_dma   = 2U, /**< HW CRC peripheral, DMA-driven (RX72N) */
00474 } rx_crc_backend_t;
00475
00476 /**
00477  * @struct rx_crc_dma_config_t
00478  * @brief DMA-specific configuration for rx_crc_compute()
00479  *
00480  * @since Version 1.0.0
00481  */
00482 typedef struct {
00483     uint32_t timeout_cycles; /**< DMA polling timeout (0 = use k_dma_crc_timeout_cycles) */
00484 } rx_crc_dma_config_t;
00485
00486 /**
00487  * @struct rx_crc_config_t
00488  * @brief Complete CRC computation configuration
00489  *
00490  * @details
00491  * Pass to rx_crc_compute() to select polynomial, bit order, and backend.
00492  *
00493  * @since Version 1.0.0
00494  */
00495 typedef struct {
00496     rx_crc_poly_t      poly;      /**< CRC polynomial selection */
00497     rx_crc_bit_order_t bit_order; /**< Bit order (LSB-first or MSB-first) */
00498     rx_crc_backend_t   backend;   /**< Computation backend */
00499     rx_crc_dma_config_t dma;      /**< DMA config (used when backend == hw_dma) */
00500 } rx_crc_config_t;
00501
00502 /**
00503  * @brief Initialize CRC backend (hardware peripheral if available)
00504  *
00505  * @details
00506  * Initializes the CRC subsystem, enabling hardware CRC peripheral on RX72N
00507  * targets for accelerated CRC-32 computation. On host builds (x86/ARM), this
00508  * is a no-op since only software CRC is available.
00509  *
00510  * **Platform Behavior:**
00511  * - **RX72N** ('RX' defined, 'RX_CRC32_USE_HARDWARE' defined):**
00512  *   - Enables CRCA (CRC Calculator) peripheral via module stop control
00513  *   - Configures CRC polynomial to 0x04C11DB7 (IEEE 802.3)
00514  *   - Sets LSB-first bit order (reflected)
00515  *   - Initializes registers for 32-bit CRC computation
00516  *   - Returns k_rx_ok if hardware available and configured
00517  * - **Host builds** (Linux, macOS, Windows for testing):
00518  *   - No-op, returns k_rx_ok immediately
00519  *   - Software CRC tables already initialized statically

```

```

00520 *
00521 * **Algorithm (RX72N Hardware Init):**
00522 * 1. **Validate not already initialized** (prevent double-init)
00523 * 2. **Enable CRCA module clock** via MSTPCRB register (bit 23 = 0)
00524 * 3. **Reset CRC peripheral** by writing control register
00525 * 4. **Configure polynomial** to 0x04C11DB7 (IEEE 802.3 standard)
00526 * 5. **Set bit order** to LSB-first (reflected, compatible with software CRC)
00527 * 6. **Set initial value** to 0xFFFFFFFF (IEEE 802.3 standard)
00528 * 7. **Mark initialized** (set internal flag)
00529 *
00530 * @return rx_err_t Initialization result
00531 * @retval k_rx_ok Initialization successful (hardware or no-op)
00532 * @retval k_rx_err_invalid_state Already initialized (RX72N only)
00533 * @retval k_rx_err_hardware Hardware CRC peripheral fault (RX72N only, rare)
00534 *
00535 * @pre System clock (PCLKA) must be configured and running
00536 * @pre Module stop registers must be accessible
00537 * @post On RX72N, CRCA peripheral ready for rx_crc32_ieee() hardware acceleration
00538 * @post On host, software CRC tables ready for use
00539 * @post rx_crc32_ieee() will use fastest available implementation
00540 *
00541 * @note Call this ONCE during system initialization before any CRC operations
00542 * @note Thread-safe ONLY if called single-threaded during init
00543 * @warning Do not call from interrupt context
00544 * @warning On RX72N, failure to call this results in slower software CRC fallback
00545 *
00546 * @par Performance Impact:
00547 * - **With init (HW CRC)**: 32 us per 100 bytes (4x faster)
00548 * - **Without init (SW CRC)**: 130 us per 100 bytes
00549 *
00550 * @par Example - System Initialization:
00551 * @code{.c}
00552 * void system_init(void)
00553 * {
00554 *     // Initialize clocks first
00555 *     rx_clock_init();
00556 *
00557 *     // Initialize CRC (enables hardware if available)
00558 *     rx_err_t err = rx_crc_init();
00559 *     if (err == k_rx_ok) {
00560 *         rx_log_info("CRC", "CRC initialized (HW accelerated)");
00561 *     } else {
00562 *         rx_log_warn("CRC", "CRC init failed: %d (SW fallback)", err);
00563 *     }
00564 *
00565 *     // Now CRC functions can use hardware acceleration
00566 *     uint32_t crc = 0U;
00567 *     (void)rx_crc32_ieee(data, len, &crc); // Fast path
00568 * }
00569 * @endcode
00570 *
00571 * @par Example - Conditional Hardware CRC:
00572 * @code{.c}
00573 * #ifdef __RX__
00574 *     // RX72N: Try to enable hardware CRC
00575 *     if (rx_crc_init() == k_rx_ok) {
00576 *         rx_log_info("CRC", "Using hardware CRC (4x faster)");
00577 *     } else {
00578 *         rx_log_warn("CRC", "Using software CRC");
00579 *     }
00580 * #else
00581 *     // Host: Software CRC always available
00582 *     rx_crc_init(); // No-op, but safe to call
00583 *     rx_log_info("CRC", "Using software CRC (portable)");
00584 * #endif
00585 * @endcode
00586 *
00587 * @see rx_crc_deinit() Disable CRC hardware when done
00588 * @see rx_crc32_ieee() CRC-32 computation (uses HW if initialized)
00589 * @see rx_crc32_hw.c Hardware CRC implementation
00590 *
00591 * @since Version 1.0.0
00592 */
00593 [[nodiscard]] rx_err_t rx_crc_init(void);
00594
00595 /**
00596 * @brief Deinitialize CRC backend (hardware peripheral shutdown)
00597 *
00598 * @details
00599 * Disables the CRC hardware peripheral on RX72N targets to save power when CRC
00600 * operations are no longer needed. On host builds (x86/ARM), this is a no-op
00601 * since software CRC requires no deinitialization.
00602 *
00603 * **Platform Behavior:**
00604 * - **RX72N** ('__RX__' defined, 'RX_CRC32_USE_HARDWARE' defined)**:
00605 *   - Disables CRCA (CRC Calculator) peripheral via module stop control
00606 *   - Sets MSTPCRB bit 23 = 1 to stop peripheral clock

```

```

00607 * - Clears internal initialization flag
00608 * - Frees hardware resource for other modules
00609 * - Note: Current implementation intentionally leaves CRC enabled for continuous availability
00610 * - Host builds (Linux, macOS, Windows for testing):
00611 * - No-op, returns k_rx_ok immediately
00612 * - Software CRC tables remain in memory (statically allocated)
00613 *
00614 * Algorithm (RX72N Hardware Deinit):
00615 * 1. Validate initialized (prevent deinit of uninitialized module)
00616 * 2. Disable CRCA module clock via MSTPCRB register (bit 23 = 1)
00617 * 3. Clear registers (reset peripheral state)
00618 * 4. Mark uninitialized (clear internal flag)
00619 *
00620 * Power Savings:
00621 * - With deinit: CRCA module stopped, saves ~0.2 mA @ 3.3V (negligible)
00622 * - Without deinit: CRCA remains clocked, minimal power overhead
00623 * - Recommendation: Leave CRC enabled unless power is critical (low-power mode)
00624 *
00625 * @return rx_err_t Deinitialization result
00626 * @retval k_rx_ok Deinitialization successful (hardware or no-op)
00627 * @retval k_rx_err_invalid_state Not initialized (RX72N only)
00628 *
00629 * @pre CRC module must have been initialized via rx_crc_init()
00630 * @post On RX72N, CRCA peripheral disabled and clock stopped (if implementation changed)
00631 * @post rx_crc32_ieee() will fall back to software CRC
00632 * @post On host, no observable change (software CRC unaffected)
00633 *
00634 * @note Call this ONCE during system shutdown if power optimization is critical
00635 * @note Thread-safe ONLY if called single-threaded during shutdown
00636 * @note After deinit, rx_crc32_ieee() still works (software fallback)
00637 * @warning Do not call from interrupt context
00638 * @warning Current implementation is a no-op to keep CRC available
00639 *
00640 * @par Design Rationale (No-Op Deinit):
00641 * The current implementation intentionally does NOT disable the CRC module
00642 * after initialization. Reasons:
00643 * 1. CRC calculations may be needed at any time for frame validation
00644 * 2. Module stop/start overhead is significant (~50 us per init)
00645 * 3. Power savings from disabling are minimal (<0.2 mA)
00646 * 4. No functional benefit in typical operation
00647 *
00648 * If power optimization is critical (low-power sleep modes),
00649 * modify implementation to actually disable peripheral.
00650 *
00651 * @par Performance Impact:
00652 * - After deinit (SW CRC): 130 us per 100 bytes (4x slower)
00653 * - After init (HW CRC): 32 us per 100 bytes
00654 *
00655 * @par Example - System Shutdown:
00656 * @code{.c}
00657 * void system_shutdown(void)
00658 * {
00659 *     // Deinitialize CRC (optional, for power savings)
00660 *     rx_err_t err = rx_crc_deinit();
00661 *     if (err == k_rx_ok) {
00662 *         rx_log_info("CRC", "CRC deinitialized (power saving)");
00663 *     } else {
00664 *         rx_log_warn("CRC", "CRC deinit failed: %d", err);
00665 *     }
00666 *
00667 *     // Other shutdown tasks...
00668 *     rx_gpio_deinit();
00669 *     rx_clock_deinit();
00670 * }
00671 * @endcode
00672 *
00673 * @par Example - Conditional Deinit (Low-Power Mode):
00674 * @code{.c}
00675 * void enter_low_power_mode(void)
00676 * {
00677 *     // Disable CRC to save power
00678 *     rx_crc_deinit();
00679 *
00680 *     // CRC still works (software fallback)
00681 *     uint32_t crc = 0U;
00682 *     (void)rx_crc32_ieee(data, len, &crc); // Slower, but functional
00683 *
00684 *     // Enter low-power sleep
00685 *     rx_power_enter_sleep();
00686 * }
00687 *
00688 * void exit_low_power_mode(void)
00689 * {
00690 *     // Re-enable CRC for performance
00691 *     rx_crc_init();
00692 *
00693 *     // CRC now fast again (hardware)

```

```

00694 *   uint32_t crc = 0U;
00695 *   (void)rx_crc32_ieee(data, len, &crc); // Fast path
00696 * }
00697 * @endcode
00698 *
00699 * @see rx_crc_init() Initialize CRC hardware
00700 * @see rx_crc32_ieee() CRC-32 computation (unaffected by deinit)
00701 * @see rx_crc32_hw.c Hardware CRC implementation
00702 *
00703 * @since Version 1.0.0
00704 */
00705 [[nodiscard]] rx_err_t rx_crc_deinit(void);
00706
00707 /**
00708 * @brief Compute CRC checksum over a buffer using the specified configuration
00709 *
00710 * @details
00711 * Unified CRC computation function supporting all five polynomials and three
00712 * backends (software, hw_cpu, hw_dma). On host builds, hw_cpu and hw_dma
00713 * fall back to software automatically. CRC-8/Maxim always uses software
00714 * because the hardware peripheral uses poly 0x07, not the required 0x31.
00715 *
00716 * @param[in] config CRC configuration (polynomial, bit order, backend)
00717 * @param[in] data Input data buffer
00718 * @param[in] len Number of bytes (1 - 65535)
00719 * @param[out] result_out Computed CRC value
00720 *
00721 * @return rx_err_t
00722 * @retval k_rx_ok CRC computed; *result_out is valid
00723 * @retval k_rx_err_null_ptr config, data, or result_out is NULL
00724 * @retval k_rx_err_invalid_arg len == 0 or len > k_crc_len_max;
00725 *                               poly or bit_order out of valid enum range;
00726 *                               software or host-fallback backend with
00727 *                               bit_order == k_rx_crc_bit_order_msb_first;
00728 *                               unknown backend value
00729 * @retval k_rx_err_not_initialized hw_cpu or hw_dma backend requested on RX72N
00730 *                               before rx_crc_init() has been called
00731 * @retval k_rx_err_timeout DMA transfer did not complete (hw_dma only)
00732 *
00733 * @pre config must point to a valid rx_crc_config_t
00734 * @pre data must point to at least len readable bytes
00735 * @post On k_rx_ok, *result_out contains the computed CRC value
00736 * @post On error, *result_out is unchanged
00737 *
00738 * @note Not thread-safe if hardware backend shares the CRC peripheral
00739 * @since Version 1.0.0
00740 */
00741 [[nodiscard]] rx_err_t rx_crc_compute(const rx_crc_config_t* config,
00742                                       const uint8_t* data,
00743                                       uint32_t len,
00744                                       uint32_t* result_out);
00745
00746 /**
00747 * @brief Compute CRC-32/IEEE 802.3 checksum over buffer
00748 *
00749 * @details
00750 * Computes 32-bit cyclic redundancy check using IEEE 802.3 polynomial
00751 * (0x04C11DB7 in reflected/LSB-first form 0xEDB88320). Automatically selects
00752 * hardware-accelerated or software table-based implementation at compile time.
00753 *
00754 * **Algorithm Selection:**
00755 * - **All targets**: Software table lookup (portable, always available, no init required)
00756 * - For hardware-accelerated CRC-32, call rx_crc_compute() with k_rx_crc_backend_hw_cpu
00757 *   or k_rx_crc_backend_hw_dma after rx_crc_init() has been called
00758 *
00759 * **CRC-32/IEEE 802.3 Specification:**
00760 * - **Polynomial**: @f{x^{32} + x^{26} + x^{23} + \dots + x + 1}@f (0x04C11DB7)
00761 * - **Initial value**: 0xFFFFFFFF
00762 * - **Final XOR**: 0xFFFFFFFF
00763 * - **Bit order**: LSB-first (reflected)
00764 * - **Byte order**: Process bytes sequentially, LSB of each byte first
00765 *
00766 * **Compatibility:**
00767 * - Matches Go `crc32.ChecksumIEEE()`
00768 * - Matches Python `zlib.crc32()`
00769 * - Matches Ethernet FCS (Frame Check Sequence)
00770 * - Matches ZIP, PNG, Gzip CRC-32
00771 *
00772 * **Algorithm (Software):**
00773 * 1. **Initialize CRC**: `crc = 0xFFFFFFFF`
00774 * 2. **For each byte**:
00775 *   - `table_index = (crc ^ byte) & 0xFF`
00776 *   - `crc = (crc » 8) ^ lookup_table[table_index]`
00777 * 3. **Finalize**: `crc ^= 0xFFFFFFFF`
00778 * 4. **Return** final CRC value
00779 *
00780 * **Algorithm (Hardware, RX72N):**

```

```

00781 * 1. **Write data to CRCA data register** (DMA or CPU)
00782 * 2. **Read CRC from CRCA result register**
00783 * 3. **Apply final XOR** (0xFFFFFFFF)
00784 * 4. **Return** final CRC value
00785 *
00786 * @param[in] data Input buffer to checksum (non-NULL, len >= 1)
00787 * @param[in] len Number of bytes to include in CRC (1 - k_crc_len_max)
00788 * @param[out] out Computed CRC-32 value; valid when return is k_rx_ok
00789 *
00790 * @return rx_err_t Error code
00791 * @retval k_rx_ok Success, *out contains the CRC value
00792 * @retval k_rx_err_null_ptr data or out is NULL
00793 * @retval k_rx_err_invalid_arg len == 0 or len > k_crc_len_max
00794 *
00795 * @pre data must point to at least len readable bytes
00796 * @pre len must be in [1, k_crc_len_max]
00797 * @post On k_rx_ok, *out contains the IEEE 802.3 CRC-32
00798 * @post On error, *out is unchanged
00799 *
00800 * @note Thread-safe: always uses the software backend (stateless, read-only table);
00801 * rx_crc_init() is NOT required and does NOT affect the result of this function
00802 * @note For incremental CRC over multiple buffers, use rx_crc32_update()
00803 * @warning Passing invalid data pointer or incorrect len causes undefined behavior
00804 * @warning For large buffers (>10 KB), consider chunking with rx_crc32_update()
00805 *
00806 * @par Performance:
00807 * - **Hardware (RX72N)**: ~32 us / 100 bytes @ 240 MHz
00808 * - **Software (RX72N)**: ~130 us / 100 bytes @ 240 MHz
00809 * - **Software (x86)**: ~50 us / 100 bytes @ 3 GHz
00810 *
00811 * @par Example - Basic Usage:
00812 * @code{.c}
00813 * uint8_t data[] = "Hello World";
00814 * uint32_t crc = 0U;
00815 * (void)rx_crc32_ieee(data, sizeof(data) - 1, &crc); // Exclude null terminator
00816 * printf("CRC-32: 0x%08X\n", crc);
00817 * // Output: CRC-32: 0x4A17B156
00818 * @endcode
00819 *
00820 * @par Example - Frame Integrity Check:
00821 * @code{.c}
00822 * // Transmit side: Append CRC to frame
00823 * uint8_t tx_frame[279];
00824 * // ... populate header and payload ...
00825 * uint32_t crc = 0U;
00826 * (void)rx_crc32_ieee(tx_frame, 275, &crc); // CRC over first 275 bytes
00827 *
00828 * // Append CRC (little-endian)
00829 * tx_frame[275] = (crc » 0) & 0xFF;
00830 * tx_frame[276] = (crc » 8) & 0xFF;
00831 * tx_frame[277] = (crc » 16) & 0xFF;
00832 * tx_frame[278] = (crc » 24) & 0xFF;
00833 *
00834 * // Receive side: Validate CRC
00835 * uint8_t rx_frame[279];
00836 * // ... receive frame from SPI ...
00837 *
00838 * uint32_t received_crc = (rx_frame[275] « 0) |
00839 * (rx_frame[276] « 8) |
00840 * (rx_frame[277] « 16) |
00841 * (rx_frame[278] « 24);
00842 *
00843 * uint32_t calculated_crc = 0U;
00844 * (void)rx_crc32_ieee(rx_frame, 275, &calculated_crc);
00845 *
00846 * if (received_crc == calculated_crc) {
00847 * // Frame valid, process data
00848 * } else {
00849 * // Corruption detected, discard frame
00850 * rx_log_error("CRC", "Mismatch: rx=0x%08X calc=0x%08X",
00851 * received_crc, calculated_crc);
00852 * }
00853 * @endcode
00854 *
00855 * @par Example - Zero-Length Buffer:
00856 * @code{.c}
00857 * uint32_t crc = 0U;
00858 * rx_err_t err = rx_crc32_ieee(nullptr, 0, &crc);
00859 * // Returns: k_rx_err_null_ptr (nullptr is invalid)
00860 * @endcode
00861 *
00862 * @par Example - Performance Comparison:
00863 * @code{.c}
00864 * #include "rx_time.h"
00865 *
00866 * uint8_t data[1024];
00867 * memset(data, 0x55, sizeof(data));

```

```

00868 *
00869 * // Without hardware CRC initialization
00870 * uint32_t start = rx_time_get_us();
00871 * uint32_t crc_sw = 0U;
00872 * (void)rx_crc32_ieee(data, sizeof(data), &crc_sw); // Software
00873 * uint32_t elapsed_sw = rx_time_get_us() - start;
00874 * rx_log_info("CRC", "Software: %u us", elapsed_sw);
00875 * // Output: Software: 1320 us
00876 *
00877 * // With hardware CRC initialization
00878 * rx_crc_init();
00879 * start = rx_time_get_us();
00880 * uint32_t crc_hw = 0U;
00881 * (void)rx_crc32_ieee(data, sizeof(data), &crc_hw); // Hardware
00882 * uint32_t elapsed_hw = rx_time_get_us() - start;
00883 * rx_log_info("CRC", "Hardware: %u us (%.1fx faster)",
00884 *             elapsed_hw, (float)elapsed_sw / elapsed_hw);
00885 * // Output: Hardware: 320 us (4.1x faster)
00886 *
00887 * // Both produce same result
00888 * assert(crc_sw == crc_hw);
00889 * @endcode
00890 *
00891 * @see rx_crc32_update() Incremental CRC for multiple buffers
00892 * @see rx_crc_init() Initialize hardware CRC (optional, for performance)
00893 * @see rx_crc32_hw.c Hardware CRC implementation (RX72N)
00894 * @see rx_crc32_sw.c Software CRC implementation (portable)
00895 *
00896 * @since Version 1.0.0
00897 */
00898 static inline rx_err_t rx_crc32_ieee(const uint8_t* data, uint32_t len, uint32_t* out)
00899 {
00900     static const rx_crc_config_t s_cfg = {
00901         .poly      = k_rx_crc_poly_crc32,
00902         .bit_order = k_rx_crc_bit_order_lsb_first,
00903         .backend    = k_rx_crc_backend_software, /* Software: always available, no rx_crc_init() needed */
00904         .dma        = {.timeout_cycles = 0U},
00905     };
00906     return rx_crc_compute(&s_cfg, data, len, out);
00907 }
00908
00909 /**
00910 * @brief Update CRC-32 with additional data (incremental computation)
00911 *
00912 * @details
00913 * Continues a CRC-32/IEEE 802.3 calculation from a previous partial result,
00914 * allowing incremental CRC computation over non-contiguous buffers or streaming
00915 * data without concatenation. Uses software table-based implementation (hardware
00916 * CRC peripheral does not support loading arbitrary initial state).
00917 *
00918 * **Use Cases:**
00919 * 1. **Streaming data**: Compute CRC as data arrives in chunks
00920 * 2. **Non-contiguous buffers**: CRC over header + payload without copying
00921 * 3. **Large data sets**: Process data in manageable chunks
00922 * 4. **Memory-constrained**: Avoid allocating large concatenation buffer
00923 *
00924 * **Implementation:**
00925 * - **Always uses software CRC** (even on RX72N with hardware support)
00926 * - **Rationale**: Hardware CRC peripheral cannot load arbitrary initial CRC state
00927 * - **Performance**: Same as software rx_crc32_ieee() (~130 us per 100 bytes @ 240 MHz)
00928 *
00929 * **Algorithm (Software Table Lookup):**
00930 * 1. **Un-finalize input CRC**: `crc ^= 0xFFFFFFFF` (reverse final XOR)
00931 * 2. **For each byte**:
00932 *   - `table_index = (crc ^ byte) & 0xFF`
00933 *   - `crc = (crc » 8) ^ lookup_table[table_index]`
00934 * 3. **Re-finalize CRC**: `crc ^= 0xFFFFFFFF` (apply final XOR)
00935 * 4. **Return** updated CRC value
00936 *
00937 * **Mathematical Equivalence:**
00938 * Computing CRC over concatenated data produces identical result:
00939 * ```
00940 * // Method 1: Incremental
00941 * uint32_t crc = 0U;
00942 * (void)rx_crc32_ieee(buf1, len1, &crc);
00943 * crc = rx_crc32_update(crc, buf2, len2);
00944 *
00945 * // Method 2: Concatenated (equivalent)
00946 * uint8_t combined[len1 + len2];
00947 * memcpy(&combined[0], buf1, len1);
00948 * memcpy(&combined[len1], buf2, len2);
00949 * uint32_t crc = 0U;
00950 * (void)rx_crc32_ieee(combined, len1 + len2, &crc);
00951 *
00952 * // Both methods produce identical CRC
00953 * ```
00954 *

```

```

00955 * @param[in] crc Previous CRC-32 value
00956 * - **Valid range**: 0x00000000 - 0xFFFFFFFF (any 32-bit value)
00957 * - **Initial value**: Use result from rx_crc32_ieee() or previous rx_crc32_update()
00958 * - **Special case**: 0xFFFFFFFF is valid previous CRC (not same as init value)
00959 *
00960 * @param[in] data Additional data buffer to include in CRC
00961 * - **Valid range**: NULL or pointer to buffer of size len
00962 * - **NULL handling**: If NULL, len must be 0 (returns input crc unchanged)
00963 * - **Alignment**: No alignment requirement (byte-addressable)
00964 *
00965 * @param[in] len Number of bytes in data buffer
00966 * - **Valid range**: 0 - 4294967295 (0 - 232-1)
00967 * - **Typical**: 10-275 bytes (frame chunks)
00968 * - **Zero handling**: len=0 returns input crc unchanged
00969 *
00970 * @return uint32_t Updated CRC-32 checksum
00971 * - **Range**: 0x00000000 - 0xFFFFFFFF (all 32-bit values possible)
00972 * - **Deterministic**: Same inputs always produce same output
00973 * - **Associative**: Order of updates matters, but partial results can be cached
00974 *
00975 * @pre If data != nullptr, data must point to valid buffer of size len
00976 * @pre len must be in [1, k_crc_len_max] for any update to occur; len == 0 or
00977 *     len > k_crc_len_max both cause pass-through (input crc returned unchanged)
00978 * @post Return value is valid IEEE 802.3 CRC-32 if data != NULL and 0 < len <= k_crc_len_max
00979 * @post Returns input crc unchanged (pass-through) if data == NULL, len == 0, or len > k_crc_len_max
00980 *
00981 * @note Thread-safe; stateless, read-only table access only
00982 * @note For single-shot CRC over contiguous buffer, use rx_crc32_ieee() (simpler)
00983 * @note Software CRC only - hardware peripheral does not support incremental mode
00984 * @warning Chunks larger than k_crc_len_max (65535 bytes) are silently ignored --
00985 *     the function returns the input crc unchanged with NO error signal;
00986 *     split chunks exceeding k_crc_len_max into multiple calls
00987 * @warning Passing invalid data pointer or incorrect len causes undefined behavior
00988 * @warning Do not mix rx_crc32_ieee() and rx_crc32_update() incorrectly (see examples)
00989 *
00990 * @par Performance:
00991 * - **Software (RX72N)**: ~130 us / 100 bytes @ 240 MHz (same as rx_crc32_ieee)
00992 * - **Software (x86)**: ~50 us / 100 bytes @ 3 GHz
00993 *
00994 * @par Example - Basic Incremental CRC:
00995 * @code{.c}
00996 * // Compute CRC over two buffers without concatenation
00997 * uint8_t header[10] = { 0xAA, 0x55, ... };
00998 * uint8_t payload[64] = { ... };
00999 *
01000 * // Start with header
01001 * uint32_t crc = 0U;
01002 * (void)rx_crc32_ieee(header, sizeof(header), &crc);
01003 *
01004 * // Update with payload
01005 * crc = rx_crc32_update(crc, payload, sizeof(payload));
01006 *
01007 * // Final CRC is over header + payload (74 bytes total)
01008 * printf("CRC-32: 0x%08X\n", crc);
01009 * @endcode
01010 *
01011 * @par Example - Streaming Data:
01012 * @code{.c}
01013 * // Compute CRC as data arrives in chunks
01014 * uint32_t crc = 0U; // Correct seed: rx_crc32_update() un-finalizes 0 to 0xFFFFFFFF internally
01015 *
01016 * // Process chunk 1
01017 * uint8_t chunk1[32];
01018 * spi_receive(chunk1, sizeof(chunk1));
01019 * crc = rx_crc32_update(crc, chunk1, sizeof(chunk1));
01020 *
01021 * // Process chunk 2
01022 * uint8_t chunk2[64];
01023 * spi_receive(chunk2, sizeof(chunk2));
01024 * crc = rx_crc32_update(crc, chunk2, sizeof(chunk2));
01025 *
01026 * // Process chunk 3
01027 * uint8_t chunk3[16];
01028 * spi_receive(chunk3, sizeof(chunk3));
01029 * crc = rx_crc32_update(crc, chunk3, sizeof(chunk3));
01030 *
01031 * // Final CRC is over all chunks (112 bytes total)
01032 * printf("Streaming CRC-32: 0x%08X\n", crc);
01033 * @endcode
01034 *
01035 * @par Example - Frame Header + Payload:
01036 * @code{.c}
01037 * // SPI frame structure
01038 * typedef struct {
01039 *     uint16_t sync;    // 0xAA55
01040 *     uint16_t seq;     // Sequence number
01041 *     uint8_t len;      // Payload length

```

```

01042 *  uint8_t type;      // Message type
01043 *  uint8_t flags;     // Control flags
01044 *  uint8_t reserved;  // Reserved
01045 * } spi_header_t;
01046 *
01047 * spi_header_t header;
01048 * uint8_t payload[255];
01049 *
01050 * // ... populate header and payload ...
01051 *
01052 * // Compute CRC over header + payload
01053 * uint32_t crc = 0U;
01054 * (void)rx_crc32_ieee((uint8_t*)&header, sizeof(header), &crc);
01055 * crc = rx_crc32_update(crc, payload, header.len);
01056 *
01057 * // Append CRC to frame
01058 * uint8_t crc_bytes[4] = {
01059 *     (crc » 0) & 0xFF,
01060 *     (crc » 8) & 0xFF,
01061 *     (crc » 16) & 0xFF,
01062 *     (crc » 24) & 0xFF,
01063 * };
01064 * spi_transmit(crc_bytes, sizeof(crc_bytes));
01065 * @endcode
01066 *
01067 * @par Example - Multi-Buffer CRC (Memory Efficient):
01068 * @code{.c}
01069 * // Compute CRC over 3 buffers without allocating concatenation buffer
01070 * uint8_t buf1[64]; // 64 bytes
01071 * uint8_t buf2[128]; // 128 bytes
01072 * uint8_t buf3[32]; // 32 bytes
01073 *
01074 * // Method 1: Incremental (no extra memory)
01075 * uint32_t crc = 0U;
01076 * (void)rx_crc32_ieee(buf1, sizeof(buf1), &crc);
01077 * crc = rx_crc32_update(crc, buf2, sizeof(buf2));
01078 * crc = rx_crc32_update(crc, buf3, sizeof(buf3));
01079 *
01080 * // Method 2: Concatenated (requires 224-byte buffer)
01081 * // uint8_t combined[224];
01082 * // memcpy(&combined[0], buf1, 64);
01083 * // memcpy(&combined[64], buf2, 128);
01084 * // memcpy(&combined[192], buf3, 32);
01085 * // uint32_t crc = 0U;
01086 * // (void)rx_crc32_ieee(combined, 224, &crc);
01087 *
01088 * // Both methods produce identical CRC, but Method 1 saves 224 bytes
01089 * @endcode
01090 *
01091 * @par Example - WRONG Usage (Common Mistake):
01092 * @code{.c}
01093 * // [FAIL] WRONG: Don't use 0xFFFFFFFF as seed for rx_crc32_update()
01094 * // rx_crc32_update() un-finalizes by XOR with 0xFFFFFFFF internally.
01095 * // Seeding with 0xFFFFFFFF produces work=0x00000000 (wrong initial state).
01096 * uint32_t crc = 0xFFFFFFFF; // Wrong: un-finalizes to 0, not 0xFFFFFFFF
01097 * crc = rx_crc32_update(crc, buf1, len1); // [FAIL] Incorrect CRC!
01098 *
01099 * // [PASS] CORRECT: Use 0U as seed for first rx_crc32_update() call
01100 * // rx_crc32_update(0, ...) un-finalizes to 0xFFFFFFFF (correct initial state).
01101 * uint32_t crc = 0U;
01102 * crc = rx_crc32_update(crc, buf1, len1); // [PASS] Correct
01103 * crc = rx_crc32_update(crc, buf2, len2); // [PASS] Continues correctly
01104 *
01105 * // OR: Start with rx_crc32_ieee() on first buffer (also correct)
01106 * uint32_t crc = 0U;
01107 * (void)rx_crc32_ieee(buf1, len1, &crc);
01108 * crc = rx_crc32_update(crc, buf2, len2); // [PASS] Correct
01109 * @endcode
01110 *
01111 * @see rx_crc32_ieee() Single-shot CRC-32 computation
01112 * @see rx_crc32_sw.c Software CRC implementation (used by this function)
01113 *
01114 * @since Version 1.0.0
01115 */
01116 uint32_t rx_crc32_update(uint32_t crc, const uint8_t* data, uint32_t len);
01117
01118 /**
01119 * @brief Compute CRC-8/Maxim checksum for Dallas/Maxim OneWire devices
01120 *
01121 * @details
01122 * Computes 8-bit cyclic redundancy check using Dallas/Maxim polynomial (0x31)
01123 * in reflected/LSB-first form (0x8C). This CRC-8 variant is used by all Dallas
01124 * Semiconductor and Maxim Integrated OneWire devices for ROM code and scratchpad
01125 * data validation.
01126 *
01127 * **CRC-8/Maxim Specification:**
01128 * - **Polynomial**: @f{x^8 + x^5 + x^4 + 1}@f$ (0x31 normal, 0x8C reflected)

```

```

01129 * - **Initial value**: 0x00
01130 * - **Final XOR**: 0x00 (no finalization)
01131 * - **Bit order**: LSB-first (reflected)
01132 * - **Byte order**: Process bytes sequentially, LSB of each byte first
01133 *
01134 * **Compatibility:**
01135 * - DS18B20 temperature sensor (ROM code validation)
01136 * - DS2482 OneWire bridge (command/response validation)
01137 * - DS2431 EEPROM (memory page validation)
01138 * - DS28EA00 IO device (ROM code validation)
01139 * - All Dallas/Maxim OneWire devices
01140 *
01141 * **Algorithm (Software Table Lookup):**
01142 * 1. **Initialize CRC**: `crc = 0x00`
01143 * 2. **For each byte**:
01144 *   - `table_index = crc ^ byte`
01145 *   - `crc = lookup_table[table_index]`
01146 * 3. **Return** final CRC value (no finalization XOR)
01147 *
01148 * **Mathematical Definition:**
01149 *
01150 * **Polynomial:**
01151 * @f[
01152 *    $P(x) = x^8 + x^5 + x^4 + 1$ 
01153 * @f]
01154 *
01155 * **Binary representation:** 0x31 (normal), 0x8C (reflected)
01156 *
01157 * **Algorithm (LSB-first):**
01158 * @f[
01159 *   \text{CRC} = 0x00 \quad (\text{initial value})
01160 * @f]
01161 * @f[
01162 *   \text{For each byte } b: \quad \text{CRC} = \text{Table}[\text{CRC} \oplus b]
01163 * @f]
01164 *
01165 * **Properties:**
01166 * - Detects all single-bit errors
01167 * - Detects all 2-bit errors within 8 bits
01168 * - Detects all burst errors  $\leq 8$  bits
01169 * - Detects 99.6% of all other errors (256 possible values)
01170 *
01171 * @param[in] data Input buffer to checksum (non-NULL, len  $\geq 1$ )
01172 * @param[in] len Number of bytes to include in CRC (1 - k_crc_len_max)
01173 * @param[out] out Computed CRC-8 value (8 significant bits); valid on k_rx_ok
01174 *
01175 * @return rx_err_t Error code
01176 * @retval k_rx_ok Success, *out contains the CRC-8 value
01177 * @retval k_rx_err_null_ptr data or out is NULL
01178 * @retval k_rx_err_invalid_arg len == 0 or len > k_crc_len_max
01179 *
01180 * @pre data must point to at least len readable bytes
01181 * @pre len must be in [1, k_crc_len_max]
01182 * @post On k_rx_ok, *out contains the CRC-8/Maxim value (0x00-0xFF)
01183 * @post On error, *out is unchanged
01184 *
01185 * @note This function is thread-safe (stateless, read-only tables)
01186 * @note Software implementation only (no hardware acceleration on RX72N)
01187 * @note For OneWire ROM codes, compute CRC over first 7 bytes, compare to byte 8
01188 * @warning Passing invalid data pointer or incorrect len causes undefined behavior
01189 * @warning Different from other CRC-8 variants (CRC-8/CCITT, CRC-8/WCDMA, etc.)
01190 *
01191 * @par Performance:
01192 * - **Software (RX72N)**: ~15 us / 8 bytes @ 240 MHz
01193 * - **Software (x86)**: ~5 us / 8 bytes @ 3 GHz
01194 *
01195 * @par Example - OneWire ROM Code Validation:
01196 * @code{.c}
01197 * // DS18B20 ROM code format: [Family][Serial 6 bytes][CRC-8]
01198 * uint8_t rom[8] = { 0x28, 0xFF, 0x64, 0x1F, 0x21, 0x16, 0x04, 0xB3 };
01199 * //
01200 * // Family (DS18B20) CRC
01201 *
01202 * // Compute CRC over first 7 bytes
01203 * uint32_t crc_out = 0U;
01204 * (void)rx_crc8_maxim(rom, 7, &crc_out);
01205 * uint8_t calculated_crc = (uint8_t)crc_out;
01206 * uint8_t received_crc = rom[7];
01207 *
01208 * if (calculated_crc == received_crc) {
01209 *   printf("ROM valid: Family=0x%02X CRC=0x%02X\n", rom[0], rom[7]);
01210 *   // Output: ROM valid: Family=0x28 CRC=0xB3
01211 * } else {
01212 *   printf("ROM invalid: expected=0x%02X actual=0x%02X\n",
01213 *         calculated_crc, received_crc);
01214 * }
01215 * @endcode

```

```

01216 *
01217 * @par Example - DS18B20 Scratchpad Validation:
01218 * @code{.c}
01219 * // DS18B20 scratchpad: 9 bytes total (8 data + 1 CRC)
01220 * uint8_t scratchpad[9];
01221 * rx_owewire_read_scratchpad(scratchpad, sizeof(scratchpad));
01222 *
01223 * // Compute CRC over first 8 bytes
01224 * uint32_t crc_out = 0U;
01225 * (void)rx_crc8_maxim(scratchpad, 8, &crc_out);
01226 * uint8_t calculated_crc = (uint8_t)crc_out;
01227 * uint8_t received_crc = scratchpad[8];
01228 *
01229 * if (calculated_crc == received_crc) {
01230 *     // Extract temperature (bytes 0-1, little-endian)
01231 *     int16_t raw_temp = (scratchpad[1] « 8) | scratchpad[0];
01232 *     float temp_c = raw_temp / 16.0f; // 12-bit resolution
01233 *     printf("Temperature: %.2f degC\n", temp_c);
01234 * } else {
01235 *     rx_log_error("OneWire", "Scratchpad CRC mismatch");
01236 * }
01237 * @endcode
01238 *
01239 * @par Example - Computing CRC for Transmission:
01240 * @code{.c}
01241 * // Send custom command to OneWire device with CRC
01242 * void send_owewire_command(uint8_t cmd, uint8_t param)
01243 * {
01244 *     uint8_t buffer[3];
01245 *     buffer[0] = cmd;
01246 *     buffer[1] = param;
01247 *
01248 *     // Compute CRC over command + parameter
01249 *     uint32_t crc_out = 0U;
01250 *     (void)rx_crc8_maxim(buffer, 2, &crc_out);
01251 *     buffer[2] = (uint8_t)crc_out;
01252 *
01253 *     // Transmit with CRC
01254 *     rx_owewire_write(buffer, sizeof(buffer));
01255 * }
01256 * @endcode
01257 *
01258 * @par Example - Batch ROM Validation:
01259 * @code{.c}
01260 * // Validate multiple DS18B20 ROM codes
01261 * typedef struct {
01262 *     uint8_t rom[8];
01263 *     bool valid;
01264 * } onewire_device_t;
01265 *
01266 * onewire_device_t devices[4];
01267 *
01268 * // Search for devices and validate ROM codes
01269 * for (uint8_t i = 0; i < 4; i++) {
01270 *     if (rx_owewire_search(&devices[i].rom[0])) {
01271 *         // Validate ROM CRC
01272 *         uint32_t crc_out = 0U;
01273 *         (void)rx_crc8_maxim(devices[i].rom, 7, &crc_out);
01274 *         devices[i].valid = ((uint8_t)crc_out == devices[i].rom[7]);
01275 *
01276 *         if (devices[i].valid) {
01277 *             rx_log_info("OneWire", "Device %u: Family=0x%02X Serial=%02X%02X%02X%02X%02X%02X",
01278 *                 i, devices[i].rom[0],
01279 *                 devices[i].rom[6], devices[i].rom[5], devices[i].rom[4],
01280 *                 devices[i].rom[3], devices[i].rom[2], devices[i].rom[1]);
01281 *         } else {
01282 *             rx_log_error("OneWire", "Device %u: Invalid ROM CRC", i);
01283 *         }
01284 *     }
01285 * }
01286 * @endcode
01287 *
01288 * @par Example - Error Handling:
01289 * @code{.c}
01290 * uint32_t crc_out = 0U;
01291 * rx_err_t err = rx_crc8_maxim(nullptr, 0, &crc_out);
01292 * // Returns: k_rx_err_null_ptr (data is NULL)
01293 * (void)err;
01294 * @endcode
01295 *
01296 * @par Example - Manual CRC Verification (Algorithm Understanding):
01297 * @code{.c}
01298 * // Demonstrate CRC-8/Maxim algorithm step-by-step
01299 * uint8_t data[] = { 0x28, 0xFF, 0x64, 0x1F, 0x21, 0x16, 0x04 };
01300 *
01301 * // Method 1: Use library function
01302 * uint32_t crc_lib_out = 0U;

```

```

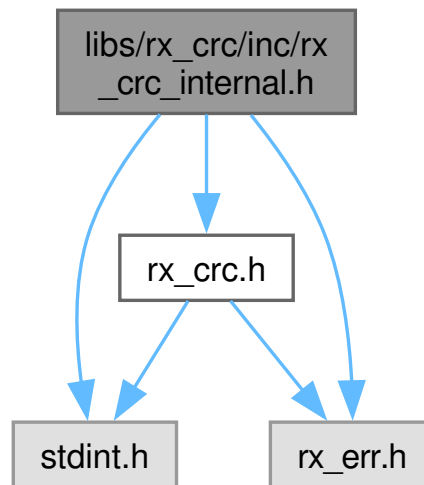
01303 * (void)rx_crc8_maxim(data, sizeof(data), &crc_lib_out);
01304 * uint8_t crc_lib = (uint8_t)crc_lib_out;
01305 * printf("Library CRC: 0x%02X\n", crc_lib);
01306 * // Output: Library CRC: 0xB3
01307 *
01308 * // Method 2: Manual bitwise computation (for algorithm understanding)
01309 * uint8_t crc_manual = 0x00; // CRC-8/Maxim initial value
01310 * for (uint8_t i = 0; i < sizeof(data); i++) {
01311 *     crc_manual ^= data[i];
01312 *     for (uint8_t bit = 0; bit < 8; bit++) {
01313 *         if (crc_manual & 0x01) {
01314 *             crc_manual = (uint8_t)((crc_manual » 1) ^ 0x8C); // reflected poly 0x31
01315 *         } else {
01316 *             crc_manual »= 1;
01317 *         }
01318 *     }
01319 * }
01320 * printf("Manual CRC: 0x%02X\n", crc_manual);
01321 * // Output: Manual CRC: 0xB3
01322 *
01323 * // Both produce identical result
01324 * assert(crc_lib == crc_manual);
01325 * @endcode
01326 *
01327 * @see rx_crc32_ieee() CRC-32 for frame protocols and Ethernet
01328 * @see rx_crc8.c CRC-8/Maxim implementation (software table)
01329 * @see rx_hcsr04.h OneWire interface for DS18B20 and similar devices
01330 *
01331 * @since Version 1.0.0
01332 */
01333 static inline rx_err_t rx_crc8_maxim(const uint8_t* data, uint32_t len, uint32_t* out)
01334 {
01335     static const rx_crc_config_t s_cfg = {
01336         .poly      = k_rx_crc_poly_crc8,
01337         .bit_order  = k_rx_crc_bit_order_lsb_first,
01338         .backend    = k_rx_crc_backend_software,
01339         .dma        = {.timeout_cycles = 0U},
01340     };
01341     return rx_crc_compute(&s_cfg, data, len, out);
01342 }
01343
01344 #ifdef __cplusplus
01345 }
01346 #endif

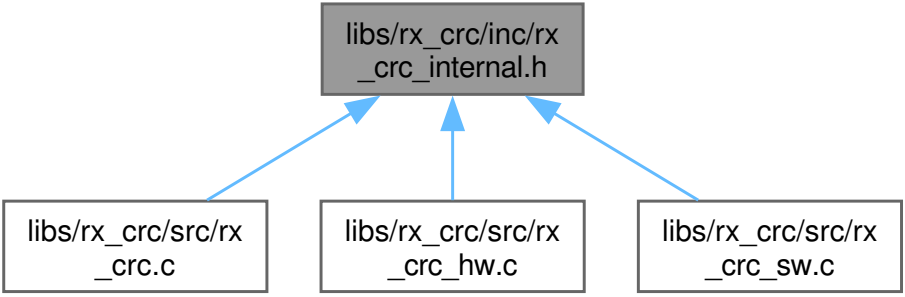
```

```

#include <stdint.h>
#include "rx_crc.h"
#include "rx_err.h"

```





*

*

```
rx_crc_compute()          (rx_crc.c - public dispatcher)
|
|--> internal_crc_sw_compute()  (rx_crc_sw.c - all 5 polys, any platform)
|
|--> internal_crc_hw_cpu_compute() (rx_crc_hw.c - RX72N only, #ifdef __RX__)
|
|--> internal_crc_hw_dma_compute() (rx_crc_hw.c - RX72N only, #ifdef __RX__)
```

```
enum rx_crc_dma_constants_t : uint32_t
```

--	--

```
00075         : uint32_t {
00076   k_crc_dma_threshold = 128U, /**< Min bytes to use DMA (below = hw_cpu fallback) */
00077 } rx_crc_dma_constants_t;
```

```
enum rx_crc_shared_constants_t : uint32_t
```


```
00059         : uint32_t {
00060   k_crc_len_min  = 1U,    /**< Minimum meaningful CRC data length (bytes) */
00061   k_crc_len_max  = 65535U, /**< Maximum CRC data length - matches DMA limit (bytes) */
00062   k_crc_idx_start = 0U,   /**< Standard loop start index (zero-based) */
00063 } rx_crc_shared_constants_t;
```

```
uint32_t internal_crc32_sw_update (  
    uint32_t crc,  
    const uint8_t * data,  
    uint32_t len)
```

in		
in		
in		>

>

```

00458 {
00459     if (len > k_crc_len_max) {
00460         return crc;
00461     }
00462     /* Un-finalize previous CRC to get internal accumulator state */
00463     uint32_t work = crc ^ k_crc32_final_xor;
00464     for (uint32_t i = 0U; i < len; i++) {
00465         work = s_crc32_table[(uint8_t)(work ^ data[i])] ^ (work » k_crc_byte_shift);
00466     }
00467     return work ^ k_crc32_final_xor;
00470 }
00471 }

```



```

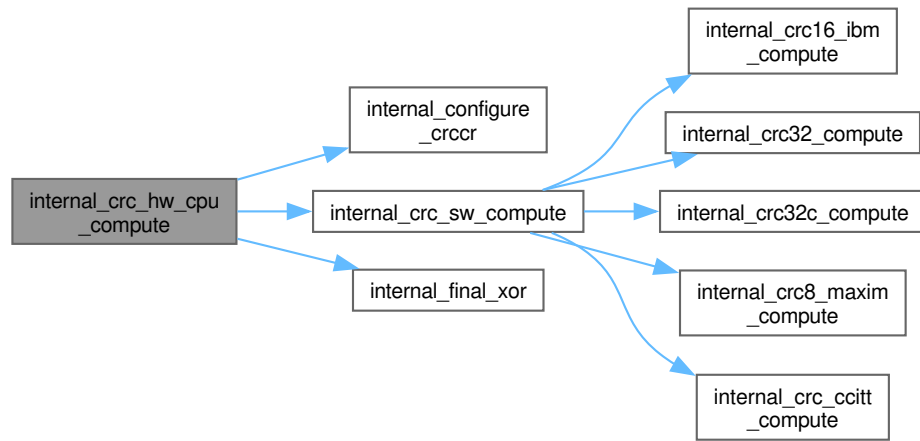
rx_err_t internal_crc_hw_cpu_compute (
    const rx_crc_config_t * config,
    const uint8_t * data,
    uint32_t len,
    uint32_t * result_out)  [nodiscard]

```

in		
in		>
in		
out		

*

```
00316 {
00317     if (config == nullptr || data == nullptr || result_out == nullptr) {
00318         return k_rx_err_null_ptr;
00319     }
00320
00321     /* CRC-8/Maxim: hardware GPS=0x00 uses poly 0x07, not 0x31 - must use SW */
00322     if (config->poly == k_rx_crc_poly_crc8) {
00323         /* Software CRC-8/Maxim only supports LSB-first bit order */
00324         if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00325             return k_rx_err_invalid_arg;
00326         }
00327         *result_out = internal_crc_sw_compute(config->poly, data, len);
00328         return k_rx_ok;
00329     }
00330
00331     internal_configure_crccr(config);
00332
00333     /* Feed data byte-by-byte (NASA Rule 2: bounded by k_crc_len_max) */
00334     volatile uint8_t* crcdir_byte = (volatile uint8_t*)&crc_regs()->crcdir;
00335     for (uint32_t i = k_crc_idx_start; i < k_crc_len_max; i++) {
00336         if (i >= len) {
00337             break;
00338         }
00339         *crcdir_byte = data[i];
00340     }
00341
00342     *result_out = crc_regs()->crcdor ^ internal_final_xor(config->poly);
00343     return k_rx_ok;
00344 }
```



```
rx_err_t internal_crc_hw_deinit (  
    void ) [nodiscard]
```

```

00271 {
00272     RX_CHECK_NULL_PTR(system_regs(), s_tag, "System registers not accessible");
00273
00274     /* Deinitialize DMA driver only if this module initialized it */
00275     if (s_dmaca_owned) {
00276         (void)rx_dmaca_deinit();
00277         s_dmaca_owned = false;
00278     }
00279
00280     /* Re-enable module stop (set MSTPCRB bit 23) */
00281     *prcr_reg() = k_rx_pcr_unlock_prc1;
00282     system_regs()->mstpcrb |= ((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00283     *prcr_reg() = k_rx_pcr_lock;
00284
00285     return k_rx_ok;
00286 }

```



```

rx_err_t internal_crc_hw_dma_compute (
    const rx_crc_config_t * config,
    const uint8_t * data,
    uint32_t len,
    uint32_t * result_out)  [nodiscard]

```

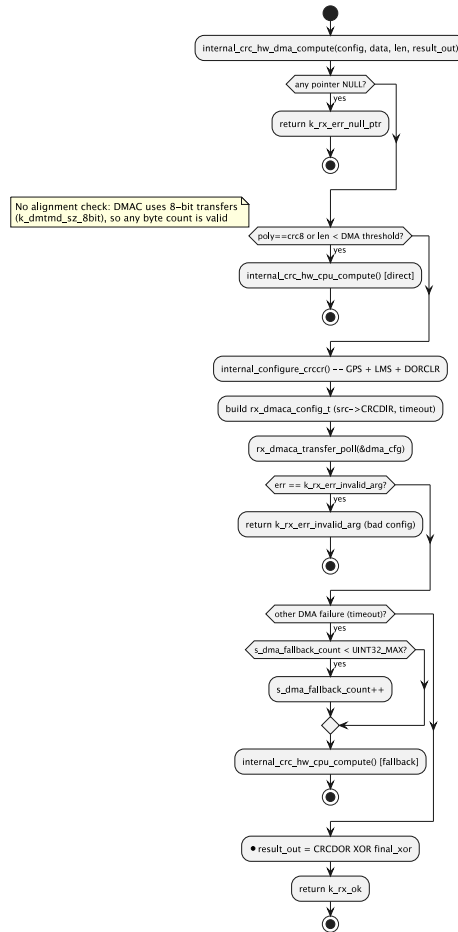
>

in		>
in		>
in		
out		

*

<

*

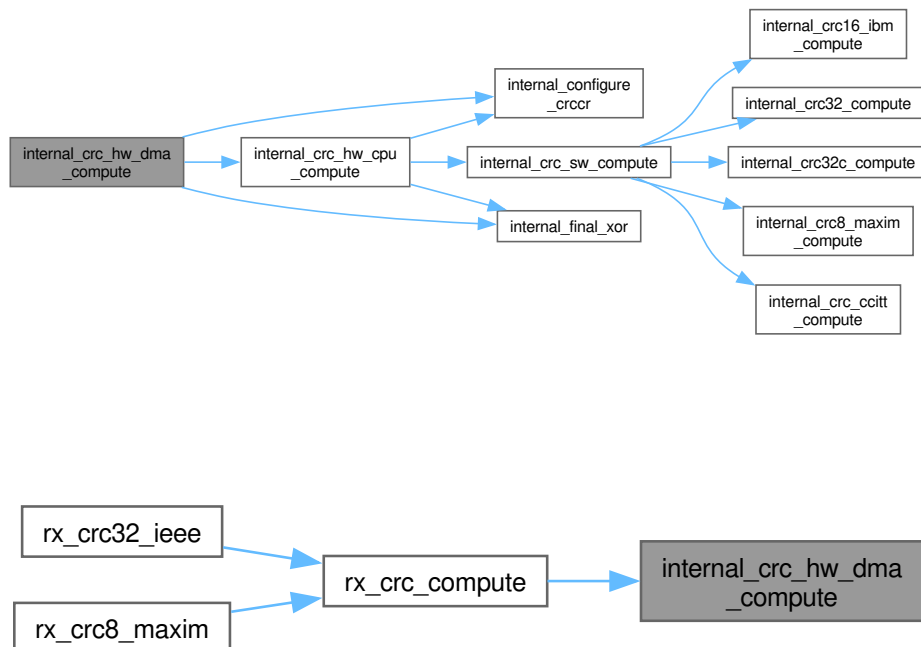


```
00410 {
00411     if (config == nullptr || data == nullptr || result_out == nullptr) {
00412         return k_rx_err_null_ptr;
00413     }
00414
00415     /* CRC-8/Maxim or small buffer: fall back to CPU-driven path */
00416     if (config->poly == k_rx_crc_poly_crc8 || len < k_crc_dma_threshold) {
00417         return internal_crc_hw_cpu_compute(config, data, len, result_out);
00418     }
00419
00420     /* No alignment requirement: DMAC is configured for 8-bit (k_dmtmd_sz bit)
```

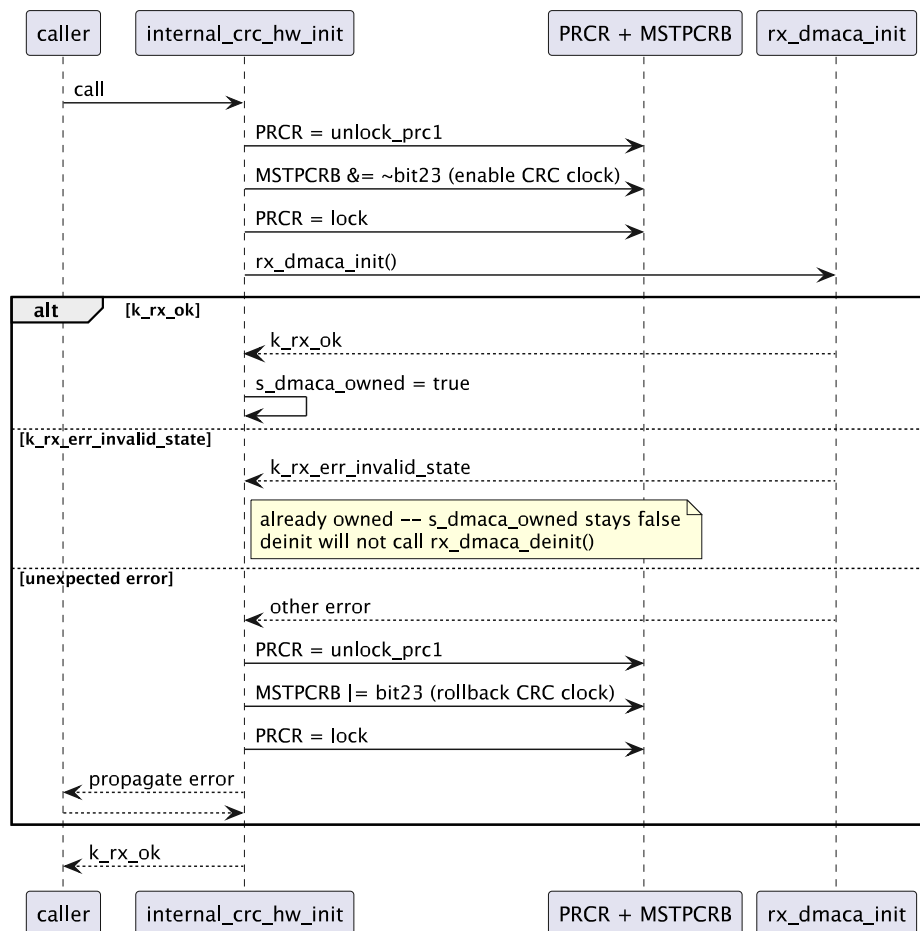
```

00421  * transfers in internal_configure_channel(), so CRCDIR receives one byte at
00422  * a time regardless of the CRC polynomial selected. */
00423  internal_configure_crccr(config);
00424
00425  /* Use config timeout if non-zero, otherwise use library default */
00426  uint32_t timeout = (config->dma.timeout_cycles != 0U) ? config->dma.timeout_cycles
00427                  : (uint32_t)k_dma_crc_timeout_cycles;
00428
00429  /* Destination: byte address of CRCDIR register (fixed, no increment) */
00430  const rx_dmaca_config_t dma_cfg = {
00431      .channel    = (uint8_t)k_dma_channel_crc,
00432      .src        = data,
00433      .len        = len,
00434      .dst_addr   = (uintptr_t)((volatile uint8_t*)&crc_regs()->crcdir),
00435      .timeout_cycles = timeout,
00436  };
00437
00438  rx_err_t err = rx_dmaca_transfer_poll(&dma_cfg);
00439  if (err == k_rx_err_invalid_arg) {
00440      /* Bad config (e.g. timeout_cycles out of range): surface to caller, do not fall back */
00441      return err;
00442  }
00443  if (err != k_rx_ok) {
00444      /* Transient DMA failure (timeout): record for observability (saturating), fall back to CPU */
00445      if (s_dma_fallback_count < UINT32_MAX) {
00446          s_dma_fallback_count++;
00447      }
00448      return internal_crc_hw_cpu_compute(config, data, len, result_out);
00449  }
00450
00451  *result_out = crc_regs()->crcdor ^ internal_final_xor(config->poly);
00452  return k_rx_ok;
00453 }

```



```
rx_err_t internal_crc_hw_init (
    void ) [nodiscard]
```



```

00226 {
00227   RX_CHECK_NULL_PTR(system_regs(), s_tag, "System registers not accessible");
00228   RX_CHECK_NULL_PTR(crc_regs(), s_tag, "CRC registers not accessible");
00229
00230   /* Unlock protection and enable CRC module clock (clear MSTPCRB bit 23) */
00231   *prcr_reg() = k_rx_prcr_unlock_prc1;
00232   system_regs()->mstpcrb &= ~(uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00233   *prcr_reg() = k_rx_prcr_lock;
00234
00235   /* Initialize DMA driver; treat k_rx_err_invalid_state as OK (already initialized),
00236    * propagate any unexpected error so callers know DMA is unavailable.
00237    * Track ownership: only deinit DMAC on teardown if we initialized it here.
00238    * On unexpected failure, roll back CRC clock to leave no partial state. */
00239   rx_err_t dma_ret = rx_dmaca_init();
00240   if (dma_ret == k_rx_ok) {
00241     s_dmaca_owned = true;
00242   } else if (dma_ret != k_rx_err_invalid_state) {
00243     /* Roll back: disable CRC clock before returning the error */
00244     *prcr_reg() = k_rx_prcr_unlock_prc1;
00245     system_regs()->mstpcrb |= ((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00246     *prcr_reg() = k_rx_prcr_lock;
00247     return dma_ret;
00248   }
00249
00250   return k_rx_ok;
00251 }

```

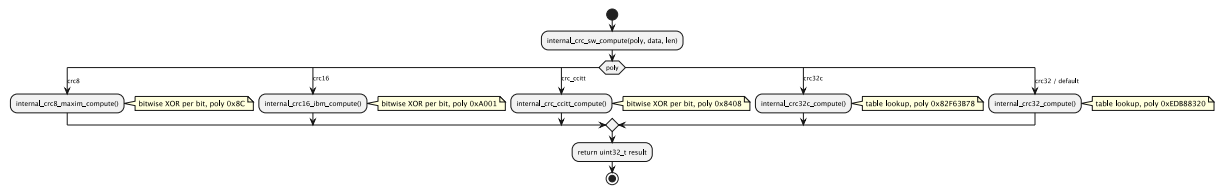


```

uint32_t internal_crc_sw_compute (
    rx_crc_poly_t poly,
    const uint8_t * data,
    uint32_t len)

```

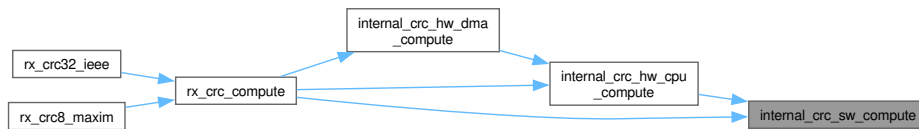
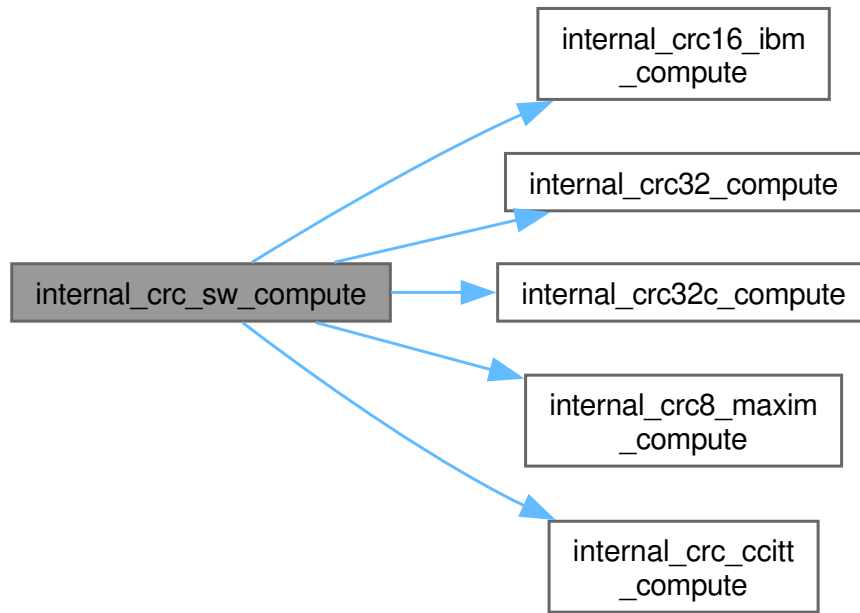
in		
in		
in		>



```

00415 {
00416     switch (poly) {
00417     case k_rx_crc_poly_crc8:
00418         return (uint32_t)internal_crc8_maxim_compute(data, len);
00419     case k_rx_crc_poly_crc16:
00420         return (uint32_t)internal_crc16_ibm_compute(data, len);
00421     case k_rx_crc_poly_crc_ccitt:
00422         return (uint32_t)internal_crc_ccitt_compute(data, len);
00423     case k_rx_crc_poly_crc32c:
00424         return internal_crc32c_compute(data, len);
00425     case k_rx_crc_poly_crc32:
00426     default:
00427         return internal_crc32_compute(data, len);
00428     }
00429 }

```



```

00001 /**
00002  * @file rx_crc_internal.h
00003  * @brief Internal CRC API - Software and Hardware Backend Declarations
00004  *
00005  * @details
00006  * Internal-only header used by rx_crc.c, rx_crc_sw.c, and rx_crc_hw.c.
00007  * Declares the internal dispatch functions for each backend. Application code
00008  * must use rx_crc.h instead.
00009  *
00010  * ## Architecture
00011  *
00012  * ```
00013  * rx_crc_compute()          (rx_crc.c - public dispatcher)
00014  * |
00015  * +--> internal_crc_sw_compute()  (rx_crc_sw.c - all 5 polys, any platform)
00016  * |
00017  * +--> internal_crc_hw_cpu_compute() (rx_crc_hw.c - RX72N only, #ifdef __RX__)
00018  * |
00019  * +--> internal_crc_hw_dma_compute() (rx_crc_hw.c - RX72N only, #ifdef __RX__)
00020  * ```
00021  *
00022  * @see rx_crc.h   Public API (use this in application code)
00023  * @see rx_crc_sw.c Software implementations
00024  * @see rx_crc_hw.c Hardware implementations
00025  *
00026  * @author Locked, Inc.
00027  * @date 2026-03-05
00028  * @copyright Copyright (c) 2026 Locked Inc.
00029  * SPDX-License-Identifier: MIT
00030  * @since Version 1.0.0
00031  */
00032
00033 #pragma once
00034
00035 #include <stdint.h>
00036
00037 #include "rx_crc.h"
00038 #include "rx_err.h"
00039
00040 #ifndef __cplusplus

```

```

00041 extern "C" {
00042 #endif
00043
00044 /*
=====
00045 * Shared Constants
00046 *
=====
00047 */
00048
00049 /**
00050  * @enum rx_crc_shared_constants_t
00051  * @brief Shared constants for CRC validation and NASA-compliant bounded loops
00052  * @details
00053  * Defines loop bounds and starting indices used by all CRC implementations.
00054  * Satisfies NASA Power of 10 Rule 2 (bounded loops).
00055  *
00056  * @since Version 1.0.0
00057  */
00058
00059 typedef enum : uint32_t {
00060     k_crc_len_min    = 1U, /**< Minimum meaningful CRC data length (bytes) */
00061     k_crc_len_max    = 65535U, /**< Maximum CRC data length - matches DMA limit (bytes) */
00062     k_crc_idx_start = 0U, /**< Standard loop start index (zero-based) */
00063 } rx_crc_shared_constants_t;
00064
00065 /**
00066  * @enum rx_crc_dma_constants_t
00067  * @brief DMA backend threshold and related constants
00068  * @details
00069  * The DMA backend falls back to hw_cpu for buffers below k_crc_dma_threshold
00070  * because DMA setup overhead exceeds the computation cost for small transfers.
00071  *
00072  * @since Version 1.0.0
00073  */
00074
00075 typedef enum : uint32_t {
00076     k_crc_dma_threshold = 128U, /**< Min bytes to use DMA (below = hw_cpu fallback) */
00077 } rx_crc_dma_constants_t;
00078
00079 /*
=====
00080 * Software Backend (rx_crc_sw.c) - All Platforms
00081 *
=====
00082 */
00083
00084 /**
00085  * @brief Compute CRC using software for the given polynomial
00086  * @details
00087  * Portable software implementation supporting all five polynomials. Uses
00088  * lookup tables for CRC-32 and CRC-32C, and bitwise algorithms for the
00089  * others. Always available (no hardware dependency).
00090  *
00091  * @param[in] poly CRC polynomial selection
00092  * @param[in] data Input data buffer (must be non-NULL)
00093  * @param[in] len Number of bytes (must be >= 1)
00094  *
00095  * @return CRC result (8, 16, or 32 significant bits depending on polynomial)
00096  *
00097  * @pre data must point to at least len bytes
00098  * @pre len must be in [k_crc_len_min, k_crc_len_max]
00099  * @post No side effects (pure function, no heap allocation, no global state modified)
00100  * @post Return value has at most 32 significant bits; upper bits zero for CRC-8 and CRC-16
00101  *
00102  * @note Thread-safe (stateless, read-only tables)
00103  * @since Version 1.0.0
00104  */
00105
00106 uint32_t internal_crc_sw_compute(rx_crc_poly_t poly, const uint8_t* data, uint32_t len);
00107
00108 /**
00109  * @brief Update CRC-32/IEEE in-progress value with additional data
00110  * @details
00111  * Continues an IEEE 802.3 CRC-32 from a finalized partial result. Un-finalizes
00112  * the input CRC, feeds the new data, then re-finalizes. Used by rx_crc32_update().
00113  *
00114  * @param[in] crc Previous finalized CRC-32 value
00115  * @param[in] data Additional data buffer (must be non-NULL)
00116  * @param[in] len Number of bytes (must be >= 1)
00117  *
00118  * @return Updated finalized CRC-32 value
00119  *
00120  * @pre data must point to at least len bytes, or be NULL (pass-through)
00121  * @pre crc must be 0U (first call) or the finalized CRC-32/IEEE value returned
00122  * by a prior rx_crc32_update() / internal_crc32_sw_update() call
00123  */

```

```

00124 * @post Returns finalized CRC-32/IEEE value; no heap or global state modified
00125 * @post No side effects (pure computation, read-only table access)
00126 *
00127 * @note Thread-safe (stateless, read-only table)
00128 * @since Version 1.0.0
00129 */
00130 uint32_t internal_crc32_sw_update(uint32_t crc, const uint8_t* data, uint32_t len);
00131
00132 /*
=====
00133 * Hardware Backend (rx_crc_hw.c) - RX72N Only
00134 *
=====
00135 */
00136
00137 #ifdef __RX__
00138
00139 /**
00140 * @brief Initialize RX72N CRC peripheral (enable CRCA module clock)
00141 *
00142 * @return rx_err_t k_rx_ok on success
00143 *
00144 * @pre System clock (PCLKB) must be configured and running
00145 * @pre PRCR register must be writable (caller must not hold PRCR lock)
00146 * @post CRCA module clock enabled (MSTPCRB bit 23 cleared)
00147 * @post DMAC driver initialized (rx_dmaca_init() called) if not already running
00148 *
00149 * @since Version 1.0.0
00150 */
00151 [[nodiscard]] rx_err_t internal_crc_hw_init(void);
00152
00153 /**
00154 * @brief Deinitialize RX72N CRC peripheral (disable CRCA module clock)
00155 *
00156 * @return rx_err_t k_rx_ok on success
00157 *
00158 * @pre internal_crc_hw_init() must have been called successfully before this
00159 * @pre No CRC or DMA transfers may be in progress at time of call
00160 * @post CRCA module clock disabled (MSTPCRB bit 23 set)
00161 * @post DMAC driver deinitialized (rx_dmaca_deinit() called) if owned by this module
00162 *
00163 * @since Version 1.0.0
00164 */
00165 [[nodiscard]] rx_err_t internal_crc_hw_deinit(void);
00166
00167 /**
00168 * @brief Compute CRC using RX72N CRC peripheral, CPU-driven byte loop
00169 *
00170 * @details
00171 * Falls back to software for CRC-8/Maxim (hardware polynomial mismatch).
00172 * Configures CRCCR with GPS and LMS, feeds data byte-by-byte, reads CRCDOR.
00173 *
00174 * @param[in] config CRC configuration
00175 * @param[in] data Input data (non-NULL, len >= 1)
00176 * @param[in] len Number of bytes
00177 * @param[out] result_out CRC result
00178 *
00179 * @return rx_err_t k_rx_ok on success
00180 *
00181 * @pre CRC peripheral must be initialized via internal_crc_hw_init()
00182 * @pre config, data, and result_out must not be NULL
00183 * @post *result_out contains the computed CRC on k_rx_ok
00184 * @post CRC peripheral registers returned to idle state (CRCDOR readable)
00185 *
00186 * @since Version 1.0.0
00187 */
00188 [[nodiscard]] rx_err_t internal_crc_hw_cpu_compute(const rx_crc_config_t* config,
00189 const uint8_t* data,
00190 uint32_t len,
00191 uint32_t* result_out);
00192
00193 /**
00194 * @brief Compute CRC using RX72N CRC peripheral, DMA-driven transfer
00195 *
00196 * @details
00197 * Falls back to hw_cpu for:
00198 * - CRC-8/Maxim (hardware polynomial mismatch)
00199 * - len < k_crc_dma_threshold (DMA overhead not worth it)
00200 *
00201 * Configures CRCCR, then calls rx_dmaca_transfer_poll() to move src -> CRCDIR.
00202 *
00203 * @param[in] config CRC configuration (dma.timeout_cycles used if > 0)
00204 * @param[in] data Input data (non-NULL, len >= 1)
00205 * @param[in] len Number of bytes
00206 * @param[out] result_out CRC result
00207 *
00208 * @return rx_err_t

```

```

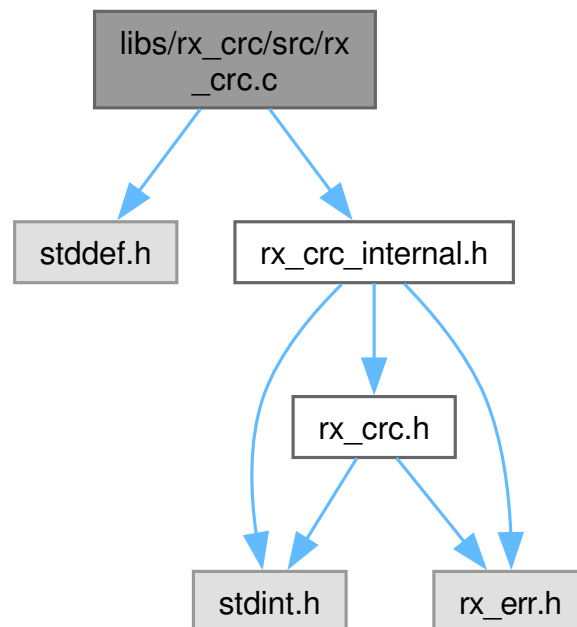
00209 * @retval k_rx_ok          Success
00210 * @retval k_rx_err_null_ptr config, data, or result_out is NULL
00211 * @retval k_rx_err_invalid_arg CRC-32/CRC-32C buffer pointer or length not aligned
00212 *                          to k_hw_crc32_alignment_bytes
00213 * @retval k_rx_err_timeout  DMA timed out
00214 *
00215 * @pre CRC and DMAC peripherals must be initialized (internal_crc_hw_init() called)
00216 * @pre For CRC-32/CRC-32C: data pointer and len must each be aligned to
00217 *      k_hw_crc32_alignment_bytes (4 bytes); unaligned inputs return
00218 *      k_rx_err_invalid_arg before touching the peripheral
00219 * @post *result_out contains the final CRC with XOR applied on k_rx_ok;
00220 *      result_out is not written on any error return
00221 * @post DMA channel is idle on return (transfer complete, or aborted on timeout)
00222 *
00223 * @note NOT thread-safe. Unlike internal_crc_sw_compute and
00224 *      internal_crc32_sw_update (which are stateless and re-entrant), this
00225 *      function programs shared DMA channel hardware and writes result_out.
00226 *      Callers must serialize all concurrent invocations. Concurrent calls
00227 *      will corrupt the in-flight DMA transfer and produce wrong result_out.
00228 *
00229 * @see k_hw_crc32_alignment_bytes Alignment requirement for CRC-32/CRC-32C (defined in rx_crc.h)
00230 *
00231 * @since Version 1.0.0
00232 */
00233 [[nodiscard]] rx_err_t internal_crc_hw_dma_compute(const rx_crc_config_t* config,
00234                                                    const uint8_t*      data,
00235                                                    uint32_t      len,
00236                                                    uint32_t*      result_out);
00237
00238 #endif /* __RX__ */
00239
00240 #ifdef __cplusplus
00241 }
00242 #endif

```

```

#include <stddef.h>
#include "rx_crc_internal.h"

```



*

*

```
uint32_t rx_crc32_update (  
    uint32_t crc,  
    const uint8_t * data,  
    uint32_t len)
```

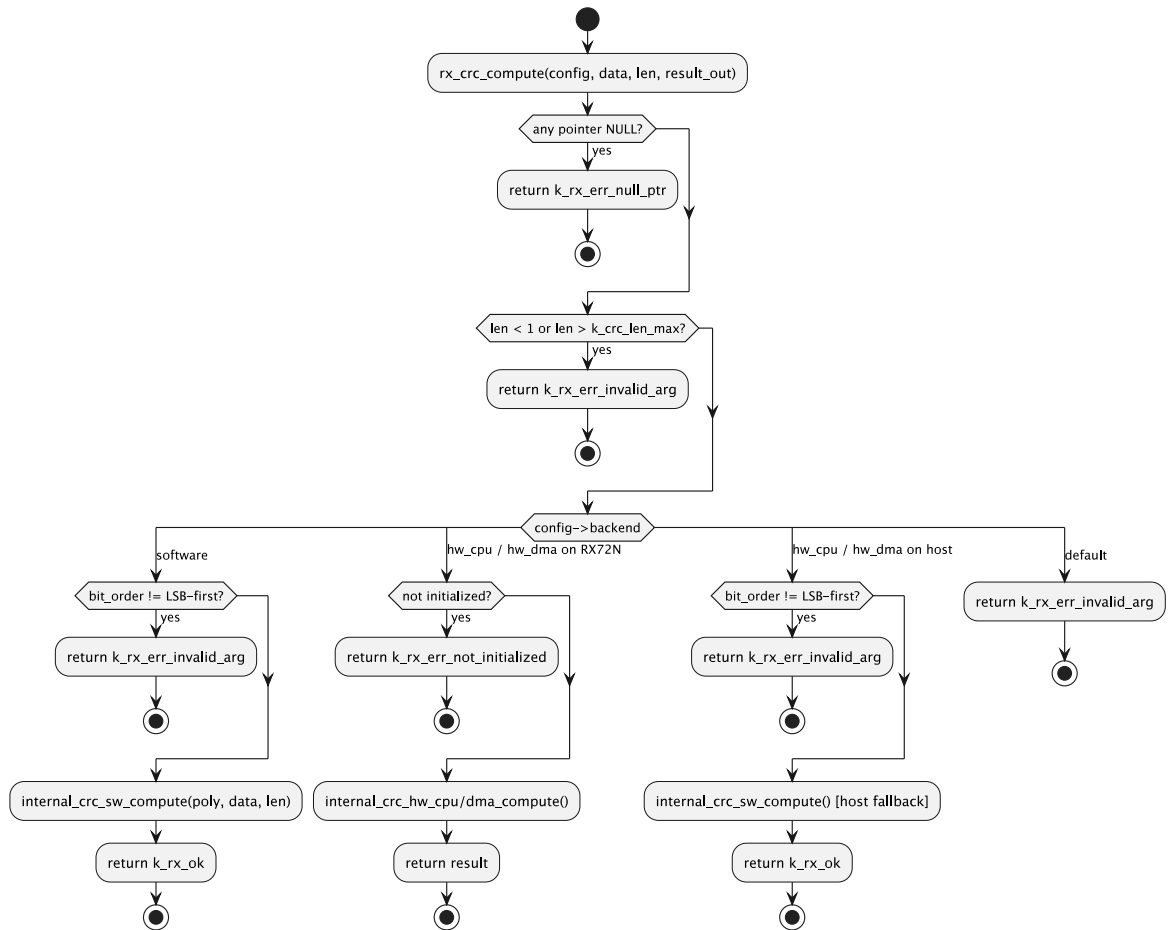
>

```
00257 {  
00258     if (data == nullptr || len == 0U) {  
00259         return crc;  
00260     }  
00261     return internal_crc32_sw_update(crc, data, len);  
00262 }  
00263 }
```



```
rx_err_t rx_crc_compute (  
    const rx_crc_config_t * config,  
    const uint8_t * data,  
    uint32_t len,  
    uint32_t * result_out)  [nodiscard]
```

*



```

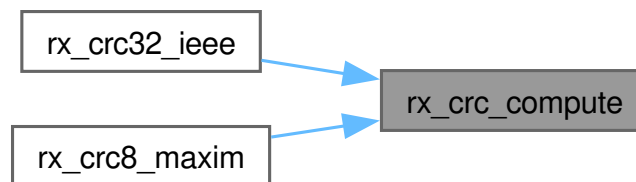
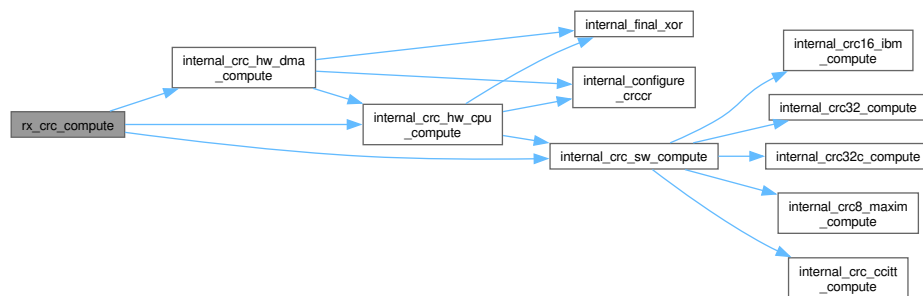
00185 {
00186     if (config == nullptr || data == nullptr || result_out == nullptr) {
00187         return k_rx_err_null_ptr;
00188     }
00189
00190     if (len < k_crc_len_min || len > k_crc_len_max) {
00191         return k_rx_err_invalid_arg;
00192     }
00193
00194     if (config->poly > k_rx_crc_poly_crc32c || config->bit_order > k_rx_crc_bit_order_msb_first) {
00195         return k_rx_err_invalid_arg;

```

```

00196 }
00197
00198 switch (config->backend) {
00199     case k_rx_crc_backend_software:
00200         if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00201             return k_rx_err_invalid_arg;
00202         }
00203         *result_out = internal_crc_sw_compute(config->poly, data, len);
00204         return k_rx_ok;
00205
00206 #ifdef __RX__
00207     case k_rx_crc_backend_hw_cpu:
00208         if (!s_crc_initialized) {
00209             return k_rx_err_not_initialized;
00210         }
00211         return internal_crc_hw_cpu_compute(config, data, len, result_out);
00212
00213     case k_rx_crc_backend_hw_dma:
00214         if (!s_crc_initialized) {
00215             return k_rx_err_not_initialized;
00216         }
00217         return internal_crc_hw_dma_compute(config, data, len, result_out);
00218 #else
00219     /* Host build: hardware backends not available - fall back to software */
00220     case k_rx_crc_backend_hw_cpu:
00221         [[fallthrough]]; /* hw_cpu has same software fallback as hw_dma on host */
00222     case k_rx_crc_backend_hw_dma:
00223         if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00224             return k_rx_err_invalid_arg;
00225         }
00226         *result_out = internal_crc_sw_compute(config->poly, data, len);
00227         return k_rx_ok;
00228 #endif
00229
00230     default:
00231         return k_rx_err_invalid_arg;
00232 }
00233 }

```



```
rx_err_t rx_crc_deinit (  
    void ) [nodiscard]
```

```
00092 {  
00093     if (!s_crc_initialized) {  
00094         return k_rx_err_invalid_state;  
00095     }  
00096  
00097     #ifdef __RX__  
00098         rx_err_t err = internal_crc_hw_deinit();  
00099         if (err != k_rx_ok) {  
00100             return err;  
00101         }  
00102     #endif  
00103  
00104     s_crc_initialized = false;  
00105     return k_rx_ok;  
00106 }
```



```
rx_err_t rx_crc_init (  
    void ) [nodiscard]
```

```
00058 {  
00059     if (s_crc_initialized) {  
00060         return k_rx_err_invalid_state;  
00061     }  
00062  
00063     #ifdef __RX__  
00064     rx_err_t err = internal_crc_hw_init();  
00065     if (err != k_rx_ok) {  
00066         return err;  
00067     }  
00068     #endif  
00069  
00070     s_crc_initialized = true;  
00071     return k_rx_ok;  
00072 }
```

rx_crc_init



internal_crc_hw_init

```
bool s_crc_initialized = false [static]
```

```

00001 /**
00002  * @file rx_crc.c
00003  * @brief CRC Public API Dispatcher
00004  *
00005  * @details
00006  * Implements the public rx_crc_* API. Routes to software, hw_cpu, or hw_dma
00007  * backends via rx_crc_compute(). On host builds (no __RX__), hardware backends
00008  * fall back to software automatically.
00009  *
00010  * @see rx_crc.h      Public API
00011  * @see rx_crc_sw.c   Software backend
00012  * @see rx_crc_hw.c   Hardware backend (RX72N only)
00013  *
00014  * @author Locked, Inc.
00015  * @date 2026-03-05
00016  * @copyright Copyright (c) 2026 Locked Inc.
00017  * SPDX-License-Identifier: MIT
00018  */
00019
00020 #include <stddef.h>
00021
00022 #include "rx_crc_internal.h"
00023
00024 /*
=====
00025  * Module State
00026  *
=====
00027  */
00028
00029 /**
00030  * @brief Tracks whether CRC hardware has been initialized
00031  * @warning Not thread-safe; call rx_crc_init() single-threaded during init
00032  */
00033 static bool s_crc_initialized = false;
00034
00035 /*
=====
00036  * Public API - Lifecycle
00037  *
=====
00038  */
00039
00040 /**
00041  * @brief Initialize the CRC module
00042  *
00043  * @details
00044  * Sets the initialized flag and, on RX72N hardware, enables the CRC peripheral
00045  * clock and DMAC via internal_crc_hw_init(). On host builds, the hardware path
00046  * is skipped and the module enters the ready state immediately.
00047  *
00048  *
00049  * @pre Not already initialized
00050  * @pre System clock (PCLKA) must be running on RX72N
00051  * @post s_crc_initialized = true
00052  * @post CRC hardware peripheral enabled (RX72N only)
00053  *
00054  * @note Not thread-safe; call during single-threaded system init
00055  * @since Version 1.0.0
00056  */
00057 rx_err_t rx_crc_init(void)
00058 {
00059     if (s_crc_initialized) {
00060         return k_rx_err_invalid_state;
00061     }
00062
00063     #ifdef __RX__
00064         rx_err_t err = internal_crc_hw_init();
00065         if (err != k_rx_ok) {
00066             return err;
00067         }
00068     #endif
00069
00070     s_crc_initialized = true;
00071     return k_rx_ok;
00072 }
00073
00074 /**
00075  * @brief Deinitialize the CRC module
00076  *
00077  * @details
00078  * Clears the initialized flag and, on RX72N hardware, disables the CRC
00079  * peripheral clock and releases DMAC ownership via internal_crc_hw_deinit().
00080  * On host builds, the hardware path is skipped.
00081  *
00082  */

```

```

00083 * @pre Module must be initialized via rx_crc_init()
00084 * @pre No concurrent CRC computations in progress
00085 * @post s_crc_initialized = false
00086 * @post CRC hardware peripheral disabled (RX72N only)
00087 *
00088 * @note Not thread-safe; call during single-threaded system teardown
00089 * @since Version 1.0.0
00090 */
00091 rx_err_t rx_crc_deinit(void)
00092 {
00093     if (!s_crc_initialized) {
00094         return k_rx_err_invalid_state;
00095     }
00096
00097     #ifdef __RX__
00098         rx_err_t err = internal_crc_hw_deinit();
00099         if (err != k_rx_ok) {
00100             return err;
00101         }
00102     #endif
00103
00104     s_crc_initialized = false;
00105     return k_rx_ok;
00106 }
00107
00108 /*
=====
00109 * Public API - Computation
00110 *
=====
00111 */
00112
00113 /**
00114 * @brief Dispatch CRC computation to the configured backend
00115 *
00116 * @details
00117 * Validates inputs then routes to one of three backends:
00118 * - software: always available; uses bitwise algorithm; only supports LSB-first
00119 *   (reflected) bit order; returns k_rx_err_invalid_arg for MSB-first requests
00120 * - hw_cpu: RX72N CRC peripheral driven byte-by-byte; falls back to software on host
00121 * - hw_dma: RX72N CRC peripheral via DMAC; falls back to software on host
00122 *
00123 * @note The software backend only supports k_rx_crc_bit_order_lsb_first. All
00124 *       software CRC implementations use the reflected (LSB-first) form.
00125 *       Requesting k_rx_crc_bit_order_msb_first with the software backend
00126 *       returns k_rx_err_invalid_arg.
00127 *
00128 *
00129 *
00130 * @pre config, data, result_out must not be NULL
00131 * @pre len must be in [1, k_crc_len_max]
00132 * @post *result_out contains the computed CRC on k_rx_ok
00133 * @post No state is modified (pure computation)
00134 *
00135 * @par Activity Diagram:
00136 * @startuml
00137     start
00138     :rx_crc_compute(config, data, len, result_out);
00139     if (any pointer NULL?) then (yes)
00140         :return k_rx_err_null_ptr;
00141     stop
00142     endif
00143     if (len < 1 or len > k_crc_len_max?) then (yes)
00144         :return k_rx_err_invalid_arg;
00145     stop
00146     endif
00147     switch (config->backend)
00148     case (software)
00149         if (bit_order != LSB-first?) then (yes)
00150             :return k_rx_err_invalid_arg;
00151         stop
00152         endif
00153         :internal_crc_sw_compute(poly, data, len);
00154         :return k_rx_ok;
00155         stop
00156     case (hw_cpu / hw_dma on RX72N)
00157         if (not initialized?) then (yes)
00158             :return k_rx_err_not_initialized;
00159         stop
00160         endif
00161         :internal_crc_hw_cpu/dma_compute();
00162         :return result;
00163         stop
00164     case (hw_cpu / hw_dma on host)
00165         if (bit_order != LSB-first?) then (yes)
00166             :return k_rx_err_invalid_arg;
00167         stop

```

```

00168 *   endif
00169 *   :internal_crc_sw_compute() [host fallback];
00170 *   :return k_rx_ok;
00171 *   stop
00172 * case (default)
00173 *   :return k_rx_err_invalid_arg;
00174 *   stop
00175 * endswitch
00176 * @enduml
00177 *
00178 * @note Thread-safe for software backend; not thread-safe for hw_cpu/hw_dma
00179 * @since Version 1.0.0
00180 */
00181 rx_err_t rx_crc_compute(const rx_crc_config_t* config,
00182                        const uint8_t* data,
00183                        uint32_t len,
00184                        uint32_t* result_out)
00185 {
00186     if (config == nullptr || data == nullptr || result_out == nullptr) {
00187         return k_rx_err_null_ptr;
00188     }
00189
00190     if (len < k_crc_len_min || len > k_crc_len_max) {
00191         return k_rx_err_invalid_arg;
00192     }
00193
00194     if (config->poly > k_rx_crc_poly_crc32c || config->bit_order > k_rx_crc_bit_order_msb_first) {
00195         return k_rx_err_invalid_arg;
00196     }
00197
00198     switch (config->backend) {
00199         case k_rx_crc_backend_software:
00200             if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00201                 return k_rx_err_invalid_arg;
00202             }
00203             *result_out = internal_crc_sw_compute(config->poly, data, len);
00204             return k_rx_ok;
00205
00206 #ifdef __RX__
00207         case k_rx_crc_backend_hw_cpu:
00208             if (!s_crc_initialized) {
00209                 return k_rx_err_not_initialized;
00210             }
00211             return internal_crc_hw_cpu_compute(config, data, len, result_out);
00212
00213         case k_rx_crc_backend_hw_dma:
00214             if (!s_crc_initialized) {
00215                 return k_rx_err_not_initialized;
00216             }
00217             return internal_crc_hw_dma_compute(config, data, len, result_out);
00218 #else
00219         /* Host build: hardware backends not available - fall back to software */
00220         case k_rx_crc_backend_hw_cpu:
00221             [[fallthrough]]; /* hw_cpu has same software fallback as hw_dma on host */
00222         case k_rx_crc_backend_hw_dma:
00223             if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00224                 return k_rx_err_invalid_arg;
00225             }
00226             *result_out = internal_crc_sw_compute(config->poly, data, len);
00227             return k_rx_ok;
00228 #endif
00229
00230         default:
00231             return k_rx_err_invalid_arg;
00232     }
00233 }
00234
00235 /**
00236  * @brief Incremental CRC-32/IEEE update (software only -- hardware cannot seed CRC state)
00237  *
00238  * @details
00239  * Delegates to internal_crc32_sw_update() after null/zero-length guard checks.
00240  * Passes NULL or zero-length data through unchanged (returns crc unmodified).
00241  *
00242  * @note The correct seed for the first call is 0U (not 0xFFFFFFFF). Internally,
00243  *       the function un-finalizes by XOR with 0xFFFFFFFF, so crc=0 produces the
00244  *       correct CRC-32 initial accumulator state of 0xFFFFFFFF.
00245  *
00246  *
00247  *
00248  * @pre data must point to len valid bytes, or be NULL (pass-through)
00249  * @pre len must be > 0 for any update to occur
00250  * @post Returns finalized CRC-32 value covering all bytes processed so far
00251  * @post No state is modified (pure computation)
00252  *
00253  * @note Always uses software CRC; hardware cannot resume a partially computed CRC
00254  * @since Version 1.0.0

```

```

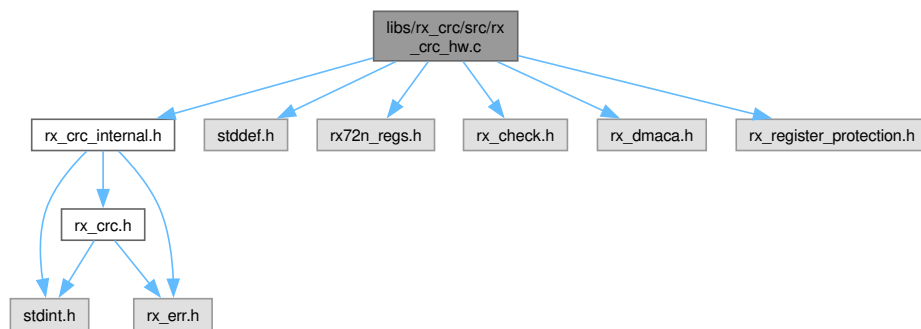
00255 */
00256 uint32_t rx_crc32_update(uint32_t crc, const uint8_t* data, uint32_t len)
00257 {
00258     if (data == nullptr || len == 0U) {
00259         return crc;
00260     }
00261     return internal_crc32_sw_update(crc, data, len);
00262 }
00263 }

```

```

#include "rx_crc_internal.h"
#include <stddef.h>
#include "rx72n_regs.h"
#include "rx_check.h"
#include "rx_dmaca.h"
#include "rx_register_protection.h"

```



*

*

		>

enum rx_crc_hw_alignment_t : uint8_t

--	--

00078 : uint8_t {
00079 k_hw_crc32_alignment_bytes = 4U, /**< CRC-32/CRC-32C require 4-byte aligned buffer and length */
00080 } rx_crc_hw_alignment_t;

```
enum rx_crc_hw_crccr_t : uint8_t
```


```
00053         : uint8_t {
00054   k_hw_crccr_lms    = k_crc_crccr_lms,    /**< Bit 6: LSB-first mode */
00055   k_hw_crccr_dorclr = k_crc_crccr_dorclr, /**< Bit 7: clear output register */
00056 } rx_crc_hw_crccr_t;
```

```
enum rx_crc_hw_final_xor_t : uint32_t
```


```
00061         : uint32_t {
00062   k_hw_crc32_final_xor = 0xFFFFFFFFU, /**< IEEE 802.3 / CRC-32C final XOR */
00063   k_hw_crc16_final_xor = 0x00000000U, /**< CRC-16/IBM: no final XOR */
00064   k_hw_ccitt_final_xor = 0x00000000U, /**< CRC-CCITT/Kermit: no final XOR */
00065 } rx_crc_hw_final_xor_t;
```

```
enum rx_crc_hw_mstpcrb_t : uint8_t
```

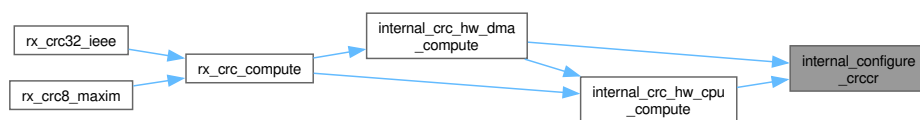

```
00070         : uint8_t {
00071   k_hw_mstpb_crc_bit = k_mstpb_crc, /**< Bit 23: CRC module stop in MSTPCRB */
00072   k_hw_bit_one       = 1U,          /**< Single bit value for shift operations */
00073 } rx_crc_hw_mstpcrb_t;
```

```
void internal_configure_crccr (
    const rx_crc_config_t * config)  [static]
```

```
>
```

```
>>
```

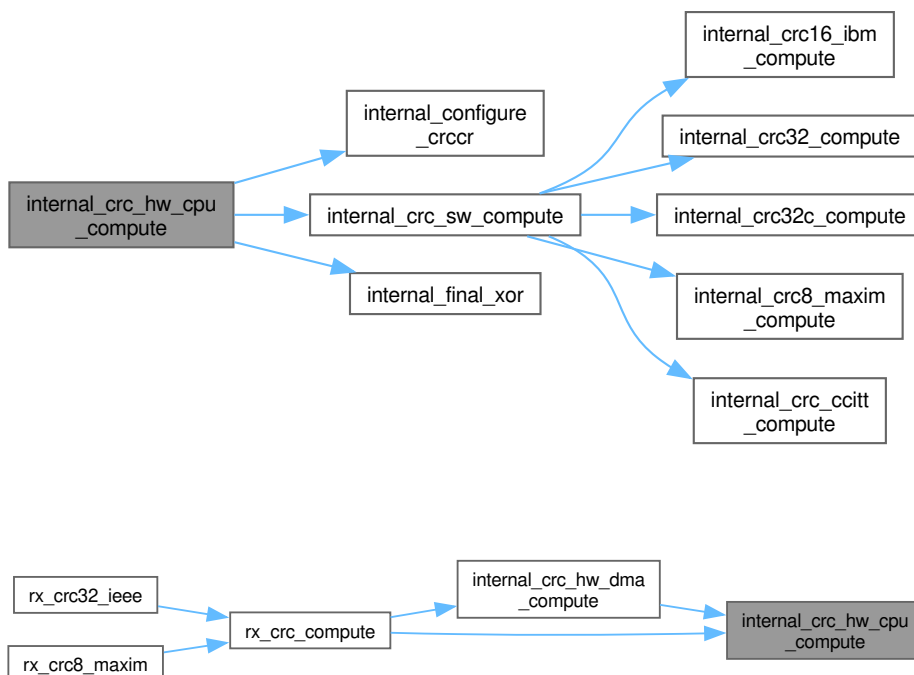
```
00165 {
00166     uint8_t lms = (config->bit_order == k_rx_crc_bit_order_lsb_first) ? (uint8_t)k_hw_crccr_lms : 0U;
00167
00168     /* GPS = poly enum value (0x01..0x04 maps directly to CRCCR GPS field) */
00169     uint8_t gps = (uint8_t)config->poly;
00170
00171     crc_regs()->crccr = (uint8_t)(gps | lms | (uint8_t)k_hw_crccr_dorclr);
00172 }
```



```
rx_err_t internal_crc_hw_cpu_compute (
    const rx_crc_config_t * config,
    const uint8_t * data,
    uint32_t len,
    uint32_t * result_out)  [nodiscard]
```

*

```
00316 {
00317     if (config == nullptr || data == nullptr || result_out == nullptr) {
00318         return k_rx_err_null_ptr;
00319     }
00320
00321     /* CRC-8/Maxim: hardware GPS=0x00 uses poly 0x07, not 0x31 - must use SW */
00322     if (config->poly == k_rx_crc_poly_crc8) {
00323         /* Software CRC-8/Maxim only supports LSB-first bit order */
00324         if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00325             return k_rx_err_invalid_arg;
00326         }
00327         *result_out = internal_crc_sw_compute(config->poly, data, len);
00328         return k_rx_ok;
00329     }
00330
00331     internal_configure_crccr(config);
00332
00333     /* Feed data byte-by-byte (NASA Rule 2: bounded by k_crc_len_max) */
00334     volatile uint8_t* crcdir_byte = (volatile uint8_t*)&crc_regs()->crcdir;
00335     for (uint32_t i = k_crc_idx_start; i < k_crc_len_max; i++) {
00336         if (i >= len) {
00337             break;
00338         }
00339         *crcdir_byte = data[i];
00340     }
00341
00342     *result_out = crc_regs()->crcdor ^ internal_final_xor(config->poly);
00343     return k_rx_ok;
00344 }
```



```
rx_err_t internal_crc_hw_deinit (  
    void ) [nodiscard]
```

```
00271 {  
00272     RX_CHECK_NULL_PTR(system_regs(), s_tag, "System registers not accessible");  
00273  
00274     /* Deinitialize DMA driver only if this module initialized it */  
00275     if (s_dmaca_owned) {  
00276         (void)rx_dmaca_deinit();  
00277         s_dmaca_owned = false;  
00278     }  
00279  
00280     /* Re-enable module stop (set MSTPCRB bit 23) */  
00281     *prcr_reg() = k_rx_prcr_unlock_prc1;  
00282     system_regs()->mstpcrb |= ((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);  
00283     *prcr_reg() = k_rx_prcr_lock;  
00284  
00285     return k_rx_ok;  
00286 }
```



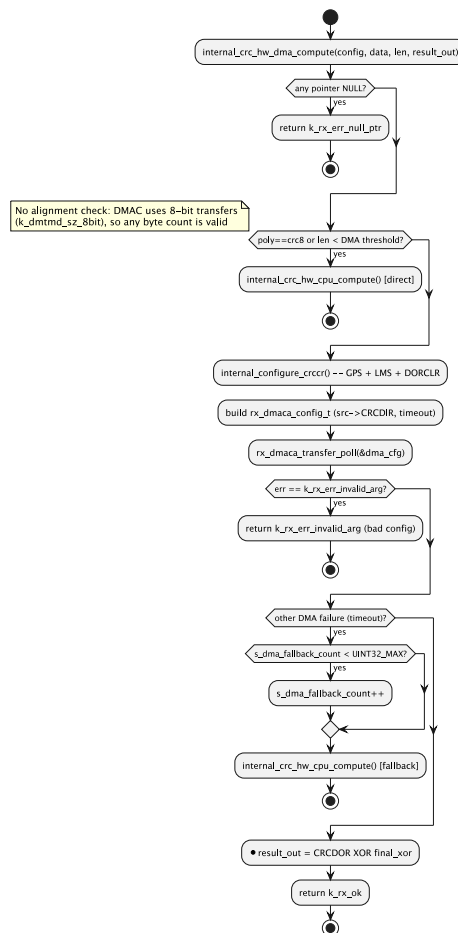
```

rx_err_t internal_crc_hw_dma_compute (
    const rx_crc_config_t * config,
    const uint8_t * data,
    uint32_t len,
    uint32_t * result_out)  [nodiscard]

```

<

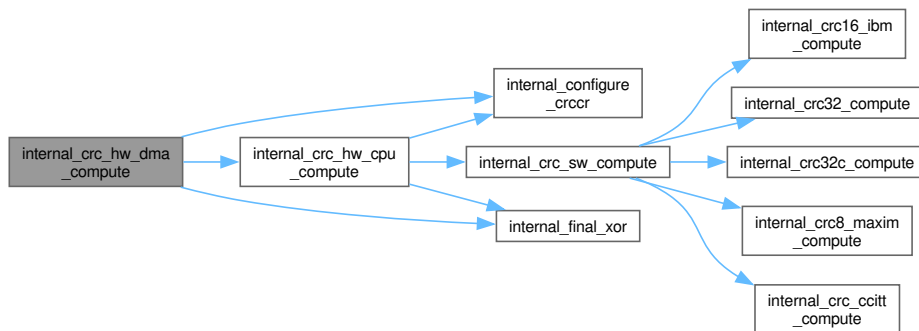
*

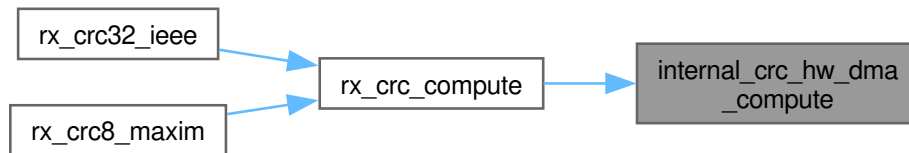


```

00410 {
00411     if (config == nullptr || data == nullptr || result_out == nullptr) {
00412         return k_rx_err_null_ptr;
00413     }
00414
00415     /* CRC-8/Maxim or small buffer: fall back to CPU-driven path */
00416     if (config->poly == k_rx_crc_poly_crc8 || len < k_crc_dma_threshold) {
00417         return internal_crc_hw_cpu_compute(config, data, len, result_out);
00418     }
00419
00420     /* No alignment requirement: DMAC is configured for 8-bit (k_dmtmd_sz bit)
00421      * transfers in internal_configure_channel(), so CRCDIR receives one byte at
00422      * a time regardless of the CRC polynomial selected. */
00423     internal_configure_crccr(config);
00424
00425     /* Use config timeout if non-zero, otherwise use library default */
00426     uint32_t timeout = (config->dma.timeout_cycles != 0U) ? config->dma.timeout_cycles
00427         : (uint32_t)k_dma_crc_timeout_cycles;
00428
00429     /* Destination: byte address of CRCDIR register (fixed, no increment) */
00430     const rx_dmaca_config_t dma_cfg = {
00431         .channel    = (uint8_t)k_dma_channel_crc,
00432         .src        = data,
00433         .len        = len,
00434         .dst_addr   = (uintptr_t)((volatile uint8_t*)&crc_regs()->crcdir),
00435         .timeout_cycles = timeout,
00436     };
00437
00438     rx_err_t err = rx_dmaca_transfer_poll(&dma_cfg);
00439     if (err == k_rx_err_invalid_arg) {
00440         /* Bad config (e.g. timeout_cycles out of range): surface to caller, do not fall back */
00441         return err;
00442     }
00443     if (err != k_rx_ok) {
00444         /* Transient DMA failure (timeout): record for observability (saturating), fall back to CPU */
00445         if (s_dma_fallback_count < UINT32_MAX) {
00446             s_dma_fallback_count++;
00447         }
00448         return internal_crc_hw_cpu_compute(config, data, len, result_out);
00449     }
00450
00451     *result_out = crc_regs()->crcdor ^ internal_final_xor(config->poly);
00452     return k_rx_ok;
00453 }

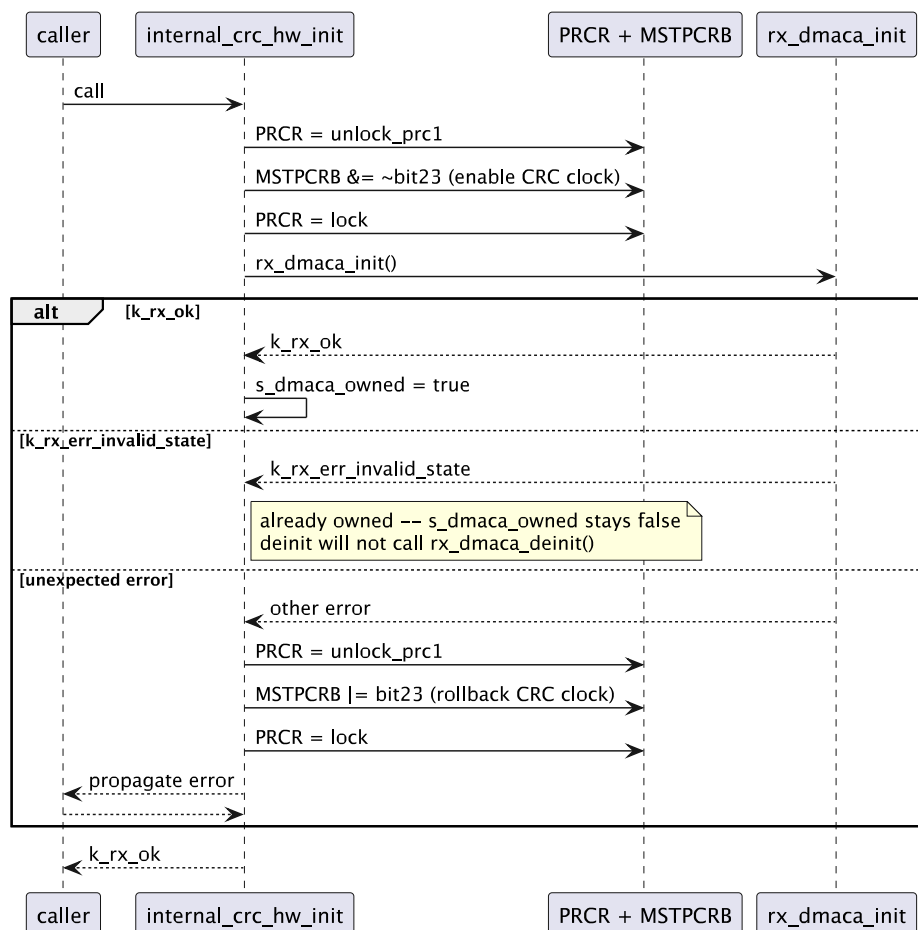
```





```

rx_err_t internal_crc_hw_init (
    void ) [nodiscard]
  
```



```

00226 {
00227   RX_CHECK_NULL_PTR(system_regs(), s_tag, "System registers not accessible");
00228   RX_CHECK_NULL_PTR(crc_regs(), s_tag, "CRC registers not accessible");
00229
00230   /* Unlock protection and enable CRC module clock (clear MSTPCRB bit 23) */
00231   *prcr_reg() = k_rx_prcr_unlock_prc1;
00232   system_regs()->mstpcrb &= ~((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00233   *prcr_reg() = k_rx_prcr_lock;
00234
00235   /* Initialize DMA driver; treat k_rx_err_invalid_state as OK (already initialized),
00236    * propagate any unexpected error so callers know DMA is unavailable.
00237    * Track ownership: only deinit DMAC on teardown if we initialized it here.
00238    * On unexpected failure, roll back CRC clock to leave no partial state. */
00239   rx_err_t dma_ret = rx_dmaca_init();
00240   if (dma_ret == k_rx_ok) {
00241     s_dmaca_owned = true;
00242   } else if (dma_ret != k_rx_err_invalid_state) {
00243     /* Roll back: disable CRC clock before returning the error */
00244     *prcr_reg() = k_rx_prcr_unlock_prc1;
00245     system_regs()->mstpcrb |= ((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00246     *prcr_reg() = k_rx_prcr_lock;
00247     return dma_ret;
00248   }
00249
00250   return k_rx_ok;
00251 }

```

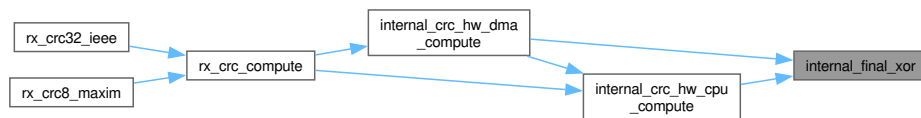


```

uint32_t internal_final_xor (
    rx_crc_poly_t poly) [static]

```

```
00143 {  
00144     if (poly == k_rx_crc_poly_crc32 || poly == k_rx_crc_poly_crc32c) {  
00145         return k_hw_crc32_final_xor;  
00146     }  
00147     return k_hw_crc16_final_xor;  
00148 }
```



```
uint32_t s_dma_fallback_count = 0U [static]
```

```
bool s_dmaca_owned = false [static]
```

```
const char* const s_tag = "CRC" [static]
```

```
00001 /**
00002  * @file rx_crc_hw.c
00003  * @brief RX72N Hardware CRC Backend Implementation
00004  *
00005  * @details
00006  * Implements internal_crc_hw_init(), internal_crc_hw_deinit(),
00007  * internal_crc_hw_cpu_compute(), and internal_crc_hw_dma_compute() for the
00008  * RX72N CRC Calculator peripheral (base 0x00088280, MSTPCRB bit 23).
00009  *
00010  * ## GPS (General Purpose Subsystem) Mapping (CRCCR bits [2:0])
00011  *
00012  * | rx_crc_poly_t | GPS | Notes |
00013  * |-----|-----|
00014  * | k_rx_crc_poly_crc8 (0x00) | 0x00 | Wrong poly for Maxim -> SW only |
00015  * | k_rx_crc_poly_crc16 (0x01) | 0x01 | CRC-16/IBM |
00016  * | k_rx_crc_poly_crc_ccitt (0x02) | 0x02 | CRC-CCITT/Kermit |
00017  * | k_rx_crc_poly_crc32 (0x03) | 0x03 | CRC-32/IEEE 802.3 |
00018  * | k_rx_crc_poly_crc32c (0x04) | 0x04 | CRC-32C/Castagnoli |
00019  *
00020  * @see rx_crc_internal.h Internal declarations
00021  * @see rx72n_crc_regs.h CRC peripheral register layout
00022  * @see rx_dmaca.h DMA driver
00023  *
00024  * @author Locked, Inc.
00025  * @date 2026-03-05
00026  * @copyright Copyright (c) 2026 Locked Inc.
00027  * SPDX-License-Identifier: MIT
00028 */
00029
00030 #include "rx_crc_internal.h"
00031
00032 #ifdef __RX__
00033
00034 #include <stddef.h>
00035
00036 #include "rx72n_regs.h"
00037 #include "rx_check.h"
00038 #include "rx_dmaca.h"
00039 #include "rx_register_protection.h"
00040
00041 /*
00042  * Private Constants
00043  *
00044  */
00045
00046 /**
00047  * @brief CRCCR configuration bits for hw_cpu compute
00048  *
00049  * @details
00050  * LMS (bit 6) = 1: LSB-first (reflected). Required for IEEE 802.3 / Kermit
00051  * compatibility. DORCLR (bit 7) = 1: clear accumulator before computation.
00052  */
00053 typedef enum : uint8_t {
00054     k_hw_crccr_lms = k_crc_crccr_lms, /**< Bit 6: LSB-first mode */
00055     k_hw_crccr_dorclr = k_crc_crccr_dorclr, /**< Bit 7: clear output register */
00056 } rx_crc_hw_crccr_t;
00057
```

```

00058 /**
00059  * @brief Final XOR constants for CRC result finalization
00060  */
00061 typedef enum : uint32_t {
00062     k_hw_crc32_final_xor = 0xFFFFFFFFU, /**< IEEE 802.3 / CRC-32C final XOR */
00063     k_hw_crc16_final_xor = 0x00000000U, /**< CRC-16/IBM: no final XOR */
00064     k_hw_ccitt_final_xor = 0x00000000U, /**< CRC-CCITT/Kermit: no final XOR */
00065 } rx_crc_hw_final_xor_t;
00066
00067 /**
00068  * @brief Module stop bit position for MSTPCRB
00069  */
00070 typedef enum : uint8_t {
00071     k_hw_mstpb_crc_bit = k_mstpb_crc, /**< Bit 23: CRC module stop in MSTPCRB */
00072     k_hw_bit_one = 1U, /**< Single bit value for shift operations */
00073 } rx_crc_hw_mstpcrb_t;
00074
00075 /**
00076  * @brief Alignment requirement for CRC-32 and CRC-32C DMA transfers
00077  */
00078 typedef enum : uint8_t {
00079     k_hw_crc32_alignment_bytes = 4U, /**< CRC-32/CRC-32C require 4-byte aligned buffer and length */
00080 } rx_crc_hw_alignment_t;
00081
00082 /*
=====
00083  * Module State
00084  *
=====
00085  */
00086
00087 /**
00088  * @var s_dmaca_owned
00089  * @brief True when this module called rx_dmaca_init() successfully
00090  *
00091  * @details
00092  * Set to true when rx_dmaca_init() returns k_rx_ok in internal_crc_hw_init().
00093  * Remains false when rx_dmaca_init() returns k_rx_err_invalid_state (DMAC was
00094  * already initialized by a third party). internal_crc_hw_deinit() calls
00095  * rx_dmaca_deinit() only when this flag is true, preventing double-deinit.
00096  *
00097  * @note Not thread-safe; only mutated during single-threaded init/deinit.
00098  * @since Version 1.0.0
00099  */
00100 static bool s_dmaca_owned = false;
00101
00102 /**
00103  * @var s_dma_fallback_count
00104  * @brief Count of DMA transfer failures that triggered CPU fallback
00105  *
00106  * @details
00107  * Incremented each time rx_dmaca_transfer_poll() returns an error and the
00108  * hw_dma backend falls back to the CPU loop. Non-zero values indicate DMA
00109  * reliability issues (timeout, misconfiguration) that are otherwise silent.
00110  *
00111  * @note Not thread-safe: rx_dmaca_transfer_poll() is itself single-channel
00112  * and not thread-safe, so concurrent access cannot occur in practice.
00113  * @warning Do not modify directly; read via debug/telemetry only.
00114  * @since Version 1.0.0
00115  */
00116 static uint32_t s_dma_fallback_count = 0U;
00117
00118 /**
00119  * @var s_tag
00120  * @brief Log tag for CRC hardware module
00121  * @details Identifies log messages from this module
00122  * @note Read-only after initialization
00123  * @since Version 1.0.0
00124  */
00125 static const char* const s_tag = "CRC";
00126
00127 /*
=====
00128  * Private Helpers
00129  *
=====
00130  */
00131
00132 /**
00133  * @brief Return the final XOR value for the given polynomial
00134  *
00135  * @pre poly is a valid rx_crc_poly_t value within the defined enum range
00136  * @pre Called only for hardware-supported polynomials (CRC-16 through CRC-32C)
00137  * @post Returns 0xFFFFFFFF for CRC-32/CRC-32C, 0 otherwise
00138  * @post No side effects; does not modify any global or peripheral state
00139  * @since Version 1.0.0
00140  */

```

```

00141 */
00142 static uint32_t internal_final_xor(rx_crc_poly_t poly)
00143 {
00144     if (poly == k_rx_crc_poly_crc32 || poly == k_rx_crc_poly_crc32c) {
00145         return k_hw_crc32_final_xor;
00146     }
00147     return k_hw_crc16_final_xor;
00148 }
00149
00150 /**
00151  * @brief Configure CRC peripheral control register and clear accumulator
00152  *
00153  * @details
00154  * Sets GPS from poly enum value (valid for CRC-16 through CRC-32C), sets LMS
00155  * from config->bit_order, and sets DORCLR to reset the CRC accumulator.
00156  *
00157  * @pre CRC peripheral module clock must be enabled
00158  * @pre config must not be NULL; config->poly and config->bit_order must be valid enum values
00159  * @post CRCCR configured; accumulator reset to initial value for the polynomial
00160  * @post Only CRCCR and the CRC accumulator (via DORCLR) are modified; no other registers changed
00161  * @since Version 1.0.0
00162  */
00163
00164 static void internal_configure_crccr(const rx_crc_config_t* config)
00165 {
00166     uint8_t lms = (config->bit_order == k_rx_crc_bit_order_lsb_first) ? (uint8_t)k_hw_crccr_lms : 0U;
00167
00168     /* GPS = poly enum value (0x01..0x04 maps directly to CRCCR GPS field) */
00169     uint8_t gps = (uint8_t)config->poly;
00170
00171     crc_regs()->crccr = (uint8_t)(gps | lms | (uint8_t)k_hw_crccr_dorclr);
00172 }
00173
00174 /*
=====
00175  * Lifecycle
00176  *
=====
00177  */
00178
00179 /**
00180  * @brief Enable CRC peripheral clock and initialize DMAC for hw_dma backend
00181  *
00182  * @details
00183  * Unlocks PRCR, clears MSTPCRB bit 23 to enable the CRC peripheral clock,
00184  * then re-locks PRCR. Calls rx_dmaca_init() to prepare the DMA channel;
00185  * if DMAC was already initialized (k_rx_err_invalid_state), the CRC module
00186  * records that it does not own DMAC and will not deinit it on teardown.
00187  *
00188  * @pre System registers (SYSTEM) must be accessible
00189  * @pre CRC registers must be accessible
00190  * @post MSTPCRB bit 23 cleared (CRC clock enabled)
00191  * @post s_dmaca_owned set to true only if rx_dmaca_init() returned k_rx_ok
00192  *
00193  * @par Sequence Diagram:
00194  * @startuml
00195  * participant "caller" as C
00196  * participant "internal_crc_hw_init" as I
00197  * participant "PRCR + MSTPCRB" as R
00198  * participant "rx_dmaca_init" as D
00199  * C -> I: call
00200  * I -> R: PRCR = unlock_prc1
00201  * I -> R: MSTPCRB &= ~bit23 (enable CRC clock)
00202  * I -> R: PRCR = lock
00203  * I -> D: rx_dmaca_init()
00204  * alt k_rx_ok
00205  *     D --> I: k_rx_ok
00206  *     I -> I: s_dmaca_owned = true
00207  * else k_rx_err_invalid_state
00208  *     D --> I: k_rx_err_invalid_state
00209  *     note right of I: already owned -- s_dmaca_owned stays false\ndeinit will not call rx_dmaca_deinit()
00210  * else unexpected error
00211  *     D --> I: other error
00212  *     I -> R: PRCR = unlock_prc1
00213  *     I -> R: MSTPCRB |= bit23 (rollback CRC clock)
00214  *     I -> R: PRCR = lock
00215  *     I --> C: propagate error
00216  *     return
00217  * end
00218  * I --> C: k_rx_ok
00219  * @enduml
00220  *
00221  * @note Not thread-safe; call during single-threaded system init
00222  * @since Version 1.0.0
00223  */
00224
00225 rx_err_t internal_crc_hw_init(void)

```

```

00226 {
00227     RX_CHECK_NULL_PTR(system_regs(), s_tag, "System registers not accessible");
00228     RX_CHECK_NULL_PTR(crc_regs(), s_tag, "CRC registers not accessible");
00229
00230     /* Unlock protection and enable CRC module clock (clear MSTPCRB bit 23) */
00231     *prcr_reg() = k_rx_prcr_unlock_prc1;
00232     system_regs()->mstpcrb &= ~(uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00233     *prcr_reg() = k_rx_prcr_lock;
00234
00235     /* Initialize DMA driver; treat k_rx_err_invalid_state as OK (already initialized),
00236      * propagate any unexpected error so callers know DMA is unavailable.
00237      * Track ownership: only deinit DMAC on teardown if we initialized it here.
00238      * On unexpected failure, roll back CRC clock to leave no partial state. */
00239     rx_err_t dma_ret = rx_dmaca_init();
00240     if (dma_ret == k_rx_ok) {
00241         s_dmaca_owned = true;
00242     } else if (dma_ret != k_rx_err_invalid_state) {
00243         /* Roll back: disable CRC clock before returning the error */
00244         *prcr_reg() = k_rx_prcr_unlock_prc1;
00245         system_regs()->mstpcrb |= ((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00246         *prcr_reg() = k_rx_prcr_lock;
00247         return dma_ret;
00248     }
00249
00250     return k_rx_ok;
00251 }
00252
00253 /**
00254  * @brief Disable CRC peripheral clock and release DMAC if owned
00255  *
00256  * @details
00257  * Calls rx_dmaca_deinit() only if s_dmaca_owned is true (this module called
00258  * rx_dmaca_init() successfully). Then unlocks PRCR, sets MSTPCRB bit 23 to
00259  * stop the CRC peripheral clock, and re-locks PRCR.
00260  *
00261  * @pre System registers (SYSTEM) must be accessible
00262  * @pre internal_crc_hw_init() must have been called successfully
00263  * @post MSTPCRB bit 23 set (CRC clock disabled)
00264  * @post s_dmaca_owned cleared to false
00265  *
00266  * @note Not thread-safe; call during single-threaded system teardown
00267  * @since Version 1.0.0
00268  */
00269 rx_err_t internal_crc_hw_deinit(void)
00270 {
00271     RX_CHECK_NULL_PTR(system_regs(), s_tag, "System registers not accessible");
00272
00273     /* Deinitialize DMA driver only if this module initialized it */
00274     if (s_dmaca_owned) {
00275         (void)rx_dmaca_deinit();
00276         s_dmaca_owned = false;
00277     }
00278
00279     /* Re-enable module stop (set MSTPCRB bit 23) */
00280     *prcr_reg() = k_rx_prcr_unlock_prc1;
00281     system_regs()->mstpcrb |= ((uint32_t)k_hw_bit_one « k_hw_mstpb_crc_bit);
00282     *prcr_reg() = k_rx_prcr_lock;
00283
00284     return k_rx_ok;
00285 }
00286
00287 /*
00288 =====
00289 * CPU-Driven Hardware CRC
00290 =====
00291 */
00292
00293 /**
00294  * @brief Compute CRC using RX72N CRC peripheral, CPU-driven byte loop
00295  *
00296  * @details
00297  * Configures CRCCR with GPS (polynomial) and LMS (bit order) bits, sets DORCLR
00298  * to reset the accumulator, then feeds data byte-by-byte to CRCDIR.
00299  * CRC-8/Maxim (GPS=0x00) maps to the wrong hardware polynomial (0x07 vs 0x31),
00300  * so it falls back to internal_crc_sw_compute() automatically.
00301  *
00302  *
00303  *
00304  * @pre CRC peripheral clock must be enabled (internal_crc_hw_init() called)
00305  * @pre data must point to valid readable memory of len bytes
00306  * @post *result_out contains the final CRC with XOR applied
00307  * @post CRCDOR accumulator is in indeterminate state after read
00308  *
00309  * @note CRC-8/Maxim always uses software fallback (hardware poly mismatch)
00310  * @since Version 1.0.0

```

```

00311 */
00312 rx_err_t internal_crc_hw_cpu_compute(const rx_crc_config_t* config,
00313                                     const uint8_t* data,
00314                                     uint32_t len,
00315                                     uint32_t* result_out)
00316 {
00317     if (config == nullptr || data == nullptr || result_out == nullptr) {
00318         return k_rx_err_null_ptr;
00319     }
00320
00321     /* CRC-8/Maxim: hardware GPS=0x00 uses poly 0x07, not 0x31 - must use SW */
00322     if (config->poly == k_rx_crc_poly_crc8) {
00323         /* Software CRC-8/Maxim only supports LSB-first bit order */
00324         if (config->bit_order != k_rx_crc_bit_order_lsb_first) {
00325             return k_rx_err_invalid_arg;
00326         }
00327         *result_out = internal_crc_sw_compute(config->poly, data, len);
00328         return k_rx_ok;
00329     }
00330
00331     internal_configure_crccr(config);
00332
00333     /* Feed data byte-by-byte (NASA Rule 2: bounded by k_crc_len_max) */
00334     volatile uint8_t* crcdir_byte = (volatile uint8_t*)&crc_regs()->crcdir;
00335     for (uint32_t i = k_crc_idx_start; i < k_crc_len_max; i++) {
00336         if (i >= len) {
00337             break;
00338         }
00339         *crcdir_byte = data[i];
00340     }
00341
00342     *result_out = crc_regs()->crcdor ^ internal_final_xor(config->poly);
00343     return k_rx_ok;
00344 }
00345
00346 /*
=====
00347 * DMA-Driven Hardware CRC
00348 *
=====
00349 */
00350
00351 /**
00352 * @brief Compute CRC using RX72N CRC peripheral, DMA-driven transfer
00353 *
00354 * @details
00355 * Configures CRCCR then uses rx_dmaca_transfer_poll() to move src to CRCDIR
00356 * without CPU intervention. Falls back to internal_crc_hw_cpu_compute() for:
00357 * - CRC-8/Maxim (hardware polynomial mismatch)
00358 * - len < k_crc_dma_threshold (DMA overhead not justified)
00359 * - DMA transfer failure (s_dma_fallback_count incremented for observability)
00360 *
00361 *
00362 *
00363 * @pre CRC and DMAC peripherals must be initialized (internal_crc_hw_init() called)
00364 * @pre data must point to valid readable memory of len bytes
00365 * @post *result_out contains the final CRC with XOR applied (on k_rx_ok)
00366 * @post s_dma_fallback_count incremented only on transient DMA failure (not on invalid_arg)
00367 *
00368 * @par Activity Diagram:
00369 * @startuml
00370 * start
00371 * :internal_crc_hw_dma_compute(config, data, len, result_out);
00372 * if (any pointer NULL?) then (yes)
00373 *   :return k_rx_err_null_ptr;
00374 *   stop
00375 * endif
00376 * if (poly==crc8 or len < DMA threshold?) then (yes)
00377 *   :internal_crc_hw_cpu_compute() [direct];
00378 *   stop
00379 * endif
00380 * note right
00381 *   No alignment check: DMAC uses 8-bit transfers
00382 *   (k_dmtmd_sz bit), so any byte count is valid
00383 * end note
00384 * :internal_configure_crccr() -- GPS + LMS + DORCLR;
00385 * :build rx_dmaca_config_t (src->CRCDIR, timeout);
00386 * :rx_dmaca_transfer_poll(&dmac_cfg);
00387 * if (err == k_rx_err_invalid_arg?) then (yes)
00388 *   :return k_rx_err_invalid_arg (bad config);
00389 *   stop
00390 * endif
00391 * if (other DMA failure (timeout)?) then (yes)
00392 *   if (s_dma_fallback_count < UINT32_MAX?) then (yes)
00393 *     :s_dma_fallback_count++;
00394 *   endif
00395 *   :internal_crc_hw_cpu_compute() [fallback];

```

```

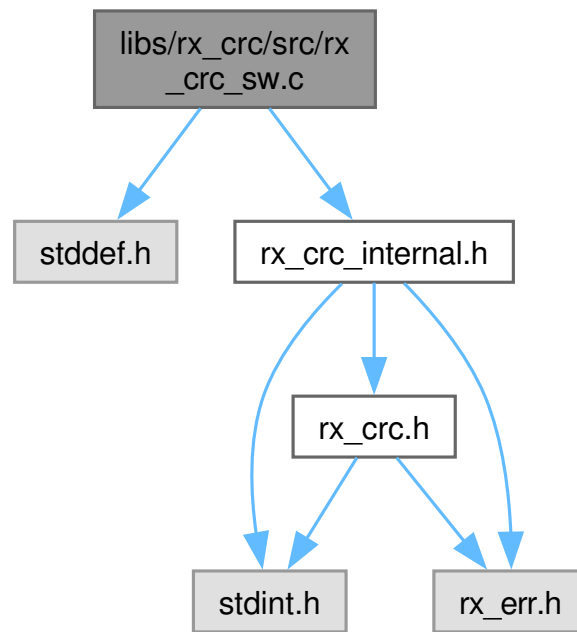
00396 * stop
00397 * endif
00398 * :*result_out = CRC_DOR XOR final_xor;
00399 * :return k_rx_ok;
00400 * stop
00401 * @enduml
00402 *
00403 * @note Transient DMA failures fall back transparently to the CPU path; result is identical
00404 * @since Version 1.0.0
00405 */
00406 rx_err_t internal_crc_hw_dma_compute(const rx_crc_config_t* config,
00407                                     const uint8_t* data,
00408                                     uint32_t len,
00409                                     uint32_t* result_out)
00410 {
00411     if (config == nullptr || data == nullptr || result_out == nullptr) {
00412         return k_rx_err_null_ptr;
00413     }
00414
00415     /* CRC-8/Maxim or small buffer: fall back to CPU-driven path */
00416     if (config->poly == k_rx_crc_poly_crc8 || len < k_crc_dma_threshold) {
00417         return internal_crc_hw_cpu_compute(config, data, len, result_out);
00418     }
00419
00420     /* No alignment requirement: DMAC is configured for 8-bit (k_dmtmd_sz bit)
00421      * transfers in internal_configure_channel(), so CRCDIR receives one byte at
00422      * a time regardless of the CRC polynomial selected. */
00423     internal_configure_crccr(config);
00424
00425     /* Use config timeout if non-zero, otherwise use library default */
00426     uint32_t timeout = (config->dma.timeout_cycles != 0U) ? config->dma.timeout_cycles
00427                                                         : (uint32_t)k_dma_crc_timeout_cycles;
00428
00429     /* Destination: byte address of CRCDIR register (fixed, no increment) */
00430     const rx_dmaca_config_t dma_cfg = {
00431         .channel = (uint8_t)k_dma_channel_crc,
00432         .src = data,
00433         .len = len,
00434         .dst_addr = (uintptr_t)((volatile uint8_t*)&crc_regs()->crcdir),
00435         .timeout_cycles = timeout,
00436     };
00437
00438     rx_err_t err = rx_dmaca_transfer_poll(&dma_cfg);
00439     if (err == k_rx_err_invalid_arg) {
00440         /* Bad config (e.g. timeout_cycles out of range): surface to caller, do not fall back */
00441         return err;
00442     }
00443     if (err != k_rx_ok) {
00444         /* Transient DMA failure (timeout): record for observability (saturating), fall back to CPU */
00445         if (s_dma_fallback_count < UINT32_MAX) {
00446             s_dma_fallback_count++;
00447         }
00448         return internal_crc_hw_cpu_compute(config, data, len, result_out);
00449     }
00450
00451     *result_out = crc_regs()->crcdor ^ internal_final_xor(config->poly);
00452     return k_rx_ok;
00453 }
00454
00455 #endif /* __RX__ */

```

```

#include <stddef.h>
#include "rx_crc_internal.h"

```



*

*

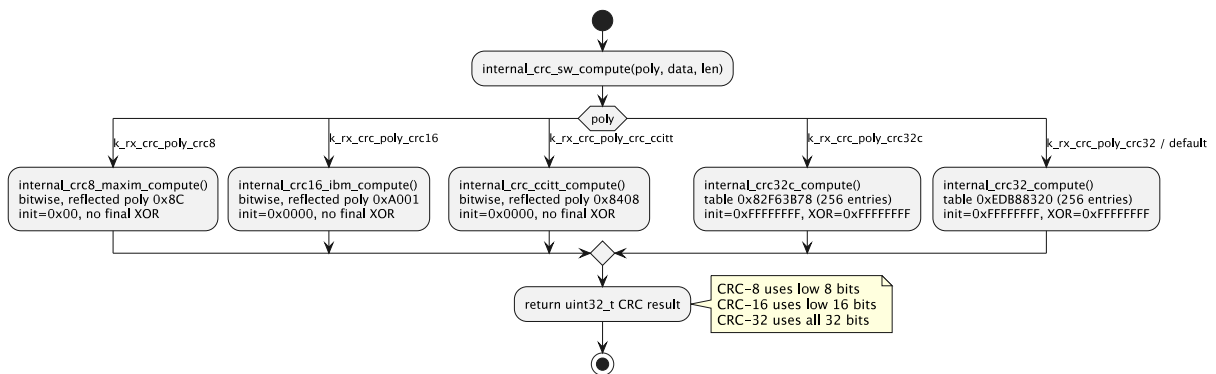
*

*

*

*

*



```
enum rx_crc16_constants_t : uint16_t
```


```
00083         : uint16_t {
00084   k_crc16_ibm_poly = 0xA001U, /**< IBM poly, reflected form */
00085   k_crc16_init     = 0x0000U, /**< Initial value */
00086   k_crc16_final_xor = 0x0000U, /**< Final XOR value (0 = no XOR applied) */
00087 } rx_crc16_constants_t;
```

```
enum rx_crc8_maxim_constants_t : uint8_t
```


```
00100         : uint8_t {
00101   k_crc8_maxim_poly = 0x8CU, /**< Maxim/Dallas poly, reflected */
00102   k_crc8_init       = 0x00U, /**< Initial value */
00103 } rx_crc8_maxim_constants_t;
```

```
enum rx_crc_ccitt_constants_t : uint16_t
```


```
00092         : uint16_t {
00093   k_crc_ccitt_poly = 0x8408U, /**< CCITT poly, reflected (Kermit) */
00094   k_crc_ccitt_init = 0x0000U, /**< Initial value for Kermit variant */
00095 } rx_crc_ccitt_constants_t;
```

```
enum rx_crc_sw_bit_constants_t : uint8_t
```

--	--

```
00074      : uint8_t {
00075  k_bits_per_byte = 8U, /**< Bits in one byte */
00076  k_crc_lsb_mask  = 1U, /**< Mask for LSB */
00077  k_crc_shift bit = 1U, /**< Shift by 1 */
00078 } rx_crc_sw_bit_constants_t;
```

```
enum rx_crc_sw_u32_constants_t : uint32_t
```


```
00063      : uint32_t {
00064  k_crc32_table_size = 256U,      /**< Lookup table entry count */
00065  k_crc_byte_mask    = 0xFFU,    /**< Byte mask for table index */
00066  k_crc32_init_value = 0xFFFFFFFFU, /**< IEEE 802.3 initial value */
00067  k_crc32_final_xor  = 0xFFFFFFFFU, /**< IEEE 802.3 final XOR */
00068  k_crc_byte_shift   = 8U,      /**< Right-shift per byte iteration */
00069 } rx_crc_sw_u32_constants_t;
```

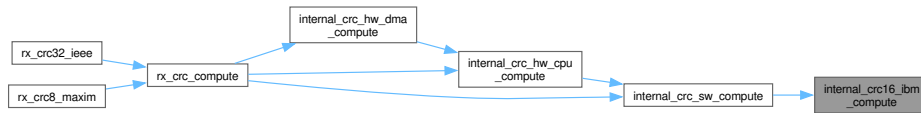
```
uint16_t internal_crc16_ibm_compute (
    const uint8_t * data,
    uint32_t len) [static]
```

<<

```

00277 {
00278     uint16_t crc = k_crc16_init;
00279
00280     for (uint32_t i = 0U; i < len; i++) {
00281         crc ^= (uint16_t)data[i];
00282         for (uint8_t b = 0U; b < k_bits_per_byte; b++) {
00283             if ((crc & (uint16_t)k_crc_lsb_mask) != 0U) {
00284                 crc = (uint16_t)((crc » k_crc_shift bit) ^ k_crc16_ibm_poly);
00285             } else {
00286                 crc »= k_crc_shift bit;
00287             }
00288         }
00289     }
00290
00291     return (uint16_t)(crc ^ k_crc16_final_xor);
00292 }

```



```

uint32_t internal_crc32_compute (
    const uint8_t * data,
    uint32_t len) [static]

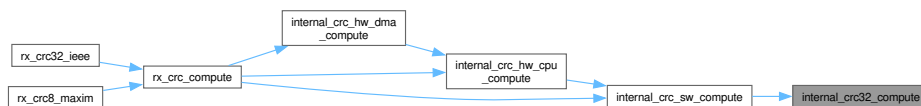
```

<<

```

00222 {
00223     uint32_t crc = k_crc32_init_value;
00224
00225     for (uint32_t i = 0U; i < len; i++) {
00226         crc = s_crc32_table[(uint8_t)(crc ^ data[i])] ^ (crc » k_crc_byte_shift);
00227     }
00228
00229     return crc ^ k_crc32_final_xor;
00230 }

```



```
uint32_t internal_crc32_sw_update (  
    uint32_t crc,  
    const uint8_t * data,  
    uint32_t len)
```

>

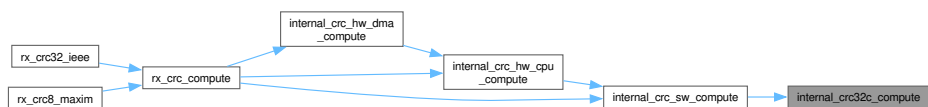
```
00458 {  
00459     if (len > k_crc_len_max) {  
00460         return crc;  
00461     }  
00462  
00463     /* Un-finalize previous CRC to get internal accumulator state */  
00464     uint32_t work = crc ^ k_crc32_final_xor;  
00465  
00466     for (uint32_t i = 0U; i < len; i++) {  
00467         work = s_crc32_table[(uint8_t)(work ^ data[i])] ^ (work » k_crc_byte_shift);  
00468     }  
00469  
00470     return work ^ k_crc32_final_xor;  
00471 }
```



```
uint32_t internal_crc32c_compute (  
    const uint8_t * data,  
    uint32_t len)  [static]
```

<<

```
00249 {  
00250     uint32_t crc = k_crc32_init_value;  
00251  
00252     for (uint32_t i = 0U; i < len; i++) {  
00253         crc = s_crc32c_table[(uint8_t)(crc ^ data[i])] ^ (crc » k_crc_byte_shift);  
00254     }  
00255  
00256     return crc ^ k_crc32_final_xor;  
00257 }
```



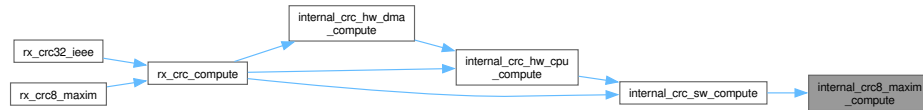
```
uint8_t internal_crc8_maxim_compute (  
    const uint8_t * data,  
    uint32_t len)  [static]
```

<<

```

00347 {
00348     uint8_t crc = k_crc8_init;
00349
00350     for (uint32_t i = 0U; i < len; i++) {
00351         crc ^= data[i];
00352         for (uint8_t b = 0U; b < k_bits_per_byte; b++) {
00353             if ((crc & k_crc_lsb_mask) != 0U) {
00354                 crc = (uint8_t)((crc » k_crc_shift bit) ^ k_crc8_maxim_poly);
00355             } else {
00356                 crc »= k_crc_shift bit;
00357             }
00358         }
00359     }
00360
00361     return crc;
00362 }

```



```

uint16_t internal_crc_ccitt_compute (
    const uint8_t * data,
    uint32_t len) [static]

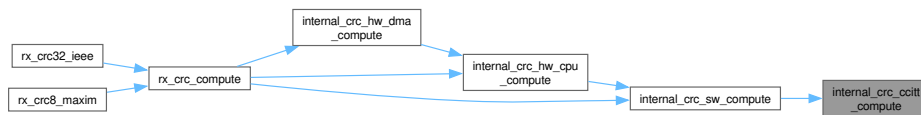
```

<<

```

00312 {
00313     uint16_t crc = k_crc_ccitt_init;
00314
00315     for (uint32_t i = 0U; i < len; i++) {
00316         crc ^= (uint16_t)data[i];
00317         for (uint8_t b = 0U; b < k_bits_per_byte; b++) {
00318             if ((crc & (uint16_t)k_crc_lsb_mask) != 0U) {
00319                 crc = (uint16_t)((crc » k_crc_shift bit) ^ k_crc_ccitt_poly);
00320             } else {
00321                 crc »= k_crc_shift bit;
00322             }
00323         }
00324     }
00325
00326     return crc;
00327 }

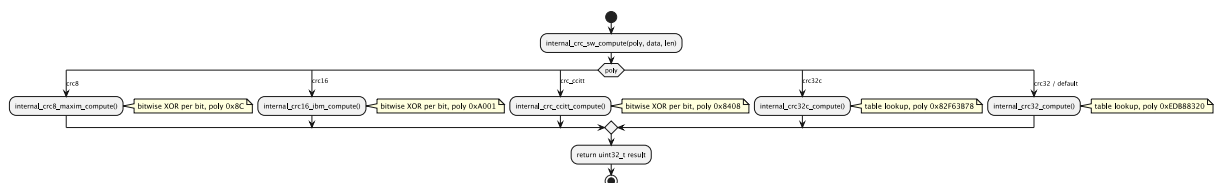
```



```

uint32_t internal_crc_sw_compute (
    rx_crc_poly_t poly,
    const uint8_t * data,
    uint32_t len)

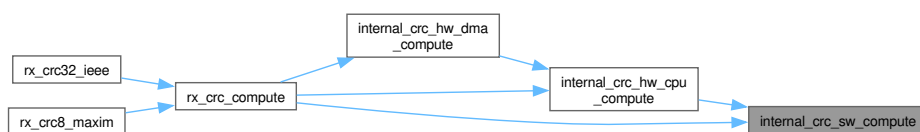
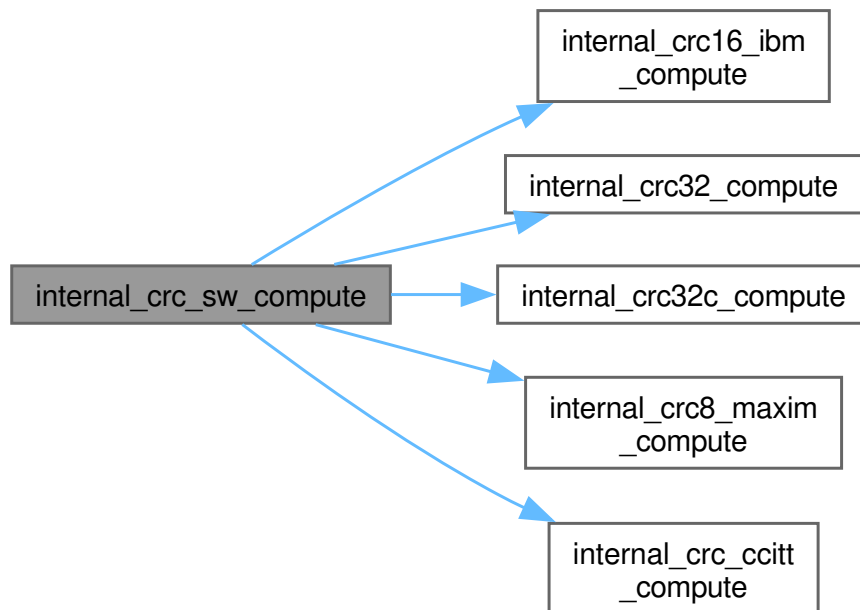
```



```

00415 {
00416     switch (poly) {
00417     case k_rx_crc_poly_crc8:
00418         return (uint32_t)internal_crc8_maxim_compute(data, len);
00419     case k_rx_crc_poly_crc16:
00420         return (uint32_t)internal_crc16_ibm_compute(data, len);
00421     case k_rx_crc_poly_crc_ccitt:
00422         return (uint32_t)internal_crc_ccitt_compute(data, len);
00423     case k_rx_crc_poly_crc32c:
00424         return internal_crc32c_compute(data, len);
00425     case k_rx_crc_poly_crc32:
00426     default:
00427         return internal_crc32_compute(data, len);
00428     }
00429 }

```



```
const uint32_t s_crc32_table[k_crc32_table_size] [static]
```

```
00117      {
00118 0x00000000, 0x77073096, 0xEE0E612C, 0x990951BA, 0x076DC419, 0x706AF48F, 0xE963A535, 0x9E6495A3,
00119 0x0EDB8832, 0x79DCB8A4, 0xE0D5E91E, 0x97D2D988, 0x09B64C2B, 0x7EB17CBD, 0xE7B82D07, 0x90BF1D91,
00120 0x1DB71064, 0x6AB020F2, 0xF3B97148, 0x84BE41DE, 0x1ADAD47D, 0x6DDDE4EB, 0xF4D4B551, 0x83D385C7,
00121 0x136C9856, 0x646BA8C0, 0xFD62F97A, 0x8A65C9EC, 0x14015C4F, 0x63066CD9, 0xFA0F3D63, 0x8D080DF5,
00122 0x3B6E20C8, 0x4C69105E, 0xD56041E4, 0xA2677172, 0x3C03E4D1, 0x4B04D447, 0xD20D85FD, 0xA50AB56B,
00123 0x35B5A8FA, 0x42B2986C, 0xDBBBC9D6, 0xACBCF940, 0x32D86CE3, 0x45DF5C75, 0xDCD60DCF, 0xABD13D59,
00124 0x26D930AC, 0x51DE003A, 0xC8D75180, 0xBFDD06116, 0x21B4F4B5, 0x56B3C423, 0xCFBA9599, 0xB8BDA50F,
00125 0x2802B89E, 0x5F058808, 0xC60CD9B2, 0xB10BE924, 0x2F6F7C87, 0x58684C11, 0xC1611DAB, 0xB6662D3D,
00126 0x76DC4190, 0x01DB7106, 0x98D220BC, 0xEFD5102A, 0x71B18589, 0x06B6B51F, 0x9FBBFE4A5, 0xE8B8D433,
00127 0x7807C9A2, 0x0F00F934, 0x9609A88E, 0xE10E9818, 0x7F6A0DBB, 0x086D3D2D, 0x91646C97, 0xE6635C01,
00128 0x6B6B51F4, 0x1C6C6162, 0x856530D8, 0xF262004E, 0x6C0695ED, 0x1B01A57B, 0x8208F4C1, 0xF50FC457,
00129 0x65B0D9C6, 0x12B7E950, 0x8BBEB8EA, 0xFCB9887C, 0x62DD1DDF, 0x15DA2D49, 0x8CD37CF3, 0xFBD44C65,
00130 0x4DB26158, 0x3AB551CE, 0xA3BC0074, 0xD4BB30E2, 0x4ADFA541, 0x3DD895D7, 0xA4D1C46D, 0xD3D6F4FB,
00131 0x4369E96A, 0x346ED9FC, 0xAD678846, 0xDA60B8D0, 0x44042D73, 0x33031DE5, 0xAA0A4C5F, 0xDD0D7CC9,
00132 0x5005713C, 0x270241AA, 0xBE0B1010, 0xC90C2086, 0x5768B525, 0x206F85B3, 0xB966D409, 0xCE61E49F,
00133 0x5EDEF90E, 0x29D9C998, 0xB0D09822, 0xC7D7A8B4, 0x59B33D17, 0x2EB40D81, 0xB7BD5C3B, 0xC0BA6CAD,
00134 0xEDB88320, 0x9ABFB3B6, 0x03B6E20C, 0x74B1D29A, 0xEAD54739, 0x9DD277AF, 0x04DB2615, 0x73DC1683,
00135 0xE3630B12, 0x94643B84, 0x0D6D6A3E, 0x7A6A5AA8, 0xE40ECF0B, 0x9309FF9D, 0x0A00AE27, 0x7D079EB1,
00136 0xF00F9344, 0x8708A3D2, 0x1E01F268, 0x6906C2FE, 0xF762575D, 0x806567CB, 0x196C3671, 0x6E6B06E7,
00137 0xFED41B76, 0x89D32BE0, 0x10DA7A5A, 0x67DD4ACC, 0xF9B9DF6F, 0x8EBEEFF9, 0x17B7BE43, 0x60B08ED5,
00138 0xD6D6A3E8, 0xA1D1937E, 0x38D8C2C4, 0x4FDDFF25, 0xD1BB67F1, 0xA6BC5767, 0x3FB506DD, 0x48B2364B,
00139 0xD80D2BDA, 0xAF0A1B4C, 0x36034AF6, 0x41047A60, 0xDF60EFC3, 0xA867DF55, 0x316E8EEF, 0x4669BE79,
00140 0xCB61B38C, 0xBC66831A, 0x256FD2A0, 0x5268E236, 0xCC0C7795, 0xBB0B4703, 0x220216B9, 0x5505262F,
00141 0xC5BA3BBE, 0xB2BD0B28, 0x2BB45A92, 0x5CB36A04, 0xC2D7FFA7, 0xB5D0CF31, 0x2CD99E8B, 0x5BDEAE1D,
00142 0x9B64C2B0, 0x6C63F226, 0x756AA39C, 0x026D930A, 0x9C0906A9, 0xEB0E363F, 0x72076785, 0x05005713,
00143 0x95BF4A2B, 0xE2B87A14, 0x7BB12BAE, 0x0CB61B38, 0x92D28E9B, 0xE5D5BE0D, 0x7CDECEFB7, 0x0BDBDF21,
00144 0x86D3D2D4, 0xF1D4E242, 0x68DDB3F8, 0x1FDA836E, 0x81BE16CD, 0xF6B9265B, 0x6FB077E1, 0x18B77477,
00145 0x88085AE6, 0xFF0F4A70, 0x66066BCA, 0x11010B5C, 0x8F659EFF, 0xF862AE69, 0x616BFDF3, 0x166CCF45,
00146 0xA00AE278, 0xD70DD2EE, 0x4E048354, 0x3903B3C2, 0xA7672661, 0xD06016F7, 0x4969474D, 0x3E6E77DB,
00147 0xAED16A4A, 0xD9D65ADC, 0x40DF0B66, 0x37D83BF0, 0xA9BCAE53, 0xDEBB9EC5, 0x47B2CFF7, 0x30B5F9E9,
00148 0xBDBDF21C, 0xCABAC28A, 0x53B39330, 0x24B4A3A6, 0xBAD03605, 0xCDD70693, 0x54DE5729, 0x23D967BF,
00149 0xB3667A2E, 0xCA614AB8, 0x5D681B02, 0x2A6F2B94, 0xB40BBE37, 0xC30C8EA1, 0x5A05DF1B, 0x2D02EF8D,
00150 };
```

```
const uint32_t s_crc32c_table[k_crc32_table_size] [static]
```

```
00164      {
00165 0x00000000, 0xF26B8303, 0xE13B70F7, 0x1350F3F4, 0xC79A971F, 0x35F1141C, 0x26A1E7E8, 0xD4CA64EB,
00166 0x8AD958CF, 0x78B2DBCC, 0x6BE22838, 0x9989AB3B, 0x4D43CFD0, 0xBF284CD3, 0xAC78BF27, 0x5E133C24,
00167 0x105EC76F, 0xE235446C, 0xF165B798, 0x030E349B, 0xD7C45070, 0x25AFD373, 0x36FF2087, 0xC494A384,
00168 0x9A879FA0, 0x68EC1CA3, 0x7BBCE5F7, 0x89D76C54, 0x5D1D08BF, 0xAF768BBC, 0xBC267848, 0x4E4DFB4B,
00169 0x20BD8EDE, 0xD2D60DDD, 0xC186FE29, 0x33ED7D2A, 0xE72719C1, 0x154C9AC2, 0x061CE693, 0xF477EA35,
00170 0xAA64D611, 0x580F5512, 0x4B5FA6E6, 0xB93425E5, 0x6DFE410E, 0x9F95C20D, 0x8CC531F9, 0x7EAE2FA,
00171 0x30E349B1, 0xC288CAB2, 0xD1D83946, 0x23B3BA45, 0xF779DEAE, 0x05125DAD, 0x1642AE59, 0xE4292D5A,
00172 0xBA3A117E, 0x4851927D, 0x5B016189, 0xA96AE28A, 0x7DA08661, 0x8FCB0562, 0x9C9BF696, 0x6EF07595,
00173 0x417B1DBC, 0xB3109EBF, 0xA0406D4B, 0x522BEE48, 0x86E18AA3, 0x748A09A0, 0x67DAFA54, 0x95B17957,
00174 0xCBA24573, 0x39C9C670, 0x2A993584, 0xD8F2B687, 0x0C38D26C, 0xFE53516F, 0xED03A29B, 0x1F682198,
00175 0x5125DAD3, 0xA34E59D0, 0xB01EAA24, 0x42752927, 0x96BF4DCC, 0x64D4CECF, 0x77843D3B, 0x85EFBE38,
00176 0xDBFC821C, 0x2997011F, 0x3AC7F2EB, 0xC8AC71E8, 0x1C661503, 0xEE0D9600, 0xFD5D65FA, 0x0F36E6F7,
00177 0x61C69362, 0x93AD1061, 0x80FDE395, 0x72966096, 0xA65C047D, 0x5437877E, 0x4767748A, 0xB50CF789,
00178 0xEB1FCBAD, 0x197448AE, 0x0A24BB5A, 0xF84F3859, 0x2C855CB2, 0xDEEDFB1, 0xCDBE2C45, 0x3FD5AF46,
00179 0x7198540D, 0x83F3D70E, 0x90A324FA, 0x62C8A7F9, 0xB602C312, 0x44694011, 0x5739B3E5, 0xA55230E6,
00180 0xFB410CC2, 0x092A8FC1, 0x1A7A7C35, 0xE811FF36, 0x3CDB9BDD, 0xCEB018DE, 0xDDE0EB2A, 0x2F8B6829,
00181 0x82F63B78, 0x709DB87B, 0x63CD4B8F, 0x91A6C88C, 0x456CAC67, 0xB7072F64, 0xA457DC90, 0x563C5F93,
00182 0x082F63B7, 0xFA44E0B4, 0xE9141340, 0x1B7F9043, 0xCFB5F4A8, 0x3DDE77AB, 0x2E8E845F, 0xDCE0575C,
00183 0x92A8FC17, 0x60C37F14, 0x73938CE0, 0x81F80FE3, 0x55326B08, 0xA759E80B, 0xB4091BFF, 0x466298FC,
00184 0x1871A4D8, 0xEA1A27DB, 0xF94AD42F, 0x0B21572C, 0xDFEB33C7, 0x2D80B0C4, 0x3ED04330, 0xCCBAC033,
00185 0xA24BB5A6, 0x502036A5, 0x4370C551, 0xB11B4652, 0x65D122B9, 0x97BAA1BA, 0x84EA524E, 0x7681D14D,
00186 0x2892ED69, 0xDADF96E6A, 0xC9A99D9E, 0x3BC21E9D, 0xEF087A76, 0x1D63F975, 0x0E330A81, 0xFC588982,
00187 0xB21572C9, 0x407EF1CA, 0x532E023E, 0xA145813D, 0x758FE5D6, 0x87E466D5, 0x94B49521, 0x66DF1622,
00188 0x38CC2AC6, 0xCA7A905, 0xD9F75AF1, 0x2B9CD9F2, 0xFF56BD19, 0x0D3D3E1A, 0x1E6DCDEE, 0xEC064EED,
00189 0xC38D26C4, 0x31E6A5C7, 0x22B65633, 0xD0DDD530, 0x0417B1DB, 0xF67C32D8, 0xE52CC12C, 0x1747422F,
00190 0x49547E0B, 0xBB3FFD08, 0xA86F0EFC, 0x5A048DFF, 0x8ECCE914, 0x7CA56A17, 0x6FF599E3, 0x9D9E1AE0,
00191 0xD3D3E1AB, 0x21B862A8, 0x32E8915C, 0xC083125F, 0x144976B4, 0xE622F5B7, 0xF5720643, 0x07198540,
```

```
00192 0x590AB964, 0xAB613A67, 0xB831C993, 0x4A5A4A90, 0x9E902E7B, 0x6CFBAD78, 0x7FAB5E8C, 0x8DC0DD8F,
00193 0xE330A81A, 0x115B2B19, 0x020BD8ED, 0xF0605BEE, 0x24AA3F05, 0xD6C1BC06, 0xC5914FF2, 0x37FACCF1,
00194 0x69E9F0D5, 0x9B8273D6, 0x88D28022, 0x7AB90321, 0xAE7367CA, 0x5C18E4C9, 0x4F48173D, 0xBD23943E,
00195 0xF36E6F75, 0x0105EC76, 0x12551F82, 0xE03E9C81, 0x34F4F86A, 0xC69F7B69, 0xD5CF889D, 0x27A40B9E,
00196 0x79B737BA, 0x8BDCB4B9, 0x988C474D, 0x6AE7C44E, 0xBE2DA0A5, 0x4C4623A6, 0x5F16D052, 0xAD7D5351,
00197 };
```

```
00001 /**
00002  * @file rx_crc_sw.c
00003  * @brief Software CRC Implementations for All Supported Polynomials
00004  *
00005  * @details
00006  * Portable software implementations for all five CRC polynomials supported by
00007  * the STAR CRC library. No hardware dependencies; compiles and runs on any
00008  * host or embedded target.
00009  *
00010  * ## Algorithms
00011  *
00012  * | Polynomial | Method | Table size | Notes |
00013  * |-----|-----|-----|-----|
00014  * | CRC-32/IEEE | Table | 1024 B | Reflected poly 0xEDB88320 |
00015  * | CRC-32C | Table | 1024 B | Reflected poly 0x82F63B78 |
00016  * | CRC-16/IBM | Bitwise | 0 | Reflected poly 0xA001 |
00017  * | CRC-CCITT | Bitwise | 0 | Reflected poly 0x8408 (Kermit) |
00018  * | CRC-8/Maxim | Bitwise | 0 | Reflected poly 0x8C (Dallas) |
00019  *
00020  * ## Algorithm Dispatch
00021  *
00022  * @startuml
00023  * start
00024  * :internal_crc_sw_compute(poly, data, len);
00025  * switch (poly)
00026  * case (k_rx_crc_poly_crc8)
00027  * :internal_crc8_maxim_compute()\nbitwise, reflected poly 0x8C\ninit=0x00, no final XOR;
00028  * case (k_rx_crc_poly_crc16)
00029  * :internal_crc16_ibm_compute()\nbitwise, reflected poly 0xA001\ninit=0x0000, no final XOR;
00030  * case (k_rx_crc_poly_crc_ccitt)
00031  * :internal_crc_ccitt_compute()\nbitwise, reflected poly 0x8408\ninit=0x0000, no final XOR;
00032  * case (k_rx_crc_poly_crc32c)
00033  * :internal_crc32c_compute()\ntable 0x82F63B78 (256 entries)\ninit=0xFFFFFFFF, XOR=0xFFFFFFFF;
00034  * case (k_rx_crc_poly_crc32 / default)
00035  * :internal_crc32_compute()\ntable 0xEDB88320 (256 entries)\ninit=0xFFFFFFFF, XOR=0xFFFFFFFF;
00036  * endswitch
00037  * :return uint32_t CRC result;
00038  * note right: CRC-8 uses low 8 bits\nCRC-16 uses low 16 bits\nCRC-32 uses all 32 bits
00039  * stop
00040  * @enduml
00041  *
00042  * @see rx_crc.h Public API
00043  * @see rx_crc_internal.h Internal declarations
00044  *
00045  * @author Locked, Inc.
00046  * @date 2026-03-05
00047  * @copyright Copyright (c) 2026 Locked Inc.
00048  * SPDX-License-Identifier: MIT
00049  */
00050
00051 #include <stddef.h>
00052
00053 #include "rx_crc_internal.h"
00054
00055 /**
=====
00056  * Private Constants
00057  *
=====
00058  */
00059
00060 /**
00061  * @brief CRC-32 SW constants
00062  */
00063 typedef enum : uint32_t {
00064     k_crc32_table_size = 256U, /**< Lookup table entry count */
00065     k_crc_byte_mask = 0xFFU, /**< Byte mask for table index */
00066     k_crc32_init_value = 0xFFFFFFFFU, /**< IEEE 802.3 initial value */
00067     k_crc32_final_xor = 0xFFFFFFFFU, /**< IEEE 802.3 final XOR */
00068     k_crc_byte_shift = 8U, /**< Right-shift per byte iteration */
00069 } rx_crc_sw_u32_constants_t;
00070
00071 /**
00072  * @brief CRC-16 and CRC-8 constants
00073  */
```

```

00074 typedef enum : uint8_t {
00075     k_bits_per_byte = 8U, /**< Bits in one byte */
00076     k_crc_lsb_mask = 1U, /**< Mask for LSB */
00077     k_crc_shift_bit = 1U, /**< Shift by 1 */
00078 } rx_crc_sw_bit_constants_t;
00079
00080 /**
00081  * @brief CRC-16/IBM reflected polynomial (0x8005 reflected = 0xA001)
00082  */
00083 typedef enum : uint16_t {
00084     k_crc16_ibm_poly = 0xA001U, /**< IBM poly, reflected form */
00085     k_crc16_init = 0x0000U, /**< Initial value */
00086     k_crc16_final_xor = 0x0000U, /**< Final XOR value (0 = no XOR applied) */
00087 } rx_crc16_constants_t;
00088
00089 /**
00090  * @brief CRC-CCITT (Kermit) reflected polynomial (0x1021 reflected = 0x8408)
00091  */
00092 typedef enum : uint16_t {
00093     k_crc_ccitt_poly = 0x8408U, /**< CCITT poly, reflected (Kermit) */
00094     k_crc_ccitt_init = 0x0000U, /**< Initial value for Kermit variant */
00095 } rx_crc_ccitt_constants_t;
00096
00097 /**
00098  * @brief CRC-8/Maxim (Dallas) reflected polynomial (0x31 reflected = 0x8C)
00099  */
00100 typedef enum : uint8_t {
00101     k_crc8_maxim_poly = 0x8CU, /**< Maxim/Dallas poly, reflected */
00102     k_crc8_init = 0x00U, /**< Initial value */
00103 } rx_crc8_maxim_constants_t;
00104
00105 /*
=====
00106  * CRC-32 / IEEE 802.3 Lookup Table
00107  *
=====
00108  */
00109
00110 /**
00111  * @brief Precomputed CRC-32 lookup table (reflected polynomial 0xEDB88320)
00112  *
00113  * @details Each entry is the CRC-32 for a single byte 0-255.
00114  * Compatible with Go's crc32.ChecksumIEEE(), Python's zlib.crc32(),
00115  * and Ethernet FCS.
00116  */
00117 static const uint32_t s_crc32_table[k_crc32_table_size] = {
00118     0x00000000, 0x77073096, 0xEE0E612C, 0x990951BA, 0x076DC419, 0x706AF48F, 0xE963A535, 0x9E6495A3,
00119     0x0EDB8832, 0x79DCB8A4, 0xE0D5E91E, 0x97D2D988, 0x09B64C2B, 0x7EB17CBD, 0xE7B82D07, 0x90BF1D91,
00120     0x1DB71064, 0x6AB020F2, 0xF3B97148, 0x84BE41DE, 0x1ADAD47D, 0x6DDDE4EB, 0xF4D4B551, 0x83D385C7,
00121     0x136C9856, 0x646BA8C0, 0xFD62F97A, 0x8A65C9EC, 0x14015C4F, 0x63066CD9, 0xFA0F3D63, 0x8D080DF5,
00122     0x3B6E20C8, 0x4C69105E, 0xD56041E4, 0xA2677172, 0x3C03E4D1, 0x4B04D447, 0xD20D85FD, 0xA50AB56B,
00123     0x35B5A8FA, 0x42B2986C, 0xDBBBC9D6, 0xACBCF940, 0x32D86CE3, 0x45DF5C75, 0xDCD60DCF, 0xABD13D59,
00124     0x26D930AC, 0x51DE003A, 0xC8D77518, 0xBFDD6116, 0x21B4F4B5, 0x56B3C423, 0xCFBA9599, 0xB8BDA50F,
00125     0x2802B89E, 0x5F058808, 0xC60CD9B2, 0xB10BE924, 0x2F6F7C87, 0x58684C11, 0xC1611DAB, 0xB6662D3D,
00126     0x76DC4190, 0x01D7B7106, 0x98D220BC, 0xEFD5102A, 0x71B18589, 0x06B6B51F, 0x9FBBF4A5, 0xE8B8D433,
00127     0x7807C9A2, 0x0F00F934, 0x9609A88E, 0xE10E9818, 0x7F6A0DBB, 0x086D3D2D, 0x91646C97, 0xE6635C01,
00128     0x6B6B51F4, 0x1C6C6162, 0x856530D8, 0xF262004E, 0x6C0695ED, 0x1B01A57B, 0x8208F4C1, 0xF50FC457,
00129     0x65B0D9C6, 0x12B7E950, 0x8BBEB8EA, 0xFCB9887C, 0x62DD1DDF, 0x15DA2D49, 0x8CD37CF3, 0xFBD44C65,
00130     0x4DB26158, 0x3AB551CE, 0xA3BC0074, 0xD4BB30E2, 0x4ADFA541, 0x3DD895D7, 0xA4D1C46D, 0xD3D6F4FB,
00131     0x1314369E96A, 0x346ED99FC, 0xAD678846, 0xDA60B8D0, 0x44042D73, 0x33031DE5, 0xAA0A4C5F, 0xDD0D7CC9,
00132     0x5005713C, 0x270241AA, 0xBE0B1010, 0xC90C2086, 0x5768B525, 0x206F85B3, 0xB966D409, 0xCE61E49F,
00133     0x5EDEF90E, 0x29D9C998, 0xB0D09822, 0xC7D7A8B4, 0x59B33D17, 0x2EB40D81, 0xB7BD5C3B, 0xC0BA6CAD,
00134     0xEDB88320, 0x9ABFB3B6, 0x03B6E20C, 0x74B1D29A, 0xEAD54739, 0x9DD277AF, 0x04DB2615, 0x73DC1683,
00135     0xE3630B12, 0x94643B84, 0x0D6D6A3E, 0x7A6A5AA8, 0xE40ECF0B, 0x9309FF9D, 0x0A00AE27, 0x7D079EB1,
00136     0xF00F9344, 0x8708A3D2, 0x1E01F268, 0x6906C2FE, 0xF762575D, 0x806567CB, 0x196C3671, 0x6E6B06E7,
00137     0xFED41B76, 0x89D32BE0, 0x10DA7A5A, 0x67DD4ACC, 0xF9B9DF6F, 0x8EBEEFF9, 0x17B7BE43, 0x60B08ED5,
00138     0xD6D6A3E8, 0xA1D1937E, 0x38D8C2C4, 0x4FDFF252, 0xD1BB67F1, 0xA6BC5767, 0x3FB506DD, 0x48B2364B,
00139     0xD80D2BDA, 0xAF0A1B4C, 0x36034AF6, 0x41047A60, 0xDF60EFC3, 0xA867DF55, 0x316E8EEF, 0x6469BE79,
00140     0xCB61B38C, 0xBC66831A, 0x256FD2A0, 0x5268E236, 0xCC0C7795, 0xBB0B4703, 0x220216B9, 0x5505262F,
00141     0xC5BA3BBE, 0xB2BD0B28, 0x2BB45A92, 0x5CB36A04, 0xC2D7FFA7, 0xB5D0CF31, 0x2CD99E8B, 0x5BDEAE1D,
00142     0x9B64C2B0, 0xEC63F226, 0x756AA39C, 0x026D930A, 0x9C0906A9, 0xEB0E363F, 0x72076785, 0x05005713,
00143     0x95BF4A82, 0xE2B87A14, 0x7BB12BAE, 0x0CB61B38, 0x92D28E9B, 0xE5D5BE0D, 0x7CDCEFB7, 0x0BDBDF21,
00144     0x86D3D2D4, 0xF1D4E242, 0x68DDB3F8, 0x1FDA836E, 0x81BE16CD, 0xF6B9265B, 0x6FB077E1, 0x18B74777,
00145     0x88085AE6, 0xFF0F6A70, 0x66063BCA, 0x11010B5C, 0x8F659EFF, 0xF862AE69, 0x616BF6D3, 0x166CCF45,
00146     0xA00AE278, 0xD70DD2EE, 0x4E048354, 0x3903B3C2, 0xA7672661, 0xD06016F7, 0x4969474D, 0x3E6E77DB,
00147     0xAED16A4A, 0xD9D65ADC, 0x40DF0B66, 0x37D83BF0, 0xA9BCAE53, 0xDEBB9EC5, 0x47B2CF7F, 0x30B5FFE9,
00148     0xBDBDF21C, 0xCABAC28A, 0x53B39330, 0x24B4A3A6, 0xBAD03605, 0xCDD70693, 0x54DE5729, 0x23D967BF,
00149     0xB3667A2E, 0xC4614AB8, 0x5D681B02, 0x2A6F2B94, 0xB40BBE37, 0xC30C8EA1, 0x5A05DF1B, 0x2D00EFD8,
00150 };
00151
00152 /*
=====
00153  * CRC-32C / Castagnoli Lookup Table
00154  *
=====
00155  */
00156

```

```

00157 /**
00158  * @brief Precomputed CRC-32C lookup table (reflected polynomial 0x82F63B78)
00159  *
00160  * @details Each entry is the CRC-32C for a single byte 0-255.
00161  * Polynomial: 0x1EDC6F41 (normal form), 0x82F63B78 (reflected).
00162  * Used by iSCSI, SCTP, BTRFS, and modern network stacks.
00163  */
00164 static const uint32_t s_crc32c_table[k_crc32_table_size] = {
00165     0x00000000, 0xF26B8303, 0xE13B70F7, 0x1350F3F4, 0xC79A971F, 0x35F1141C, 0x26A1E7E8, 0xD4CA64EB,
00166     0x8AD958CF, 0x78B2DBCC, 0x6BE22838, 0x9989AB3B, 0x4D43CFD0, 0xBF284CD3, 0xAC78BF27, 0x5E133C24,
00167     0x105EC76F, 0xE235446C, 0xF165B798, 0x030E349B, 0xD7C45070, 0x25AFD373, 0x36FF2087, 0xC494A384,
00168     0x9A879FA0, 0x68EC1CA3, 0x7BBCF57, 0x89D76C54, 0x5D1D08BF, 0xAF768BBC, 0xBC267848, 0xE4DDFB4B,
00169     0x20BD8EDE, 0xD2D60DDD, 0xC186FE29, 0x33ED7D2A, 0xE72719C1, 0x154C9AC2, 0x061C6936, 0xF477EA35,
00170     0xAA64D611, 0x580F5512, 0x4B5FA6E6, 0xB93425E5, 0x6DFE410E, 0x9F95C20D, 0x8CC531F9, 0x7EAE2FA,
00171     0x30E349B1, 0xC288CAB2, 0xD1D83946, 0x23B3BA45, 0xF779DEAE, 0x05125DAD, 0x1642AE59, 0xE4292D5A,
00172     0xBA3A117E, 0x4851927D, 0x5B016189, 0xA96AE28A, 0x7DA08661, 0x8FCB0562, 0x9C9BF696, 0x6EF07595,
00173     0x417B1DBC, 0xB3109EBF, 0xA0406D4B, 0x522BEE48, 0x86E18AA3, 0x748A09A0, 0x67DAFA54, 0x95B17957,
00174     0xCBA24573, 0x39C9C670, 0x2A993584, 0xD8F2B687, 0x0C38D26C, 0xFE53516F, 0xED03A29B, 0x1F682198,
00175     0x5125DAD3, 0xA34E59D0, 0xB01EAA24, 0x42752927, 0x96BF4DCC, 0x64D4CECF, 0x77843D3B, 0x85EFBE38,
00176     0xDBFC821C, 0x2997011F, 0x3AC7F2EB, 0xC8AC71E8, 0x1C661503, 0xEE0D9600, 0xFD5D65F4, 0xF36E6F7,
00177     0x61C69362, 0x93AD1061, 0x80FDE395, 0x72966096, 0xA65C047D, 0x5437877E, 0x4767748A, 0xB50CF789,
00178     0xEB1FCBAD, 0x197448AE, 0x0A24BB5A, 0xF84F3859, 0x2C855CB2, 0xDEEDFB1, 0xCDBE2C45, 0x3FD5AF46,
00179     0x7198540D, 0x83F3D70E, 0x90A324FA, 0x62C8A7F9, 0xB602C312, 0x44694011, 0x5739B3E5, 0xA55230E6,
00180     0xFB410CC2, 0x092A8FC1, 0x1A7A7C35, 0xE811FF36, 0x3CDB9BDD, 0xCEB018DE, 0x0DDE0EB2A, 0x2F8B6829,
00181     0x82F63B78, 0x709DB87B, 0x63CD4B8F, 0x91A6C88C, 0x456CAC67, 0xB7072F64, 0xA457DC90, 0x563C5F93,
00182     0x082F63B7, 0xFA44E0B4, 0xE9141340, 0x1B7F9043, 0xCFB5F4A8, 0x3DDE77AB, 0x2E8E845F, 0xDCE5075C,
00183     0x92A8FC17, 0x60C37F14, 0x73938CE0, 0x81F80FE3, 0x55326B08, 0xA759E80B, 0xB4091BFF, 0x466298FC,
00184     0x1871A4D8, 0xEA1A27DB, 0xF94AD42F, 0x0B21572C, 0xDFEB33C7, 0x2D80B0C4, 0x3ED04330, 0xCCBBC033,
00185     0xA24BB5A6, 0x502036A5, 0x4370C551, 0xB11B4652, 0x65D122B9, 0x97BAA1BA, 0x84EA524E, 0x7681D14D,
00186     0x2892ED69, 0xDADF96EA, 0xC9A99D9E, 0x3BC21E9D, 0xEF087A76, 0x1D63F975, 0x0E330A81, 0xFC588982,
00187     0xB21572C9, 0x407EF1CA, 0x532E023E, 0xA145813D, 0x758FE5D6, 0x87E466D5, 0x94B49521, 0x66DF1622,
00188     0x38CC2A06, 0xCAA7A905, 0xD9F75AF1, 0x2B9CD9F2, 0xFF56BD19, 0x0D3D3E1A, 0x1E6DCDEE, 0xEC064EED,
00189     0xC38D26C4, 0x31E6A5C7, 0x22B65633, 0xD0DD530, 0x0417B1DB, 0xF67C32D8, 0xE52CC12C, 0x1747422F,
00190     0x49547E0B, 0xBB3FFD08, 0xA86F0EFC, 0x5A048DFF, 0x8ECEEE914, 0x7CA56A17, 0x6FF599E3, 0x9D9E1AE0,
00191     0xD3D3E1AB, 0x21B862A8, 0x32E8915C, 0xC083125F, 0x144976B4, 0xE622F5B7, 0xF5720643, 0x07198540,
00192     0x590AB964, 0xAB613A67, 0xB831C993, 0x4A5A4A90, 0x9E902E7B, 0x6CFBAD78, 0x7FAB5E8C, 0x8DC0DD8F,
00193     0xE330A81A, 0x115B2B19, 0x020BD8ED, 0xF0605BEE, 0x24AA3F05, 0xD6C1BC06, 0xC5914FF2, 0x37FACCF1,
00194     0x69E9F0D5, 0x9B8273D6, 0x88D28022, 0x7AB90321, 0xAE7367CA, 0x5C18E4C9, 0x4F48173D, 0xBD23943E,
00195     0xF36E6F75, 0x0105EC76, 0x12551F82, 0xE03E9C81, 0x34F4F86A, 0xC69F7B69, 0xD5CF889D, 0x27A40B9E,
00196     0x79B737BA, 0x8BDCB4B9, 0x988C474D, 0x6AE7C44E, 0xBE2DA0A5, 0x4C4623A6, 0x5F16D052, 0xAD7D5351,
00197 };
00198
00199 /
=====
00200 * Internal SW Implementations
00201 *
=====
00202 /
00203
00204 /**
00205  * @brief Compute CRC-32/IEEE 802.3 using table lookup
00206  *
00207  * @details
00208  * Initializes accumulator to 0xFFFFFFFF (k_crc32_init_value), processes each
00209  * byte via s_crc32_table lookup, then XORs final result with 0xFFFFFFFF
00210  * (k_crc32_final_xor). Implements CRC-32/ISO-HDLC (reflected poly 0xEDB88320).
00211  *
00212  *
00213  *
00214  * @pre data != NULL
00215  * @pre 0 < len <= k_crc_len_max
00216  * @post Returns finalized CRC with XOR applied; accumulator state not retained
00217  * @post No side effects; pure computation
00218  *
00219  * @since Version 1.0.0
00220  */
00221 static uint32_t internal_crc32_compute(const uint8_t* data, uint32_t len)
00222 {
00223     uint32_t crc = k_crc32_init_value;
00224
00225     for (uint32_t i = 0; i < len; i++) {
00226         crc = s_crc32_table[(uint8_t)(crc ^ data[i])] ^ (crc » k_crc_byte_shift);
00227     }
00228
00229     return crc ^ k_crc32_final_xor;
00230 }
00231
00232 /**
00233  * @brief Compute CRC-32C (Castagnoli) using table lookup
00234  *
00235  * @details
00236  * Same structure as internal_crc32_compute() but uses s_crc32c_table
00237  * (reflected poly 0x82F63B78, iSCSI / Castagnoli polynomial).
00238  *
00239  *
00240  *
00241  * @pre data != NULL

```

```

00242 * @pre 0 < len <= k_crc_len_max
00243 * @post Returns finalized CRC-32C with XOR applied; accumulator not retained
00244 * @post No side effects; pure computation
00245 *
00246 * @since Version 1.0.0
00247 */
00248 static uint32_t internal_crc32c_compute(const uint8_t* data, uint32_t len)
00249 {
00250     uint32_t crc = k_crc32_init_value;
00251     for (uint32_t i = 0U; i < len; i++) {
00252         crc = s_crc32c_table[(uint8_t)(crc ^ data[i])] ^ (crc » k_crc_byte_shift);
00253     }
00254     return crc ^ k_crc32_final_xor;
00255 }
00256
00257 /**
00258 * @brief Compute CRC-16/IBM (ARC) bitwise
00259 *
00260 * @details
00261 * Initializes accumulator to 0 (k_crc16_init), processes each byte with
00262 * reflected polynomial 0x8005 (k_crc16_ibm_poly = 0xA001 reflected).
00263 * No final XOR (k_crc16_final_xor = 0). Implements CRC-16/ARC.
00264 *
00265 * @pre data != NULL
00266 * @pre 0 < len <= k_crc_len_max
00267 * @post Returns 16-bit CRC with k_crc16_final_xor applied (currently 0x0000, i.e., no-op)
00268 * @post No side effects; pure computation
00269 *
00270 * @since Version 1.0.0
00271 */
00272 static uint16_t internal_crc16_ibm_compute(const uint8_t* data, uint32_t len)
00273 {
00274     uint16_t crc = k_crc16_init;
00275     for (uint32_t i = 0U; i < len; i++) {
00276         crc ^= (uint16_t)data[i];
00277         for (uint8_t b = 0U; b < k_bits_per_byte; b++) {
00278             if ((crc & (uint16_t)k_crc_lsb_mask) != 0U) {
00279                 crc = (uint16_t)((crc » k_crc_shift bit) ^ k_crc16_ibm_poly);
00280             } else {
00281                 crc »= k_crc_shift bit;
00282             }
00283         }
00284     }
00285     return (uint16_t)(crc ^ k_crc16_final_xor);
00286 }
00287
00288 /**
00289 * @brief Compute CRC-CCITT/Kermit (reflected 0x1021) bitwise
00290 *
00291 * @details
00292 * Initializes accumulator to 0 (k_crc_ccitt_init), processes each byte
00293 * with reflected polynomial 0x1021 (k_crc_ccitt_poly = 0x8408 reflected).
00294 * No final XOR. Implements CRC-16/KERMIT.
00295 *
00296 * @pre data != NULL
00297 * @pre 0 < len <= k_crc_len_max
00298 * @post Returns 16-bit CRC; no final XOR applied
00299 * @post No side effects; pure computation
00300 *
00301 * @since Version 1.0.0
00302 */
00303 static uint16_t internal_crc_ccitt_compute(const uint8_t* data, uint32_t len)
00304 {
00305     uint16_t crc = k_crc_ccitt_init;
00306     for (uint32_t i = 0U; i < len; i++) {
00307         crc ^= (uint16_t)data[i];
00308         for (uint8_t b = 0U; b < k_bits_per_byte; b++) {
00309             if ((crc & (uint16_t)k_crc_lsb_mask) != 0U) {
00310                 crc = (uint16_t)((crc » k_crc_shift bit) ^ k_crc_ccitt_poly);
00311             } else {
00312                 crc »= k_crc_shift bit;
00313             }
00314         }
00315     }
00316     return crc;
00317 }
00318
00319
00320
00321
00322
00323
00324
00325
00326
00327
00328

```

```

00329 /**
00330  * @brief Compute CRC-8/Maxim (Dallas) bitwise
00331  *
00332  * @details
00333  * Initializes accumulator to 0 (k_crc8_init), processes each byte with
00334  * reflected polynomial 0x31 (k_crc8_maxim_poly = 0x8C reflected).
00335  * No final XOR. Implements CRC-8/MAXIM-DOW (used by 1-Wire devices).
00336  *
00337  *
00338  *
00339  * @pre data != NULL
00340  * @pre 0 < len <= k_crc_len_max
00341  * @post Returns 8-bit CRC; no final XOR applied
00342  * @post No side effects; pure computation
00343  *
00344  * @since Version 1.0.0
00345  */
00346 static uint8_t internal_crc8_maxim_compute(const uint8_t* data, uint32_t len)
00347 {
00348     uint8_t crc = k_crc8_init;
00349
00350     for (uint32_t i = 0U; i < len; i++) {
00351         crc ^= data[i];
00352         for (uint8_t b = 0U; b < k_bits_per_byte; b++) {
00353             if ((crc & k_crc_lsb_mask) != 0U) {
00354                 crc = (uint8_t)((crc » k_crc_shift bit) ^ k_crc8_maxim_poly);
00355             } else {
00356                 crc »= k_crc_shift bit;
00357             }
00358         }
00359     }
00360
00361     return crc;
00362 }
00363
00364 /**
=====
00365  * Public Internal API
00366  *
=====
00367  */
00368
00369 /**
00370  * @brief Compute CRC using software for the given polynomial
00371  *
00372  * @details
00373  * Dispatches to the appropriate software CRC implementation based on poly.
00374  * CRC-32 and CRC-32C use 256-entry lookup tables; CRC-16/IBM, CRC-CCITT, and
00375  * CRC-8/Maxim use bitwise algorithms. All implementations use the reflected
00376  * (LSB-first) form of the polynomial. Caller must ensure data != NULL and
00377  * len is in [k_crc_len_min, k_crc_len_max]; these are not re-validated here.
00378  *
00379  *
00380  *
00381  * @pre data != NULL (enforced by caller rx_crc_compute())
00382  * @pre len in [k_crc_len_min, k_crc_len_max] (enforced by caller)
00383  * @post Returns CRC with final XOR applied; no persistent state modified
00384  * @post No side effects (stateless, read-only tables)
00385  *
00386  * @par Activity Diagram:
00387  * @startuml
00388  * start
00389  * :internal_crc_sw_compute(poly, data, len);
00390  * switch (poly)
00391  * case (crc8)
00392  *   :internal_crc8_maxim_compute();
00393  *   note right: bitwise XOR per bit, poly 0x8C
00394  * case (crc16)
00395  *   :internal_crc16_ibm_compute();
00396  *   note right: bitwise XOR per bit, poly 0xA001
00397  * case (crc_ccitt)
00398  *   :internal_crc_ccitt_compute();
00399  *   note right: bitwise XOR per bit, poly 0x8408
00400  * case (crc32c)
00401  *   :internal_crc32c_compute();
00402  *   note right: table lookup, poly 0x82F63B78
00403  * case (crc32 / default)
00404  *   :internal_crc32_compute();
00405  *   note right: table lookup, poly 0xEDB88320
00406  * endswitch
00407  * :return uint32_t result;
00408  * stop
00409  * @enduml
00410  *
00411  * @note Thread-safe; stateless, read-only table access only
00412  * @since Version 1.0.0
00413  */

```

```

00414 uint32_t internal_crc_sw_compute(rx_crc_poly_t poly, const uint8_t* data, uint32_t len)
00415 {
00416     switch (poly) {
00417         case k_rx_crc_poly_crc8:
00418             return (uint32_t)internal_crc8_maxim_compute(data, len);
00419         case k_rx_crc_poly_crc16:
00420             return (uint32_t)internal_crc16_ibm_compute(data, len);
00421         case k_rx_crc_poly_crc_ccitt:
00422             return (uint32_t)internal_crc_ccitt_compute(data, len);
00423         case k_rx_crc_poly_crc32c:
00424             return internal_crc32c_compute(data, len);
00425         case k_rx_crc_poly_crc32:
00426             default:
00427                 return internal_crc32_compute(data, len);
00428     }
00429 }
00430
00431 /**
00432  * @brief Update CRC-32/IEEE in-progress value with additional data
00433  *
00434  * @details
00435  * Continues an IEEE 802.3 CRC-32 from a finalized partial result. Un-finalizes
00436  * the input CRC by XOR with k_crc32_final_xor to recover the internal accumulator
00437  * state, feeds the new data through the s_crc32_table lookup loop, then
00438  * re-finalizes by XOR with k_crc32_final_xor. Used by rx_crc32_update() to
00439  * support incremental (streaming) CRC-32 computation over multiple chunks.
00440  *
00441  * Pass-through case: returns crc unchanged if len exceeds k_crc_len_max.
00442  * Caller (rx_crc32_update) is responsible for filtering null data and zero len
00443  * before invoking this function.
00444  *
00445  *
00446  *
00447  * @pre crc is either 0U (first call) or the return value of a prior
00448  *       internal_crc32_sw_update() / rx_crc32_update() call
00449  * @pre data points to len valid bytes (caller enforces non-NULL)
00450  * @pre len > 0 (caller enforces)
00451  * @post Returns finalized CRC-32 with final XOR applied
00452  * @post No side effects (stateless, read-only table access)
00453  *
00454  * @note Thread-safe; stateless, read-only table access only
00455  * @since Version 1.0.0
00456  */
00457 uint32_t internal_crc32_sw_update(uint32_t crc, const uint8_t* data, uint32_t len)
00458 {
00459     if (len > k_crc_len_max) {
00460         return crc;
00461     }
00462
00463     /* Un-finalize previous CRC to get internal accumulator state */
00464     uint32_t work = crc ^ k_crc32_final_xor;
00465
00466     for (uint32_t i = 0U; i < len; i++) {
00467         work = s_crc32_table[(uint8_t)(work ^ data[i])] ^ (work » k_crc_byte_shift);
00468     }
00469
00470     return work ^ k_crc32_final_xor;
00471 }

```
